Universitatea Tehnică a Moldovei

APLICAȚIE PENTRU VERIFICAREA GRADULUI DE SIMILITUDINE

APPLICATION FOR THE ASSESMENT OF DOCUMENT SIMILARITY

Student: Botnarenco Ecaterina, TI-223

Coordonator: Bîtca Ernest, asistent universitar

Chişinău, 2026

MINISTERUL EDUCAŢIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică şi Microelectronică
Departamentul Ingineria Software și Automatică

Admis la susţinere
Şef departament:
FIODOROV Ion dr., conf.univ.

„___” _____ 2025

APLICAȚIE PENTRU DETECTAREA GRADULUI DE SIMILITUDINE

Proiect de licenţă

Student: _____ **Botnarenco Ecaterina, TI-223**

Coordonator: _____ **Bîtca Ernest, asistent universitar**

Consultant: _____ **Cojocaru Svetlana, asistent universitar**

Chişinău, 2026

DECLARAŢIA STUDENTULUI

Subsemnata *Botnarenco Ecaterina*, declar pe proprie răspundere că lucrarea de faţă este rezultatul muncii mele, pe baza propriilor cercetări şi pe baza informaţiilor obţinute din surse care au fost citate şi indicate, conform normelor etice, în note şi în bibliografie. Declar că lucrarea nu a mai fost prezentată sub această formă la nici o instituţie de învăţământ superior în vederea obţinerii unui grad sau titlu ştiinţific ori didactic.

Botnarenco

REZUMAT

Prezenta lucrare este consacrată analizei, proiectării, realizării și documentării unei aplicații software pentru verificarea gradului de similitudine dintre documente, având drept scop sprijinirea procesului de identificare a conținutului preluat și susținerea integrității academice. Tema abordată este de actualitate în contextul creșterii volumului de informații disponibile în mediul digital și al necesității dezvoltării unor instrumente capabile să detecteze nu doar potrivirile lexicale, ci și asemănările de natură semantică. Lucrarea este structurată logic, urmărind etapele esențiale ale dezvoltării unui sistem informatic, de la fundamentarea teoretică până la prezentarea modului de utilizare a soluției propuse.

Analiza domeniului de studiu prezintă fundamentele teoretice ale temei abordate și evidențiază problema plagiatului în contextul educațional actual. În cadrul acestuia sunt analizate sisteme similare existente, sunt formulate scopul și obiectivele proiectului, iar apoi sunt definite cerințele funcționale și nefuncționale ale aplicației. De asemenea, capitolul include specificațiile tehnice, descrierea interfeței utilizatorului, planul de testare, precum și principalele constrângeri și riscuri asociate dezvoltării sistemului. Acest capitol stabilește baza conceptuală și tehnică necesară pentru etapele ulterioare.

Proiectarea sistemului informatic este dedicată modelării soluției propuse prin metode specifice proiectării orientate pe obiect și prin utilizarea limbajului UML. În acest capitol sunt descrise atât aspectele comportamentale ale sistemului, cât și cele structurale. Sunt prezentate imaginea generală asupra sistemului, fluxurile operaționale, stările de tranzacție, scenariile de utilizare și fluxurile de mesaje dintre componente, iar ulterior sunt analizate structura statică a sistemului, relațiile de dependență dintre module și modelarea mediului de implementare. Prin acest capitol se conturează arhitectura logică și tehnică a aplicației.

Realizarea sistemului informatic tratează etapa de implementare efectivă a aplicației. În cadrul acestuia sunt justificate tehnologiile alese și arhitectura software utilizată, după care sunt prezentate realizarea bazei de date, dezvoltarea componentei backend și realizarea componentei frontend. Capitolul reflectă transformarea modelelor teoretice și a cerințelor definite anterior într-o soluție funcțională, capabilă să preia documente, să proceseze conținutul textual, să determine gradul de similitudine și să genereze rapoarte de analiză. Tot aici este inclusă și etapa de testare și verificare a funcționalităților implementate.

Documentarea produsului realizat este orientat spre componenta practică de utilizare a aplicației. Acesta include ghidul de utilizare și descrierea fluxurilor operaționale, oferind o imagine clară asupra modului în care utilizatorul interacționează cu sistemul și asupra principalelor operații disponibile în cadrul aplicației. Capitolul are rolul de a evidenția valoarea practică a produsului software și de a facilita utilizarea sa în condiții reale.

Lucrarea este completată prin secțiunea de concluzii, bibliografie și anexe, care susțin atât fundamentarea științifică a temei, cât și documentarea tehnică a soluției dezvoltate.

ABSTRACT

This thesis is dedicated to the analysis, design, implementation, and documentation of a software application developed to verify the degree of similarity between documents, aiming to support the detection of appropriated content and uphold academic integrity. Given the increasing volume of digital information and the necessity for tools capable of detecting not only lexical matches but also semantic similarities, the topic addressed is of high contemporary relevance. The thesis is logically structured, following the essential stages of information system development, from theoretical foundation to the presentation of the proposed solution's practical utility.

The Domain Analysis presents the theoretical foundations of the subject and highlights the issue of plagiarism within the current educational landscape. This section analyzes existing similar systems, defines the project's purpose and objectives, and outlines the functional and non-functional requirements of the application. Furthermore, the chapter includes technical specifications, user interface descriptions, the testing plan, and the primary constraints and risks associated with system development, thereby establishing the conceptual and technical baseline for subsequent stages.

The System Design focuses on modeling the proposed solution using object-oriented design methodologies and the Unified Modeling Language (UML). This chapter describes both the behavioral and structural aspects of the system. It presents a high-level overview of the system, operational workflows, state transitions, use case scenarios, and message flows between components. Subsequently, it analyzes the system's static structure, modular dependencies, and the deployment environment modeling. This chapter delineates the logical and technical architecture of the application.

The System Implementation addresses the actual development phase of the application. It justifies the selection of technologies and the software architecture employed, followed by the development of the database, the backend component, and the realization of the frontend interface. This chapter reflects the transformation of theoretical models and predefined requirements into a functional solution capable of ingesting documents, processing textual content, determining similarity degrees, and generating analytical reports. The testing and verification phase of the implemented functionalities is also included here.

The Product Documentation is oriented toward the practical application of the software. It includes the user manual and descriptions of operational flows, providing a clear overview of user-system interaction and the primary operations available within the application. This chapter serves to highlight the practical value of the software product and to facilitate its deployment in real-world conditions.

The thesis concludes with a summary of findings, a bibliography, and appendices, which provide both the scientific foundation of the research and the technical documentation of the developed solution.

CUPRINS

REZUMAT 13
ABSTRACT 14
LISTA DE ABREVIERI ȘI DEFINIȚII 11
INTRODUCERE 14
1 ANALIZA DOMENIULUI DE STUDIU 15
1.1 Sisteme similare cu proiectul realizat 16
1.2 Scopul, obiectivele și cerințele sistemului 19
1.3 Cerințe funcționale 20
1.4 Cerințe nefuncționale 21
1.5 Specificațiile tehnice 22
1.6 Interfața utilizatorului 24
1.7 Planul de testare 26
1.8 Constrângeri și riscuri 27
2 PROIECTAREA SISTEMULUI INFORMATIC 29
2.1 Descrierea comportamentală a sistemului 30
2.1.1 Imaginea generală asupra sistemului 31
2.1.2 Modelarea vizuală a fluxurilor 33
2.1.3 Stările de tranzacție a sistemului 35
2.1.4 Descrierea scenariilor de utilizare a aplicației 37
2.1.5 Fluxurile de mesaje și legăturile dintre componentele sistemului 38
2.2 Descrierea structurală a sistemului 40
2.2.1 Descrierea structurii statice a sistemului 41
2.2.2 Relațiile de dependență între componentele sistemului 42
2.2.3 Modelarea echipamentelor mediului de implementare 44
3 REALIZAREA SISTEMULUI INFORMATIC 46
3.1 Alegerea tehnologiilor și arhitecturii aplicației 47
3.2 Realizarea sistemului informatic 51
3.2.1 Proiectarea și crearea bazei de date 52
3.2.2 Realizarea componentei backend 53
3.2.3 Realizarea componentei frontend 59
1.3 Testarea și verificarea funcționalităților 65
4 DOCUMENTAREA PRODUSULUI REALIZAT 69
4.1 Ghidul de utilizare a aplicației și fluxurile operaționale 70
CONCLUZII 74

BIBLIOGRAFIE 75
ANEXA B 79
ANEXA C 80
ANEXA D 87

LISTA DE ABREVIERI ȘI DEFINIȚII

IA – Inteligența Artificială;

OOD – Object Oriented Design: Design orientat pe obiect;

UML – Unified Modeling Language: Limbaj de modelare unificat;

Copy-Paste – Copiere-Lipire

NLP – Natural Language Processing: Limbajul de procesare natural;

ER – Entity Relationship: Entitățile relaționale;

Mock-up - reprezentare vizuală detaliată, statică, a unui produs final;

Upload – procesul de încărcare a fișierelor sau datelor de pe un dispozitiv local către un server;

Cloud – model de furnizare a resurselor informatice (servere, stocare, baze de date, aplicații) prin intermediul internetului, care permite acces flexibil, scalabilitate și reducerea costurilor de infrastructură locală;

Frontend – partea aplicației software care interacționează direct cu utilizatorul final, incluzând interfața grafică, elementele vizuale și logica de prezentare;

Backend – partea aplicației responsabilă de logica de business, procesarea datelor, comunicarea cu baza de date și gestionarea securității, care nu este vizibilă utilizatorului final;

Lowercase – proces de transformare a tuturor caracterelor dintr-un text în litere mici, utilizat pentru uniformizarea textului înainte de analiză;

Stopwords – cuvinte frecvente dintr-o limbă (de exemplu: „și”, „sau”, „este”) care nu aduc valoare semantică semnificativă și sunt eliminate în procesarea textului;

Browser – aplicație software utilizată pentru accesarea și vizualizarea paginilor web și a aplicațiilor web;

Back-up – proces de creare și păstrare a unor copii de siguranță ale datelor, utilizat pentru prevenirea pierderii informațiilor în cazul unor erori, defecțiuni hardware sau incidente de securitate;

TF-IDF – (Term Frequency – Inverse Document Frequency) metodă statistică utilizată pentru a evalua importanța unui termen într-un document, raportată la un set de documente;

REST – (Representational State Transfer) stil arhitectural utilizat pentru proiectarea serviciilor web, bazat pe protocoale standard și operații HTTP;

ASP.NET – (Active Server Pages Network Enabled Technologies) platformă de dezvoltare creată de Microsoft pentru realizarea aplicațiilor web și a serviciilor backend;

SSL – (Secure Sockets Layer) protocol de securitate utilizat pentru criptarea comunicațiilor între client și server;

TLS – (Transport Layer Security) versiune modernă și mai sigură a protocolului SSL, utilizată pentru protejarea transferului de date;

GDPR – (General Data Protection Regulation) regulament al Uniunii Europene privind protecția datelor cu caracter personal;

UI – (User Interface) interfața prin care utilizatorul interacționează cu aplicația;

UX – (User Experience) experiența generală a utilizatorului în interacțiunea cu aplicația, incluzând ușurința de utilizare și satisfacția;

Feedback – opinii sau recenzii oferite de utilizatori în urma utilizării aplicației;

Models – Modele;

Data – Date;

Device – Dispozitiv.

Clean Architecture – O arhitectură de software care promovează separarea clară a responsabilităților între diferitele componente ale aplicației (frontend, backend, bază de date) pentru a facilita întreținerea și extinderea sistemului;

DOM – (Document Object Model) O reprezentare structurală a unui document HTML sau XML, care permite manipularea acestuia prin scripturi;

WebSockets – Un protocol de comunicație care oferă o conexiune persistentă între client și server pentru a permite schimbul de date în timp real;

API – (Application Programming Interface) Un set de funcții și metode care permit interacțiunea dintre diverse componente software, cum ar fi frontend-ul și backend-ul;

Open-source – Software al cărui cod sursă este disponibil publicului pentru a fi utilizat, modificat și distribuit liber;

EF Core – (Entity Framework Core) Un framework ORM (Object-Relational Mapping) care permite dezvoltatorilor să interacționeze cu bazele de date folosind obiecte C# în loc de SQL;

Layer – Un strat sau nivel într-o arhitectură de software, care poate include frontend, backend sau alte componente;

DbContext – O clasă care reprezintă conexiunea dintre aplicație și baza de date, gestionând interacțiunile cu datele;

Change Tracker – O componentă a Entity Framework Core care urmărește modificările aduse entităților, pentru a le salva în baza de date;

Query Pipeline – Un proces prin care interogările sunt procesate și trimise către baza de date;

Database Connection – O conexiune între aplicație și baza de date, esențială pentru accesarea datelor;

Dependency Injection – Un principiu de design care ajută la crearea de instanțe ale claselor și la injectarea acestora în locuri unde sunt necesare, pentru a decupla componentele aplicației;

SQL – (Structured Query Language) Limbajul standard utilizat pentru a interacționa cu bazele de date relaționale;

SSMS – Un instrument de gestionare a bazelor de date SQL Server, folosit pentru administrarea și executarea interogărilor SQL;

Database Providers – Component software care facilitează conexiunea între o aplicație și un sistem de gestionare a bazelor de date;

HTTP – Protocolul folosit pentru transferul de date între client și server pe web;

Controllers – Componente din cadrul aplicației care gestionează cererile HTTP și trimit răspunsuri corespunzătoare;

Down – Acțiunea de a anula sau a reveni la o stare anterioară a unui proces sau modificări (ex: rollback în baza de date);

ID – (Identifier) Un identificator unic pentru entități sau obiecte;

Hash – Un algoritm care transformă datele într-o valoare fixă, utilizată pentru verificarea integrității acestora;

Services – Componente care furnizează funcționalități specifice aplicației, cum ar fi procesarea datelor sau interacțiunea cu baze de date;

POST – tip de metodă HTTP utilizată în comunicația client-server pentru a trimite date către server, de obicei pentru a crea sau actualiza o resursă pe server.

INTRODUCERE

Dezvoltarea unei aplicații software presupune, în mod necesar, parcurgerea unor etape fundamentale de analiză, proiectare, implementare și testare, fiecare având un rol important în obținerea unei soluții coerente și bine structurate. Analiza domeniului de studiu reprezintă punctul de pornire al acestui proces, deoarece permite înțelegerea mediului în care sistemul va funcționa, a cerințelor utilizatorilor, a constrângerilor tehnice și organizaționale, precum și a obiectivelor urmărite. În cadrul acestei etape sunt identificate procesele existente, actorii implicați și relațiile dintre componentele sistemului, fiind create premisele pentru definirea unei arhitecturi clare și pentru reducerea riscurilor de proiectare.

Un rol important în această etapă îl are și studierea soluțiilor software existente, care oferă o imagine asupra modului în care sunt abordate funcționalități similare, asupra avantajelor pe care acestea le oferă, dar și asupra limitărilor lor. Analiza comparativă a sistemelor consacrate contribuie la evidențierea principiilor de funcționare, a metodelor utilizate pentru procesarea și prezentarea rezultatelor, precum și la identificarea aspectelor care pot fi îmbunătățite în cadrul noii soluții propuse. În lipsa unei asemenea fundamentări, etapele ulterioare ale dezvoltării pot fi afectate de neconcordanțe între cerințe și implementare sau de apariția unor costuri suplimentare.

În cadrul metodologiilor inginerești de dezvoltare software, analiza și proiectarea preced implementarea propriu-zisă, având drept scop realizarea unor modele clare, logice și coerente ale sistemului. Utilizarea limbajului unificat de modelare facilitează descrierea structurii și comportamentului aplicației prin diagrame specifice, precum diagrame de cazuri de utilizare, de clase, de componente, de activitate sau de secvență. Aceste modele servesc atât ca documentație tehnică, cât și ca suport pentru comunicarea între părțile implicate și pentru validarea soluției înainte de etapa de implementare.

Realizarea unei asemenea aplicații implică nu doar proiectarea bazei de date și dezvoltarea componentelor backend și frontend, ci și integrarea mecanismelor de procesare textuală, comparare a documentelor și generare a rapoartelor de analiză. Totodată, este necesară asigurarea unei structuri arhitecturale care să permită separarea responsabilităților între module, extinderea ulterioară a funcționalităților și adaptarea aplicației în funcție de cerințele utilizatorilor. O astfel de abordare contribuie la obținerea unei soluții software robuste, scalabile și adecvate contextului pentru care a fost concepută.

Prin urmare, lucrarea urmărește fundamentarea teoretică și practică a dezvoltării unei aplicații pentru detectarea nivelului de similitudine dintre documente, punând accent atât pe analiza domeniului și modelarea sistemului, cât și pe implementarea și evaluarea funcționalităților propuse. În același timp, prezentarea modului de utilizare a aplicației și a fluxurilor sale operaționale contribuie la evidențierea valorii practice a soluției dezvoltate și a utilității acesteia în susținerea unui cadru educațional bazat pe corectitudine, transparență și responsabilitate academică.

1 ANALIZA DOMENIULUI DE STUDIU

Începutul anilor 2000 a fost remarcat de o evoluție considerabilă atât în domeniul științific cât și în cel tehnologic. Tendința de automatizare a oricărui proces manual a dus la creșterea nivelului de trai a majoritatea populației. Însă odată cu această creștere au apărut și diverse provocări.

În paralel cu apariția provocărilor din domeniul economic și social, au apărut și provocări în domeniul educațional. Informația, care altădată putea fi găsită doar în cărți și materiale special dedicate învățăturii și cercetării, acum a devenit ceva nu atât de greu de găsit. La doar un click distanță, oricine poate avea acces la orice. Cărți, enciclopedii, materiale didactice, articole științifice, toate acestea pot fi ușor găsite în mai puțin de un minut. Dorința de a utiliza vechile metode pentru a căpăta informația necesară deja este din ce în ce mai mică.

Această accesibilitate sporită a informației a dus cu sine nu doar beneficii majore în procesul educațional, ci și o serie de riscuri. Una dintre cele mai des întâlnite provocări cu care se confruntă instituțiile educaționale până și în ziua de azi este plagiatul, o practică ce a evoluat odată cu dezvoltarea accesului la resursele online. Universitatea Oxford definește plagiatul ca fiind: „Prezentarea unei lucrări sau a unei idei dintr-o altă sursă ca fiind proprii, cu sau fără consimțământul autorului original, prin încorporarea lor în lucrarea proprie fără a le recunoaște pe deplin. Toate materialele publicate și nepublicate, fie în manuscris, fie în formă tipărită sau electronică, sunt acoperite de această definiție, la fel ca și utilizarea materialului generat integral sau parțial prin utilizarea inteligenței artificiale (cu excepția cazului în care utilizarea IA pentru evaluare a primit autorizație prealabilă, de exemplu, ca o ajustare rezonabilă pentru dizabilitatea unui student). Plagiatul poate include și reutilizarea propriei lucrări fără citare. Conform regulamentului pentru examene, plagiatul intenționat sau imprudent este o abatere disciplinară. [1] ”

Într-un studiu din anul 2017, o echipă de cercetători de la două universități din Slovenia și Serbia au realizat un studiu în care au stabilit care sunt motivele pentru care tinerii plagiază, la sondaj participând 139 de studenți cu vârstele cuprinse între 19 și 39 de ani, de la 17 facultăți ale Universității din Maribor, Slovenia. Rezultatele au arătat că tehnologia informației și comunicațiilor este în mare parte responsabilă pentru plagiat, evidențiindu-se două motive: ușurința copierii și ușurința accesului la materiale și noile tehnologii [2].

În acest context, o soluție utilă pentru detectarea plagiatului ar fi de ajutor nu doar profesorilor și învățătorilor ci și studenților. În elaborarea unui articol științific, a unui proiect de curs, a unei teze de licență, ar fi binevenit un astfel de instrument pentru a putea verifica ce procentaj din lucrarea studentului reprezintă conținut autentic și ce procentaj reprezintă plagiat. Detectarea plagiatului bazată pe similaritatea semantică reprezintă o soluție inovatoare care nu se limitează doar la identificarea potrivirilor exacte de cuvinte, ci analizează înțelegerea și structura ideilor din sursele utilizate, oferind astfel un instrument mai precis și mai eficace pentru prevenirea și combaterea tentativelor de plagiat și a plagiatului însuși.

Detectarea plagiatului a evoluat de la potriviri lexicale simple la metode capabile să surprindă asemănarea de sens dintre fragmente cu înțeles identic dar scrise diferit.

În acest context, tehnologiile de detectare a plagiatului bazate pe similaritatea semantică sunt o abordare avansată, care folosesc algoritmi avansați pentru a analiza nu doar cuvintele exact copiate, ci și ideile și conceptele dintr-un text. Aceasta presupune compararea nu doar a structurii sintactice, ci și a semnificației profunde a cuvintelor și frazelor, permițând astfel identificarea unor asemănări care, altfel, ar fi imposibil de observat cu tehnologiile tradiționale bazate pe potriviri exacte de cuvinte.

Prin urmare, această tehnologie nu se limitează doar la documentele de tipul „copy-paste”, ci poate detecta și forme mai subtile de plagiat, precum parafrazarea excesivă sau preluarea de idei fără a le cita corect. Un astfel de sistem ar putea, de asemenea, să ofere profesorilor și cercetătorilor un instrument eficace pentru a evalua autenticitatea lucrărilor științifice, permițându-le să identifice mai rapid orice intenție de plagiat și să reducă astfel riscurile legate de integritatea academică.

1.1 Sisteme similare cu proiectul realizat

Nevoia menținerii integrității academice a dus la apariția unui număr considerabil de platforme digitale specializate în detectarea plagiatului. Printre cele mai utilizate se numără TextGuard, JustDone și Grammarly Plagiarism Checker, fiecare bazându-se pe algoritmi proprii de comparare textuală și pe baze de date extinse, care includ publicații științifice, arhive web și lucrări studențești. Aceste instrumente sunt recunoscute în mediul universitar internațional și sunt utilizate frecvent în procesele de validare a autenticității lucrărilor științifice, a tezelor și a articolelor publicate.

Pentru a evalua acuratețea și consistența acestor sisteme, același document a fost încărcat succesiv pe aceste trei platforme diferite. Rezultatele au evidențiat discrepanțe semnificative între procentele de similitudine raportate.

Prima aplicație utilizată a fost TextGuard, bazată pe tehnologia iThenticate. În figura 1.1 sunt afișate rezultatele nivelului de plagiat în urma încărcării documentului „Raport_Proiect_de_Curs.docx”, indicând un nivel de 92 de procente. Aplicația respectivă este orientată în mare parte spre detectarea plagiatului și a conținutului generat de inteligența artificială. Abordarea predominant lexicală identifică doar potrivirile exacte de text.

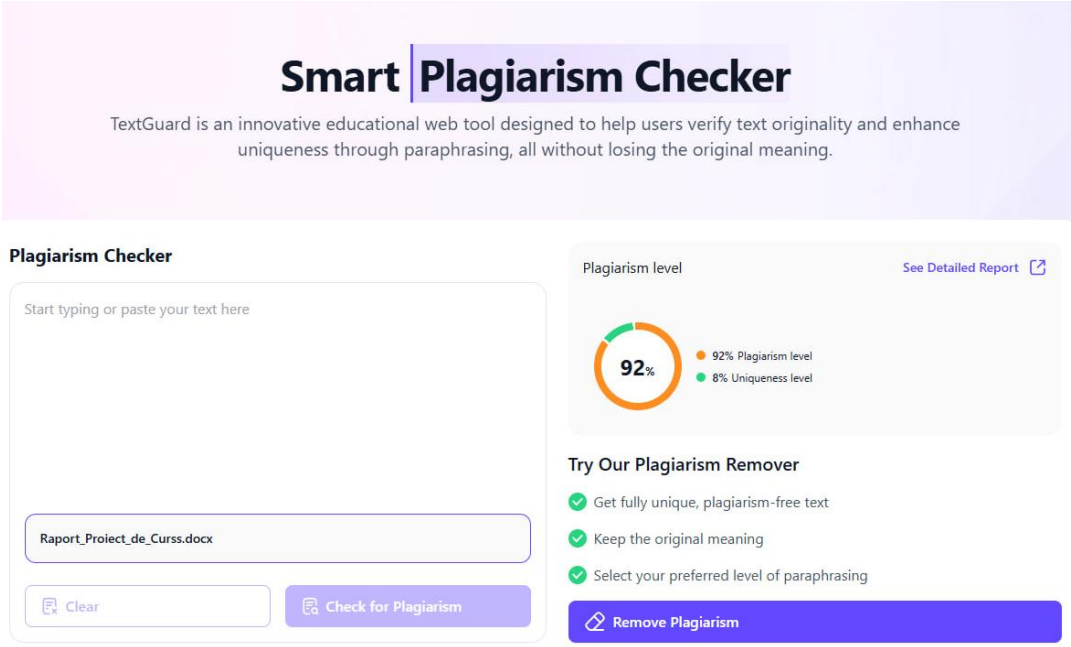

Figura 1.1 – Rezultatele obținute de sistemul TextGuard

A doua platformă digitală utilizată pentru verificarea nivelului de plagiat a fost JustDone, aceasta oferind un rezultat de 78 de procente de conținut plagiat pentru același document încărcat. În figura 1.2 este prezentat rezultatul returnat. Spre deosebire de platforma anterioară, JustDone utilizează un model hibrid ce combină potrivirile lexicale cu analiza contextuală, de aceea raportul oferă un procentaj aparte pentru conținutul identic și un alt procentaj pentru părțile cu mici modificări.

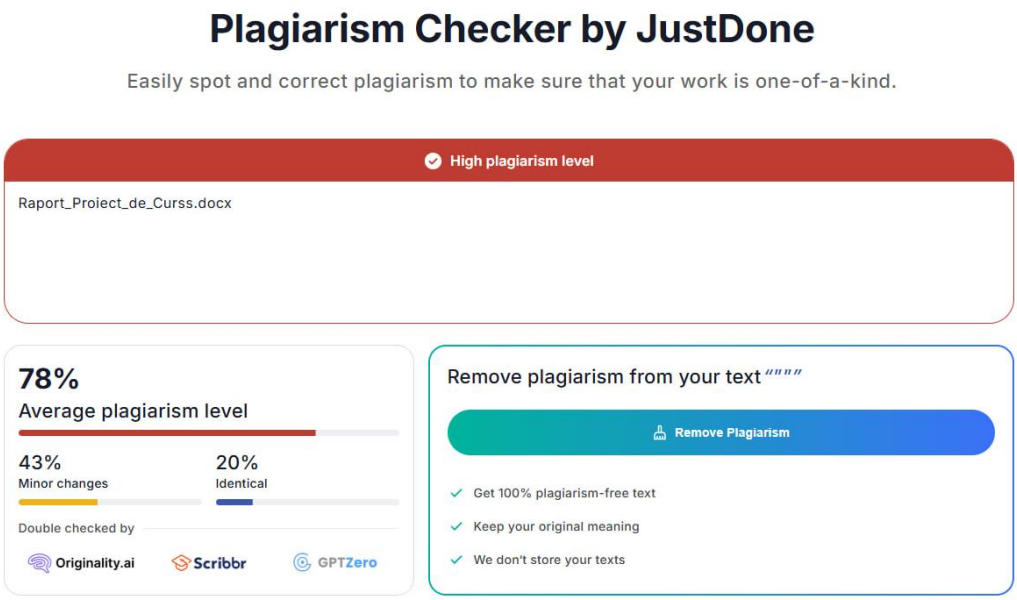

Figura 1.2 – Rezultatele obținute de sistemul TextGuard

A treia aplicație utilizată pentru comparare a fost Grammarly Plagiarism Checker, aceasta returnând un rezultat de doar 55 de procente de conținut plagiat. Platforma dată a fost concepută ca un instrument

pentru uz general, integrând o componentă pentru parafrazarea și rearanjarea textului, motiv pentru care procentele sale tind să fie mai scăzute – indicând o toleranță mai mare față de modificările stilistice și semantice. Raportul obținut în urma verificării aceluiași document este prezentat în figura 1.3.

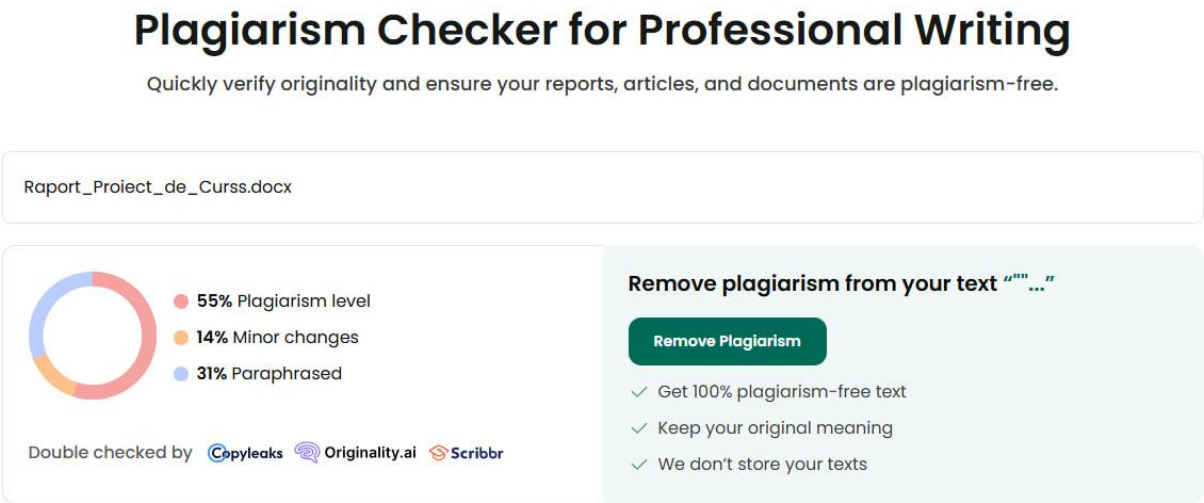

Figura 1.3 – Rezultatele obținute de sistemul Grammarly Plagiarism Checker

Variațiile rezultatelor obținute în urma utilizării a trei platforme digitale distincte demonstrează că deși principiul de funcționare este comun rezultatele diferă în funcție de algoritmul de potrivire lexicală, structura bazelor de date și nivelul de analiză semantică implementat. Aceste diferențe confirmă importanța abordării bazate pe similaritatea semantică, care nu se limitează la potriviri exacte de cuvinte, ci analizează sensul și structura conceptuală a propozițiilor.

O soluție care aplică tehnici moderne de procesare a limbajului natural și învățare automată ar putea reduce erorile de evaluare provenite din sinonimie, parafrazare sau traduceri, asigurând o măsurare mai obiectivă a gradului de originalitate. De asemenea, un astfel de sistem ar permite interpretarea și explicarea diferențelor dintre scorurile raportate, contribuind la creșterea transparenței în procesul de verificare academică. Prin urmare, analiza acestor platforme subliniază limitările sistemelor tradiționale și justifică necesitatea dezvoltării unui instrument care să combine acuratețea lexicală cu capacitatea de înțelegere semantică, astfel încât detectarea plagiatului să fie mai exactă, mai interpretabilă și mai utilă atât pentru profesori, cât și pentru studenți.

1.2 Scopul, obiectivele și cerințele sistemului

Scopul principal al proiectului constă în dezvoltarea unei aplicații software capabile să detecteze plagiatul nu doar pe baza potrivirilor exacte de text, ci și prin analizarea sensului conținutului, utilizând metode moderne de procesare a limbajului natural și învățare automată. Implementarea unei asemenea soluții contribuie la creșterea corectitudinii academice și la prevenirea fraudelor intelectuale, oferind un instrument de verificare mai fidel și mai explicabil decât platformele deja existente.

Ceea ce nu urmărește dezvoltarea acestui proiect, este limitarea totală de la utilizarea textelor și idelor străine în lucrări proprii. Unele teme fiind deja cercetate și unele subiecte fiind deja dezbătute – nu mai există nevoia de a le aborda din nou, ele devenind o sursă de inspirație pentru cei care au nevoie să-și dezvolte ideile proprii. Deseori și utilizarea textelor sau lucrărilor străine în propriile lucrări este binevenită, doar că acestea trebuie menționate ca fiind preluate din alte surse, pentru a nu fi încălcate drepturile de autor.

Pentru realizarea scopului propus au fost stabilite următoarele obiective specifice:

- analizarea domeniului plagiatului academic și identificarea problemelor curente ale soluțiilor existente;
- cercetarea și evaluarea metodelor și algoritmilor de detecție a similarității semantice din literatura de specialitate;
- stabilirea cerințelor funcționale și nefuncționale pentru aplicația propusă;
- proiectarea arhitecturală a aplicației și interfața utilizatorului;
- dezvoltarea unui modul de preprocesare a textului (normalizare, eliminarea cuvintelor neimportante etc.);
- implementarea metodelor pentru redarea sensului textelor;
- integrarea unui mecanism de comparare a textelor bazat pe scoruri de similaritate semantică;
- realizarea testării și validării aplicației pe seturi de date reale, inclusiv documente academice și articole științifice.

Implementarea soluției propuse va conduce la crearea unui instrument software performant și inteligent, capabil să identifice plagiatul nu doar pe baza potrivirilor exacte de text, ci și prin analizarea sensului propozițiilor și a relațiilor semantice dintre fragmente. Astfel, aplicația va permite detectarea formelor complexe de plagiat, cum ar fi parafrazarea, traducerea neautorizată sau reformularea ideilor, oferind rezultate cu un grad ridicat de acuratețe. De asemenea, aplicațial va furniza rapoarte explicabile, care vor evidenția atât sursele identificate, cât și nivelul de similaritate, facilitând înțelegerea procesului de verificare și eliminând interpretările ambigue.

Prin utilizarea acestei soluții, se preconizează o îmbunătățire vizibilă a procesului de verificare a lucrărilor academice, atât din perspectiva profesorilor, cât și a studenților. Profesorii vor beneficia de un instrument de evaluare rapid și obiectiv, capabil să reducă volumul de muncă și riscul de erori umane, în

timp ce studenții vor avea posibilitatea de a-și autoverifica lucrările înainte de predare, asigurând un conținut original și corect redactat. În consecință, proiectul va contribui la consolidarea integrității academice și la promovarea unei culturi educaționale bazate pe corectitudine, responsabilitate și respect față de drepturile de autor.

1.3 Cerințe funcționale

Cerințele funcționale ale unui proiect IT definesc comportamentele aplicației și descriu modul în care aceasta procesează documentele încărcate de utilizatori. Proiectul este orientat exclusiv spre analiza textelor și generarea de rapoarte de similitudine. Utilizarea aplicației presupune parcurgerea mai multor etape funcționale, începând cu autentificarea utilizatorilor și continuând cu încărcarea, procesarea și analiza documentelor.

Primul pas în utilizarea aplicației este autentificarea utilizatorului. Accesul la funcționalitățile oferite va fi realizat pe baza unei adrese de e-mail de tip Gmail și a unei parole asociate contului. Soluția trebuie să permită crearea unui cont nou prin introducerea numelui, familiei, adresei de Gmail valide și a unei parole, precum și autentificarea utilizatorilor existenți pe baza acestor date. Parolele vor fi stocate în formă securizată, iar aplicația va valida corectitudinea datelor introduse înainte de a permite accesul la funcțiile de analiză. În cazul introducerii unor date incorecte, utilizatorul va fi informat corespunzător, fără a i se permite accesul neautorizat la sistem.

După autentificare, utilizatorul poate iniția procesul de încărcare a documentelor, care are loc în conformitate cu următorii pași:

- utilizatorul trebuie să poată încărca un fișier text (ex. .docx, .pdf, .txt) pentru analiză, aplicația fiind responsabilă de preluarea corectă a conținutului acestuia, indiferent de formatul ales;
- în cazul verificării de tip document–document, aplicația trebuie să permită încărcarea unui al doilea fișier, care va fi utilizat exclusiv ca sursă de comparație;
- aplicația trebuie să verifice dacă documentele sunt lizibile, necorupte și conțin text procesabil, iar în situațiile în care aceste condiții nu sunt îndeplinite, utilizatorul va fi notificat cu privire la imposibilitatea continuării analizei.

Pasul care urmează după încărcarea documentelor este procesarea și analiza textului. Acest proces va avea loc în felul următor:

- se va extrage textul brut din fișiere, indiferent de formatul acestora, pentru a asigura o bază comună de prelucrare;
- se va normaliza textul prin eliminarea caracterelor redundante, conversia tuturor cuvintelor la litere mici și eliminarea cuvintelor frecvente fără relevanță analitică (stopwords);
- se vor identifica secvențele reproduse identic din alte surse sau din documentul de referință atașat;

- se vor identifica fragmente care nu sunt copiate textual, dar prezintă un grad ridicat de apropiere semantică, sugerând reformulări sau parafrazări;
- se va compara documentul încărcat cu o colecție internă sau predefinită de resurse, în vederea identificării potențialelor surse de similitudine;
- se va calcula gradul de similitudine între două documente încărcate de utilizator, în cazul analizelor de tip document–document.

După finalizarea procesării documentelor și a analizei textului, aplicația va calcula nivelul de similitudine și va genera raportul de analiză. Acest proces va avea loc în felul următor:

- se va determina procentul total de conținut suspectat ca fiind preluat din alte surse;
- se vor identifica și marca segmentele din text unde au fost detectate potriviri sau parafrazări, indicând poziția acestora în document;
- se va crea un raport detaliat care va include procentul final de similitudine, lista fragmentelor identificate și localizarea acestora;
- utilizatorul va avea posibilitatea de a descărca raportul final într-un format lizibil, precum PDF sau alt format similar, pentru utilizare ulterioară.

1.4 Cerințe nefuncționale

Pentru ca un sistem anti plagiat să fie fiabil și scalabil, pe lângă cerințele funcționale trebuie detaliate și cerințele nefuncționale, axate pe securitate, performanță, stabilitate. În cazul soluției propuse, cerințele nefuncționale vor include următoarele aspecte:

- capacitatea procesării documentelor de dimensiuni mari într-un timp scurt;
- procesarea cererilor mai multor utilizatori simultan;
- utilizarea surselor existente în mediul online pentru detectarea plagiatului;
- utilizarea algoritmilor pentru optimizarea procesului.

Pentru evaluarea similitudinii între documente se va folosi TF-IDF, Cosine Similarity, algoritmi de comparare a textelor, și SimHash, algoritm de hashing pentru o comparare mai rapidă.

Securitatea datelor și protecția împotriva accesului neautorizat trebuie să fie asigurate prin măsuri stricte de protecție. Aceste măsuri vor fi implementate la nivel de:

- criptare a datelor sensibile la nivel de stocare;
- autentificare a utilizatorilor pentru a garanta accesul doar persoanelor autorizate.

Aplicația trebuie să fie disponibilă și funcțională 24/7, asigurându-se că datele sunt procesate corect și că nu există pierderi de informații. Funcționalitatea va fi garantată de:

- sincronizarea datelor pentru a asigura că documentele și rezultatele comparațiilor sunt actualizate în timp real;

– mecanismele de backup și de recuperare a datelor, pentru a asigura continuitatea activităților în caz de eroare sau defecțiuni ale sistemului.

Compatibilitatea aplicației va fi asigurată prin posibilitatea rulării fără probleme pe majoritatea browserelor web moderne (Chrome, Firefox, Safari) și va fi optimizată pentru utilizare pe desktopuri și dispozitive mobile cu sisteme de operare uzuale.

Accesibilitatea unui soft este determinată de dezvoltarea produsului în așa mod, încât el să poată fi folosit de către mai multe grupuri de utilizatori. Soluția va fi concepută astfel încât să fie accesibilă unui public cât mai larg, inclusiv utilizatorilor cu diverse tipuri de dizabilități. Accesibilitatea aplicației va fi garantată printr-o interfață intuitivă și ușor de navigat, instrucțiuni clare și vizibile și respectare a standardelor de accesibilitate.

1.5 Specificațiile tehnice

Soluția destinată verificării gradului de similitudine între documente de tip text trebuie să utilizeze o arhitectură modernă, având la bază aplicația dezvoltată în limbajul C# pe platforma .NET, cu un backend bazat pe ASP.NET Core Web API. Aplicația trebuie să asigure procesarea, analizarea și compararea conținutului textual, precum și gestionarea sigură a autentificării utilizatorilor, stocarea documentelor și generarea rapoartelor.

Arhitectura aplicației este prezentată într-un mod mai detaliat în figura 1.4, fiind structurată în baza principiului client-server, unde utilizatorul final interacționează cu interfața frontend care transmite cereri către backend prin intermediul unor API-uri REST. Backend-ul trebuie să fie responsabil de preprocesarea textului, aplicarea algoritmilor de analiză a similitudinii și gestionarea datelor în baza de date. De asemenea, arhitectura include componente dedicate pentru securitate, audit și sincronizare, asigurând integritatea și disponibilitatea datelor.

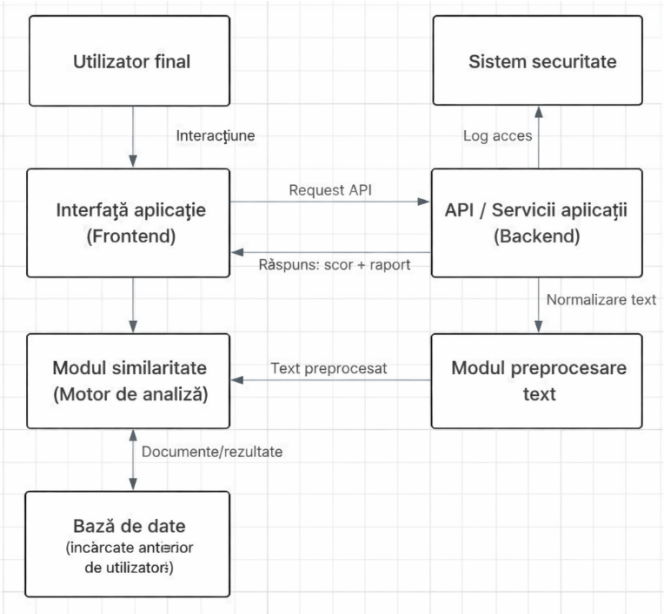

Figura 1.4 – Arhitectura aplicației pentru verificarea plagiatului

Partea de front a aplicației trebuie dezvoltată utilizând C# și .NET, oferind o interfață intuitivă pentru modulele de autentificare a utilizatorilor, încărcarea documentelor text, inițierea procesului de verificare a similitudinii și afișarea rezultatelor sub formă de procente și rapoarte detaliate. Interfața va facilita interpretarea rezultatelor printr-o structură clară și ușor de utilizat.

Partea de server trebuie implementată folosind tehnologia ASP.NET Core, expunând un Restful API pentru gestionarea logicii aplicației. Backend-ul răspunzând de validarea documentelor încărcate, de preprocesarea textului, de aplicarea algoritmilor de calcul al similitudinii și de generarea și salvarea rapoartelor de similitudine. Comunicarea dintre client și server trebuie securizată, iar accesul la funcționalitățile aplicației controlat în baza autentificării utilizatorului.

Pentru detectarea gradului de similitudine între documente, aplicația va utiliza algoritmi consacrați în domeniul procesării limbajului natural, precum:

- Cosine Similarity, utilizat pentru compararea vectorială a documentelor;
- TF-IDF, pentru ponderea relevanței termenilor;
- Tehnici de hashing, utilizate pentru optimizarea performanței în cazul volumelor mari de date.

Acești algoritmi permit identificarea rapidă și precisă a conținutului similar, reducând complexitatea procesului de analiză.

Baza de date trebuie să stocheze informații despre utilizatori, documente analizate, rezultate ale verificărilor și istoricul comparațiilor, utilizându-se SQL Server. Documentele stocate în format securizat, rezultatele analizei vor fi păstrate pentru audit și consultare ulterioară. Se vor implementa politici de back-up automat și mecanisme de recuperare a datelor.

Securitatea și protecția datelor reprezintă un aspect important al aplicației asigurând criptarea datelor sensibile la nivel de stocare și transfer, autentificarea și utilizarea utilizatorilor și protecția împotriva accesului neautorizat și a manipulării documentelor. Transferul de date va fi realizat prin protocoale securizate (SSL/TLS), iar aplicația va respecta cerințele privind protecția datelor cu caracter personal, în conformitate cu reglementările GDPR.

Arhitectura modulară a soluției permite extinderea acesteia cu funcționalități suplimentare, precum exportul rapoartelor în format PDF și trimiterea notificărilor automate utilizatorilor.

1.6 Interfața utilizatorului

Interfața oricărei aplicații trebuie să fie concepută în așa fel încât să fie intuitivă și ușor de utilizat diferitelor grupuri de utilizatori, precum studenți, profesori, administratori. UI-ul aplicației pentru verificarea gradului de similitudine va fi realizat cu ajutorul tehnologiilor moderne bazate pe C# și .NET, având o structură clară, organizată pe module și meniuri, care permite acces rapid la funcționalitățile principale ale sistemului: autentificare, încărcare a documentelor, inițierea verificării similitudinii și vizualizarea rezultatelor.

Accesul în aplicație se realizează printr-o pagină de autentificare simplă, bazată pe introducerea adresei de email și a parolei, fără diferențiere pe roluri de utilizator. Această abordare contribuie la simplificarea fluxului de utilizare și la reducerea complexității interfeței, aplicația fiind destinată utilizării generale pentru verificarea gradului de similitudine a documentelor.

În figura 1.5 este prezentat prototipul paginii de autentificare a aplicației pentru verificarea gradului de similitudine. Aceasta oferă utilizatorului un punct de acces securizat în sistem, incluzând câmpuri pentru introducerea credențialelor, opțiunea de recuperare a parolei și posibilitatea de înregistrare a unui cont nou. Designul paginii este simplist și intuitiv, facilitând accesul rapid la funcționalitățile aplicației.

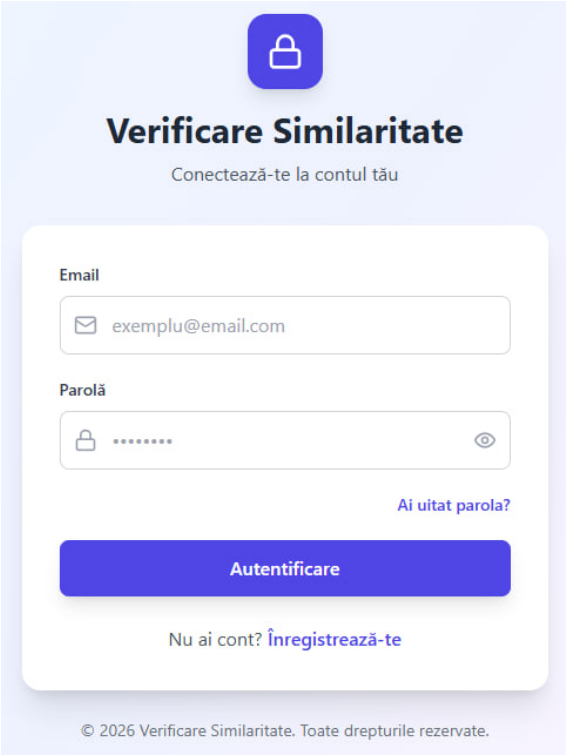

Figura 1.5 – Pagina pentru autentificarea utilizatorului

În figura 1.6 este ilustrat prototipul paginii de încărcare a documentelor, unde utilizatorul poate adăuga unul sau mai multe fișiere pentru analiză. Sunt afișate instrucțiuni clare privind formatele acceptate și dimensiunea maximă a fișierelor. Interfața permite încărcarea documentelor prin selecție directă, oferind

control asupra procesului de analiză prin opțiuni precum ștergerea fișierelor încărcate sau inițierea verificării gradului de similitudine.

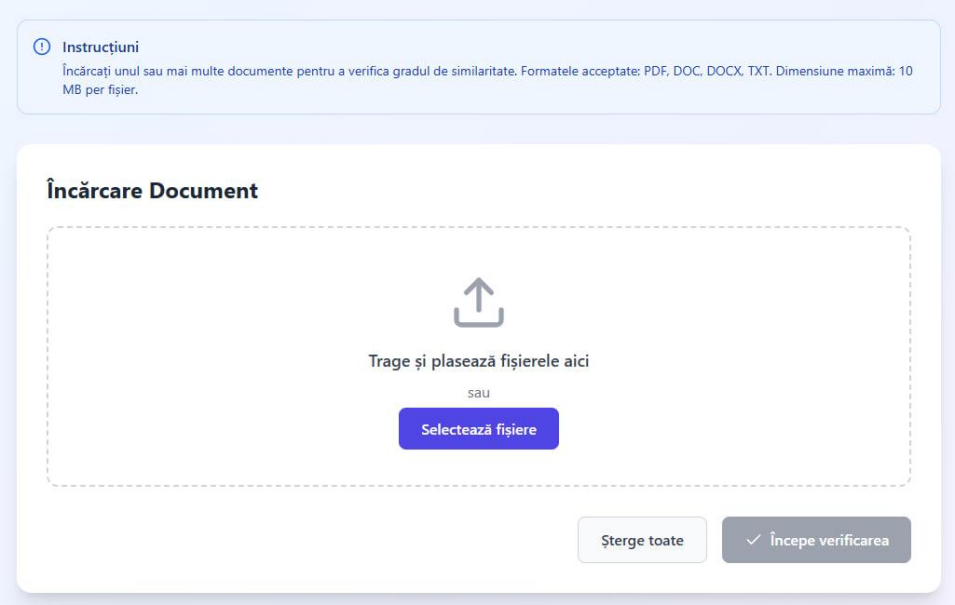

Figura 1.6 – Pagina pentru încărcarea documentelor spre verificare

În figura 1.7 este prototipul unui raport care va afișa rezultatele analizei de similitudine. Aceasta oferă utilizatorului informații sintetizate și ușor de interpretat, precum scorul de similitudine exprimat procentual, nivelul de atenție asociat rezultatului, numărul total de cuvinte analizate, numărul surselor verificate și timpul de procesare. Rezultatele sunt prezentate sub formă de indicatori vizuali și valori numerice, permițând evaluarea rapidă a gradului de originalitate al documentului. Utilizatorul are posibilitatea de a descărca documentul analizat și de a consulta informații suplimentare privind sursele identificate.

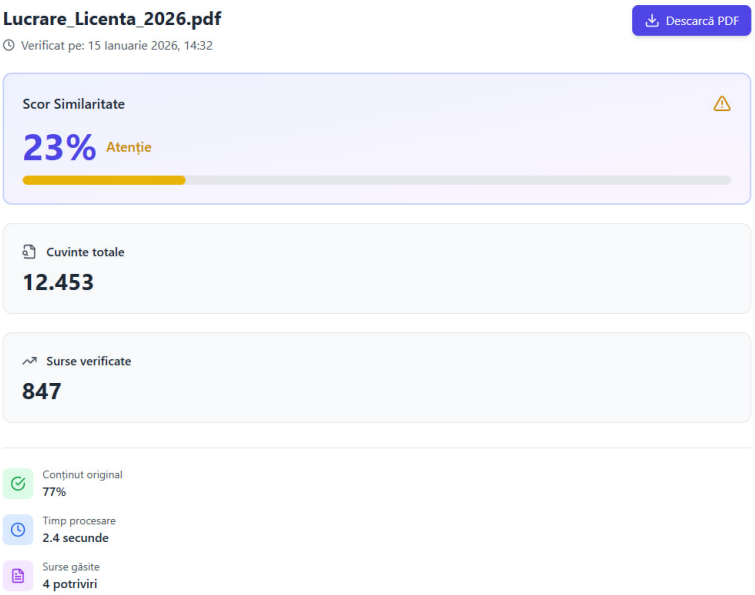

Figura 1.7 – Raportul cu rezultatele analizei

Interfața aplicației este organizată pe secțiuni bine delimitate, utilizând carduri și elemente vizuale sugestive pentru a evidenția informațiile relevante. Accentul este pus pe lizibilitate, claritatea datelor și feedback vizual imediat, astfel încât utilizatorul să poată înțelege rezultatele fără cunoștințe tehnice avansate. Pe parcursul dezvoltării aplicației, designul interfeței va mai fi ajustat în funcție de cerințele utilizatorilor și de necesitățile mediului academic sau profesional în care aceasta va fi utilizată pentru a obține o experiență cât mai utilă.

1.7 Planul de testare

Planul de testare destinat unui produs software are ca scop principal validarea îndeplinirii tuturor cerințelor funcționale și nefuncționale ale aplicației. În cazul unei aplicații pentru verificarea gradului de similitudine, acesta asigură corectitudinea rezultatelor, performanța sistemului, securitatea datelor și o experiență de utilizare adecvată. Procesul de testare va fi realizat sistematic, prin aplicarea unor metode și tehnici specifice, adaptate diferitelor scenarii de utilizare și mediilor de operare.

Obiectivele testării constau în verificarea corectitudinii procesului de autentificare a utilizatorilor, validarea mecanismelor de încărcare și analiză a documentelor, evaluarea acurateței algoritmilor de calcul al similitudinii, precum și asigurarea integrității și confidențialității datelor. Astfel, se urmărește evaluarea comportamentului aplicației în condiții de utilizare intensă, pe echipamente cu resurse limitate și în situații de conectivitate variabilă.

Testarea funcțională va fi realizată pentru a verifica funcționarea corectă a tuturor componentelor aplicației. În cadrul acestei etape vor fi testate următoarele aspecte:

- procesul de autentificare și acces al utilizatorilor în aplicație;
- încărcarea documentelor în formatele acceptate;
- inițierea procesului de verificare a gradului de similitudine;
- afișarea corectă a rezultatelor, inclusiv scorul de similitudine, procentul de conținut original, numărul de surse identificate și timpul de procesare;
- generarea și descărcarea rapoartelor de analiză.

Se va verifica comportamentul aplicației atât în situații normale de utilizare, cât și în cazuri de eroare, cum ar fi fișiere invalide, dimensiuni depășite sau întreruperi ale procesului de analiză. Se va monitoriza timpul total de procesare al documentelor, utilizarea resurselor de sistem și stabilitatea aplicației în condiții de sarcină ridicată, fără degradarea semnificativă a performanței.

Unele scenarii relevante pentru testarea aplicației de verificare a gradului de similitudine vizează autentificarea utilizatorului și accesarea corectă a aplicației, încărcarea unui document valid și obținerea unui rezultat de analiză corect, precum și încărcarea unor documente invalide pentru a verifica afișarea mesajelor de eroare corespunzătoare. De asemenea, se urmărește compararea unui document cu baza de date existentă în vederea identificării fragmentelor similare, testarea comportamentului aplicației în cazul

întreruperii conexiunii în timpul procesului de analiză, precum și verificarea afișării corecte și a posibilității de descărcare a rapoartelor de similitudine generate de sistem.

Aplicația va fi considerată corect implementată dacă sunt îndeplinite următoarele criterii:

- scorul de similitudine este calculat corect și afișat clar pentru utilizator;
- datele și documentele sunt stocate și transmise în mod securizat;
- timpii de răspuns se încadrează în limite acceptabile pentru utilizare normală;
- aplicația funcționează stabil în condiții de utilizare simultană;
- toate erorile și incidentele sunt înregistrate și pot fi analizate ulterior.

Testarea va combina metode de testare manuală și automată. Testarea automată va include teste unitare pentru componentele de bază, teste de integrare și teste de performanță, iar testarea manuală va fi utilizată pentru validarea interfeței utilizatorului și a experienței de utilizare (UI/UX). Pentru fiecare problemă identificată se va genera un raport de testare care va include descrierea scenariului, pașii urmați, rezultatul așteptat și rezultatul obținut.

Prin aplicarea acestui plan de testare se va asigura o implementare completă, corectă și sigură a aplicației pentru verificarea gradului de similitudine, adecvată utilizării în mediul academic sau profesional.

1.8 Constrângeri și riscuri

Dezvoltarea unei aplicații pentru verificarea gradului de similitudine implică o serie de constrângeri tehnice și riscuri operaționale care pot influența corectitudinea rezultatelor, performanța sistemului și securitatea datelor procesate. Aceste constrângeri pot apărea la nivelul infrastructurii hardware, al soluțiilor software utilizate, al bazelor de date, precum și al algoritmilor de analiză și comparare a documentelor.

Una dintre principalele constrângeri este legată de volumul și complexitatea documentelor analizate. Aplicația trebuie să gestioneze fișiere de dimensiuni variabile, în formate diferite, ceea ce presupune utilizarea unor mecanisme eficiente de extragere și normalizare a textului. Documentele de mari dimensiuni sau cele care conțin structuri complexe pot conduce la creșterea timpului de procesare și la consum ridicat de resurse de calcul, afectând astfel performanța generală a aplicației.

Din perspectiva resurselor hardware, procesarea simultană a mai multor documente și aplicarea algoritmilor de similitudine pot necesita o capacitate adecvată de procesare și memorie. Sistemele cu resurse limitate pot genera timpi de răspuns crescuți, în special în scenarii de utilizare intensivă sau în cazul analizării unor corpusuri extinse de documente. Acest risc impune optimizarea algoritmilor utilizați și o gestionare corespunzătoare a memoriei și a proceselor de calcul.

Baza de date reprezintă un element critic al aplicației, fiind responsabilă de stocarea documentelor, a rezultatelor analizelor și a istoricului verificărilor. Constrângerile apar în special în cazul gestionării unui număr mare de documente și rezultate, unde lipsa unei structuri adecvate de indexare sau a mecanismelor

de optimizare poate conduce la degradarea performanței interogărilor. De asemenea, riscurile asociate pierderii de date sau coruperii acestora impun implementarea unor politici stricte de backup și recuperare.

Algoritmii utilizați pentru calculul gradului de similitudine constituie o altă sursă importantă de riscuri. Alegerea unor metode inadecvate sau configurarea incorectă a parametrilor poate conduce la rezultate eronate, fie prin supraestimarea gradului de similitudine, fie prin omiterea unor fragmente relevante. În plus, algoritmii bazați pe procesarea limbajului natural pot fi influențați de particularitățile limbii, de structura textului sau de prezența parafrazărilor, ceea ce necesită testare și calibrare atentă.

Din punct de vedere software, compatibilitatea aplicației dezvoltate în C# și .NET trebuie asigurată cu sistemele de operare moderne și cu bibliotecile externe utilizate pentru procesarea textului. Actualizările platformei .NET sau ale componentelor software pot introduce incompatibilități sau modificări de comportament, reprezentând un risc pentru stabilitatea aplicației dacă nu sunt gestionate corespunzător.

Un alt risc este asociat dependenței de conectivitatea la rețea și de eventualele surse externe de comparație. În cazul întreruperilor de conexiune sau al indisponibilității resurselor externe, procesul de verificare a similitudinii poate fi afectat, conducând la rezultate incomplete sau întârzieri semnificative. Aceste situații necesită mecanisme de gestionare a erorilor și proceduri clare de reluare a procesului de analiză.

Prin identificarea și gestionarea corespunzătoare a acestor constrângeri și riscuri, aplicația pentru verificarea gradului de similitudine poate asigura funcționarea stabilă, rezultate corecte și un nivel adecvat de securitate, reducând semnificativ probabilitatea apariției erorilor critice sau a deficiențelor operaționale.

2 PROIECTAREA SISTEMULUI INFORMATIC

Proiectarea unui sistem reprezintă etapa fundamentală din ciclul de dezvoltare software în care sunt definite arhitectura, structura și modul de funcționare al sistemului înainte de implementarea propriu-zisă. În această fază, cerințele funcționale și nefuncționale identificate în etapa de analiză sunt transformate în modele conceptuale și tehnice care descriu componentele sistemului, relațiile dintre acestea, fluxurile de date și comportamentul proceselor interne. Proiectarea asigură o viziune clară asupra modului în care sistemul va fi construit, contribuind la reducerea riscurilor, optimizarea resurselor și asigurarea coerenței structurale a aplicației. O proiectare corect realizată facilitează ulterior implementarea, testarea și mentenanța sistemului informatic.

Metoda de proiectare aleasă este metoda OOD datorită capacității sale de a modela sistemele complexe prin intermediul conceptelor de clasă și obiect. Aceasta se bazează pe cele 4 principii fundamentale: încapsularea, moștenirea, polimorfismul și abstractizarea, care permit organizarea logică a componentelor și reutilizarea codului. Proiectarea orientată pe obiect este adecvată pentru dezvoltarea aplicațiilor moderne, facilitând modularitatea, extensibilitatea și mentenanța sistemului. În plus, această abordare permite reprezentarea clară a entităților din domeniul problemei și a relațiilor dintre acestea, asigurând o corespondență directă între modelul conceptual și implementarea software.

Modelarea sistemului s-a realizat prin intermediul limbajului UML, un standard internațional pentru descrierea, vizualizarea și documentarea sistemelor software. UML oferă un set de diagrame structurale și comportamentale care permit reprezentarea diferitelor perspective ale sistemului, cum ar fi structura statică, interacțiunile dintre componente și fluxurile operaționale. Printre diagramele utilizate se numără diagrama de cazuri de utilizare, diagrama de clase, diagrama de secvență și diagrama de activitate, fiecare contribuind la o înțelegere detaliată a funcționalității și arhitecturii aplicației. Utilizarea UML asigură un limbaj comun de comunicare între dezvoltatori, analiști și alți factori implicați în proiect.

Figura 2.1 – Logo-ul softului Enterprise Architect [3]

În figura 2.1 este reprezentat logo-ul softului Enterprise Architect, utilizat pentru crearea diagramelor UML și a modelării întregului sistem, acesta fiind un instrument profesional destinat analizei și proiectării sistemelor informatice. Enterprise Architect permite crearea și gestionarea diagramelor UML, organizarea modelelor în pachete logice și menținerea coerenței între elementele modelate. Platforma oferă suport pentru diferite tipuri de diagrame, validarea relațiilor dintre elemente și posibilitatea generării documentației tehnice. Utilizarea acestui instrument contribuie la o modelare structurată și riguroasă a sistemului, facilitând documentarea completă a arhitecturii și a comportamentului aplicației dezvoltate.

2.1 Descrierea comportamentală a sistemului

Descrierea comportamentală a sistemului scoate în evidență modul în care sistemul reacționează la acțiunile utilizatorilor sau la evenimente interne, punând accent pe dinamica proceselor și pe interacțiunile dintre componente. Spre deosebire de modelarea structurală, care prezintă elementele statice ale sistemului, descrierea comportamentală urmărește evoluția în timp a sistemului, succesiunea operațiilor, fluxurile de control și schimbul de mesaje. Acestea contribuie la înțelegerea modului de funcționare al aplicației în diferite scenarii de utilizare și permite identificarea clară a responsabilităților fiecărei componente implicate în proces.

Imaginea generală asupra sistemului este modelată prin intermediul diagramelor cazurilor de utilizare, oferind o perspectivă de ansamblu asupra funcționalităților sistemului și a modului în care actorii externi interacționează cu acesta. Această diagramă evidențiază principalele servicii oferite de aplicație și delimitează clar frontiera sistemului. Prin intermediul cazurilor de utilizare sunt identificate cerințele funcționale, iar relațiile dintre actori și funcționalități permit înțelegerea rolurilor și a responsabilităților în cadrul sistemului. Diagrama constituie punctul de plecare pentru detalierea scenariilor de operare și pentru modelarea comportamentului ulterior.

Diagrama de activitate este utilizată în mod special pentru modelarea vizuală a fluxurilor și descrie succesiunea activităților și fluxul logic al operațiilor din cadrul unui proces. Aceasta evidențiază pașii parcurși de utilizator sau de sistem, deciziile condiționale și eventualele ramificații ale fluxului de execuție. Diagrama de activitate este utilă pentru reprezentarea proceselor complexe, oferind o imagine clară asupra modului în care se desfășoară o funcționalitate de la inițiere până la finalizare. Prin utilizarea acesteia se pot identifica eventuale blocaje sau redundanțe în fluxul operațional.

Stările de tranzacție ale sistemului sunt proiectare cu ajutorul diagramei de stare, aceasta prezentând evoluția unui obiect sau a sistemului în funcție de evenimentele care determină schimbarea stării acestuia. Diagrama în cauză evidențiază diferitele stări prin care poate trece un element al sistemului și condițiile care provoacă tranzițiile dintre ele. Modelarea stărilor este importantă în situațiile în care comportamentul sistemului depinde de context sau de statutul curent al unei entități. Diagrama contribuie la clarificarea logicii interne și la prevenirea erorilor de procesare.

Diagrama de secvență este utilizată pentru descrierea scenariilor de utilizare a aplicației și ilustrează interacțiunea dintre actori și componentele sistemului într-un anumit scenariu concret. Aceasta pune accent pe ordinea cronologică a mesajelor schimbate între obiecte, evidențiind responsabilitățile și fluxul de control. Diagrama de secvență permite analizarea detaliată a modului în care o funcționalitate este implementată la nivel de interacțiuni interne, fiind un instrument util pentru validarea logicii aplicației și pentru corelarea modelului conceptual cu implementarea efectivă.

Fluxurile de mesaje și legăturile dintre componente sunt modelate prin intermediul diagramei de colaborare, acestea reprezentând relațiile structurale dintre obiecte și modul în care acestea colaborează

prin schimb de mesaje pentru realizarea unei funcționalități. Spre deosebire de diagrama de secvență, care pune accent pe ordinea temporală, diagrama de colaborare evidențiază conexiunile dintre componente și distribuția responsabilităților în cadrul sistemului. Aceasta contribuie la o înțelegere clară a arhitecturii comportamentale și a modului în care elementele sistemului cooperează pentru atingerea obiectivelor definite.

2.1.1 Imaginea generală asupra sistemului

Diagramele caz de utilizare afișează comportamentul așteptat al sistemului. Acestea nu afișează ordinea în care se efectuează pașii. Definirea limită de sistem determină ceea ce este considerat extern sau intern pentru sistem. Un actor reprezintă un rol jucat de un obiect extern. Un obiect poate juca mai multe roluri și, prin urmare, este reprezentat de mai mulți actori. O asociere ilustrează participarea actorului la cazul de utilizare.

Un caz de utilizare este un set de evenimente care au loc atunci când un actor utilizează un sistem pentru a finaliza un proces. În mod normal, un caz de utilizare este un proces relativ mare, nu un pas individual sau o tranzacție [4].

Diagrama din figura 2.1 ilustrează interacțiunea directă dintre utilizator și funcționalitățile principale ale aplicației „SimilarityChecker”, evidențiind procesul complet de verificare a unui document. Utilizatorul se autentifică, încarcă fișierul și lansează analiza de similitudine, proces care include validarea documentului, extragerea textului, preprocesarea și normalizarea conținutului, precum și calculul propriu-zis al gradului de similitudine. În urma analizei este generat raportul, care poate fi vizualizat și descărcat. Relațiile de tip «include» indică faptul că anumite operații sunt componente obligatorii ale procesului principal de analiză.

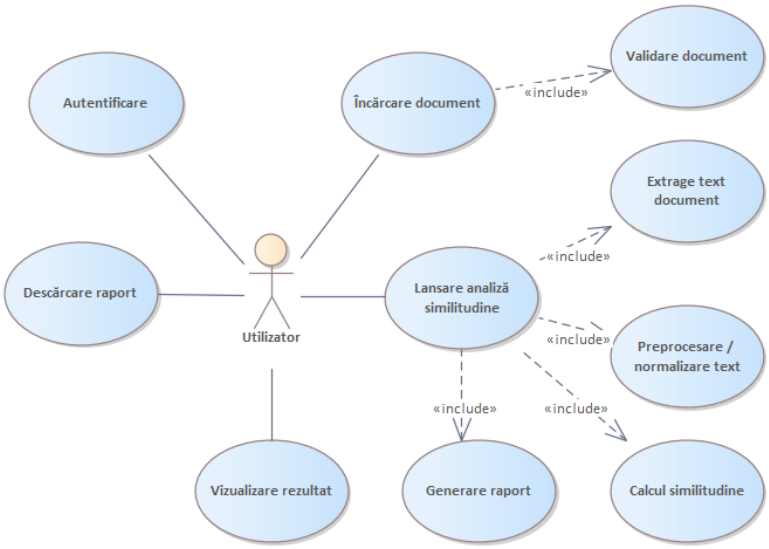

Figura 2.2 – Cazurile de utilizare pentru analiza similitudinii documentelor

În figura 2.3 diagrama prezintă modul în care interfața utilizatorului orchestrează gestionarea fișierelor și inițierea procesului de analiză. Gestionarea fișierului încărcat include validarea și salvarea acestuia, iar inițializarea analizei declanșează un lanț de operații interne precum extragerea textului, normalizarea conținutului, calcularea similitudinii, generarea raportului și persistarea rezultatelor. Totodată, sistemul furnizează statusul analizei și pune la dispoziție mecanisme pentru vizualizarea rezultatului și descărcarea raportului. Diagrama evidențiază separarea clară între responsabilitățile interfeței și logica de procesare internă.

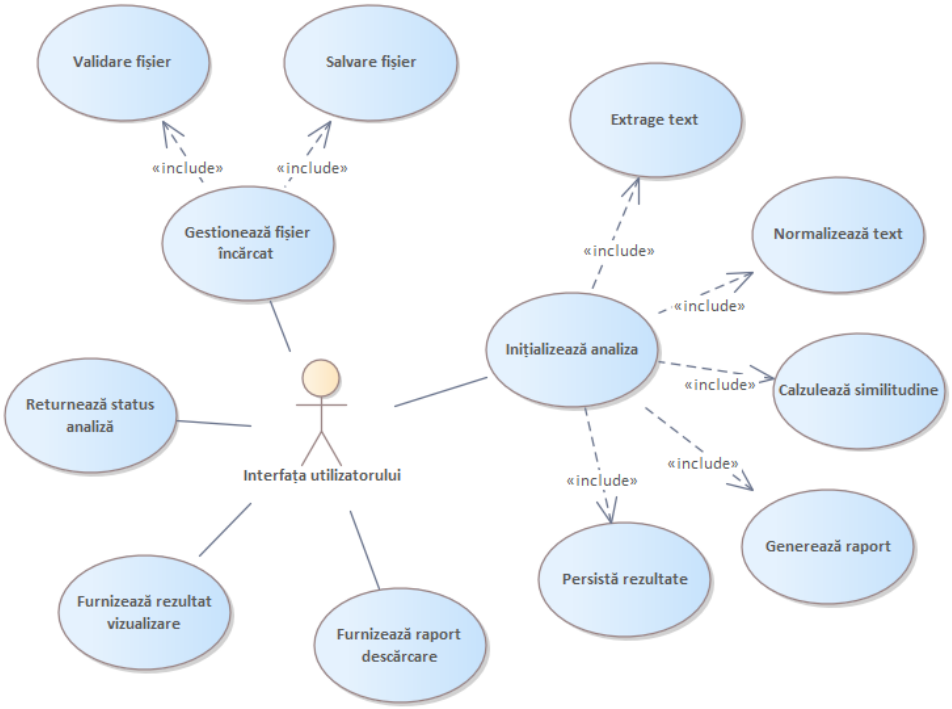

Figura 2.3 - Cazurile de utilizare ale procesării documentelor în interfața aplicației

În figura 2.4 este reprezentată diagrama interacțiunii cu serviciul de stocare și gestionare a datelor. Această diagramă evidențiază rolul serviciului bazei de date în cadrul aplicației „SimilarityChecker”. Componenta este responsabilă pentru interogarea rezultatelor analizelor, actualizarea informațiilor și stocarea fișierelor asociate procesului de verificare. Prin aceste operații se asigură persistența datelor, integritatea informațiilor și disponibilitatea rezultatelor pentru consultări ulterioare. Diagrama subliniază faptul că serviciul bazei de date funcționează ca un suport infrastructural esențial pentru funcționarea întregului sistem.

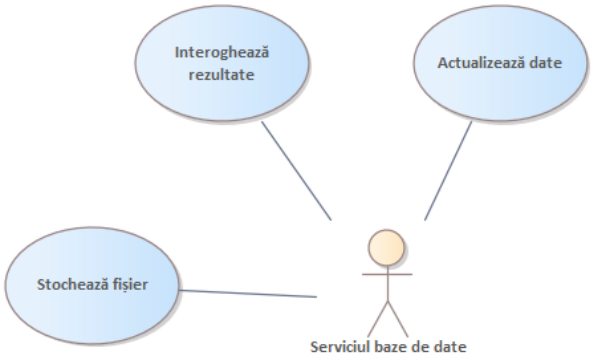

Figura 2.4 – Cazurile de utilizare asociate serviciului bazei de date

Modelele prezentate oferă o imagine clară asupra modului în care aplicația gestionează procesul de verificare a documentelor, de la interacțiunea utilizatorului până la procesarea internă și stocarea rezultatelor. Prin structurarea funcționalităților și evidențierea relațiilor dintre componente, se asigură o înțelegere coerentă a fluxului operațional și a responsabilităților fiecărui element al sistemului.

2.1.2 Modelarea vizuală a fluxurilor

O diagramă de activitate este o diagramă flux care arată cum o activitate duce la alta. Acțiunea poate fi denumită o operație de sistem. O operație duce la următoarea în fluxul de control. Acest flux poate fi paralel, contemporan sau ramificat. Diagramele de activitate folosesc multe caracteristici pentru a face față tuturor tipurilor de control al fluxului. Similar altor diagrame, diagramele de activitate servesc unor obiective fundamentale similare. Captează comportamentul dinamic al sistemului. O activitate este o funcție specifică a sistemului. Diagramele de activitate construiesc sistemul executabil utilizând abordări de inginerie directă și inversă. De asemenea, este de a vizualiza natura dinamică a unui sistem. Porțiunea de mesaj este singurul element din care lipsește diagrama de activitate. Nu este afișat niciun flux de mesaje de la o activitate la alta. Ocazional, o diagramă de activitate este utilizată în locul unei organigrame. Diagramele nu sunt organigrame, în ciuda aspectului lor. Afișează diverse fluxuri, inclusiv unice, paralele, ramificate și concurente [5].

Fluxul de interacțiune al utilizatorului pentru verificare documentului este procesul ilustrat în figura 2.5 și reprezintă succesiunea acțiunilor realizate de utilizator în procesul de verificare a unui document în cadrul aplicației. Fluxul începe cu accesarea paginii dedicate și selectarea opțiunii de încărcare, urmată de alegerea și confirmarea fișierului. Sistemul verifică validitatea documentului, iar în cazul unui fișier valid este inițiată analiza, timp în care sunt afișate informații privind progresul și sunt restricționate acțiunile suplimentare pentru a preveni erori. După finalizarea procesării, utilizatorul primește răspunsul și poate descărca raportul generat, marcând astfel încheierea fluxului operațional.

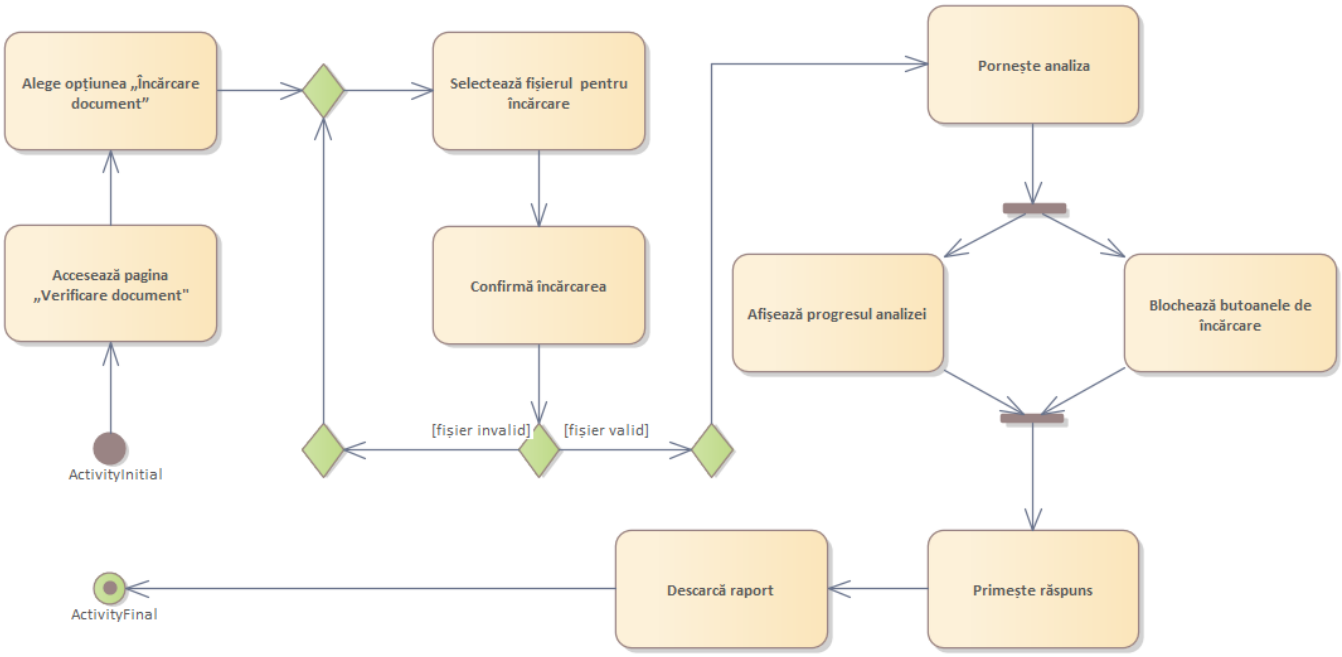

Figura 2.5 - Fluxul de interacțiune al utilizatorului pentru verificarea documentului

Diagrama din figura 2.6 prezintă etapele interne parcurse de sistem din momentul primirii cererii de analiză până la generarea raportului final. După validarea cererii, fișierul este salvat și supus proceselor de extragere și normalizare a textului. Sistemul interoghează baza de date pentru identificarea documentelor candidate, calculează nivelul de similitudine și generează raportul corespunzător. În situația unei cereri invalide, fluxul se întrerupe prin returnarea unei erori. Modelul evidențiază clar separarea dintre validarea inițială, procesarea conținutului și etapa finală de raportare, asigurând o înțelegere structurată a mecanismului intern de analiză.

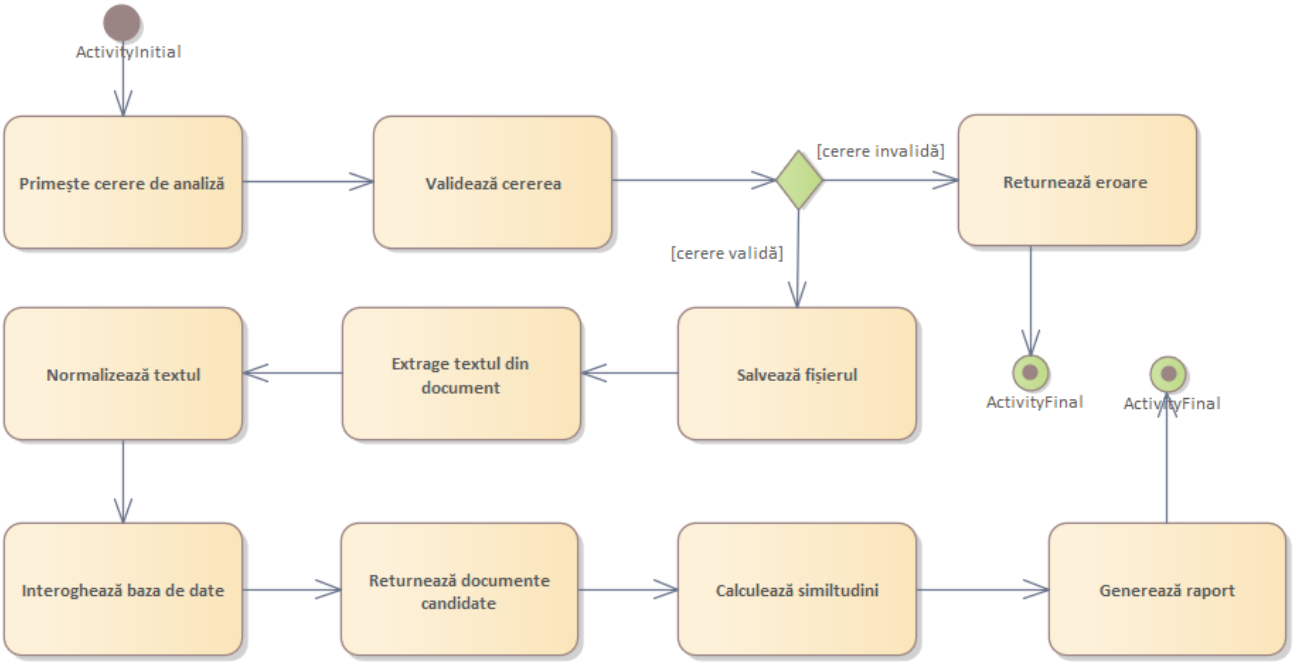

Figura 2.6 - Fluxul intern de procesare și generare a raportului de similitudine

În ansamblu, fluxurile prezentate evidențiază atât perspectiva utilizatorului, cât și mecanismele interne prin care aplicația în cauză procesează și analizează documentele. Corelarea etapelor de interacțiune cu operațiile de validare, prelucrare și generare a raportului demonstrează coerența procesului și delimitarea clară a responsabilităților în cadrul sistemului.

2.1.3 Stările de tranzacție a sistemului

Diagrama de stare este un tip de diagramă comportamentală utilizată pentru a descrie evoluția în timp a unui obiect sau a unei componente a sistemului, prin reprezentarea stărilor prin care aceasta poate trece și a tranzițiilor dintre ele. O stare reflectă o condiție sau o situație stabilă în care se află un element la un moment dat, iar tranzițiile indică schimbările determinate de evenimente, condiții sau acțiuni specifice. Diagrama permite modelarea reacției sistemului la diferite evenimente externe sau interne, evidențiind clar modul în care acesta își modifică comportamentul în funcție de context.

Prin utilizarea diagramei de stare se poate analiza comportamentul dinamic al unui proces, identificând stările inițiale, stările intermediare, punctele de decizie și stările finale. De asemenea, aceasta facilitează modelarea tratării excepțiilor și a ramificațiilor logice, contribuind la definirea unui flux coerent și previzibil al sistemului. În contextul unei aplicații informatice, diagrama de stare este fundamentală pentru înțelegerea ciclului de viață al unei cereri, al unui document sau al unei sesiuni de utilizator, oferind o perspectivă clară asupra modului în care sistemul reacționează la diferite acțiuni și condiții de execuție.

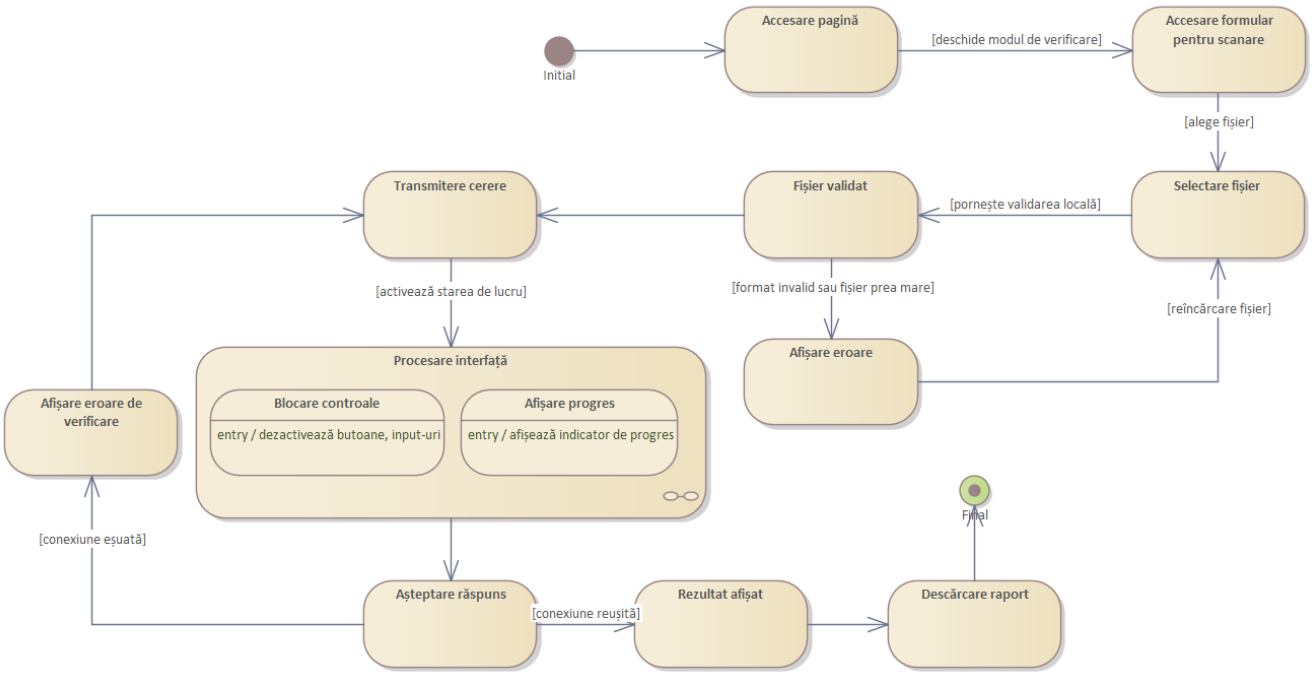

Figura 2.7 - Modelarea stărilor interfeței în procesul de verificare a documentului

Diagrama de stare reprezentată în figura 2.7 ilustrează succesiunea stărilor prin care trece componenta de interfață a aplicației în timpul procesului de verificare a unui document. Fluxul începe din

starea inițială, odată cu accesarea paginii și deschiderea formularului de scanare, continuând cu starea de selecție și validare a fișierului. În situația unui document invalid, sistemul tranzitează către starea de afișare a erorii, permițând reluarea procesului prin reîncărcare. Pentru un fișier valid, interfața intră în starea de procesare, în cadrul căreia sunt dezactivate controalele și este afișat progresul analizei. După primirea răspunsului, aplicația trece în starea de afișare a rezultatului, oferind utilizatorului posibilitatea de descărcare a raportului, înainte de atingerea stării finale. Diagrama evidențiază clar tranzițiile dintre stări și condițiile care determină schimbarea acestora.

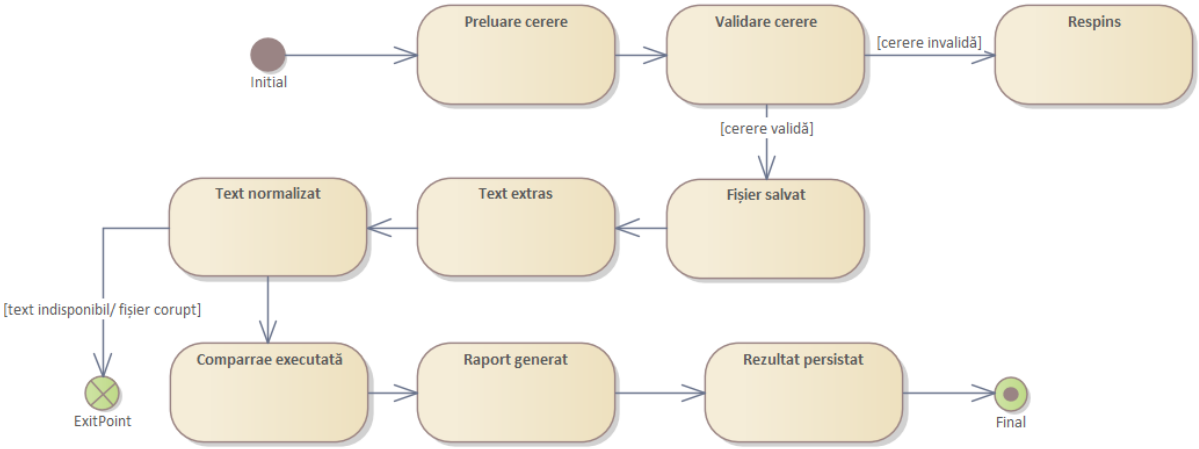

Figura 2.8 - Modelarea stărilor interne ale procesului de analiză a documentului

Figura 2.8 reprezintă o altă diagramă de stare care evidențiază ciclul de viață al unei cereri de analiză în cadrul aplicației. Procesul pornește din starea inițială, urmată de preluarea și validarea cererii. În cazul unei cereri invalide, sistemul tranzitează către starea „Respins”, încheind fluxul. Pentru o cerere validă, documentul este salvat, textul este extras și normalizat, iar ulterior se execută compararea conținutului. Dacă apar situații excepționale, precum text indisponibil sau fișier corupt, fluxul este întrerupt printr-un punct de ieșire alternativ. În scenariul favorabil, sistemul trece prin starea de generare a raportului și finalizează procesul prin persistarea rezultatului, ajungând în starea finală. Diagrama reflectă clar succesiunea stărilor și condițiile care determină tranziția dintre acestea.

Prin evidențierea stărilor, a condițiilor de trecere și a punctelor de întrerupere, acestea contribuie la o înțelegere riguroasă a comportamentului aplicației și la asigurarea coerenței funcționale în cadrul procesului de verificare a similitudinii.

2.1.4 Descrierea scenariilor de utilizare a aplicației

O diagramă de secvență este o componentă cheie a Limbajului de Modelare Unificat utilizată pentru a vizualiza interacțiunea dintre obiecte într-o ordine secvențială. Aceasta se concentrează pe ordinea și sincronizarea mesajelor sau evenimentelor schimbate între diferite elemente ale sistemului. O diagramă de secvență arată cum obiectele interacționează între ele în timp într-un sistem. Aceasta ajută la vizualizarea ordinii mesajelor și operațiilor dintre obiecte [6].

Prin utilizarea diagramei de secvență se poate modela comportamentul dinamic al sistemului, oferind o perspectivă detaliată asupra fluxului de date și a modului în care sunt coordonate operațiile interne. Aceasta este deosebit de utilă pentru descrierea scenariilor specifice, cum ar fi înregistrarea unui utilizator, autentificarea, inițierea unei analize sau generarea unui raport. În plus, diagrama facilitează identificarea dependențelor dintre componente, verificarea coerenței logice a proceselor și detectarea eventualelor erori de proiectare înainte de etapa de implementare.

Diagrama de secvență, reprezentată în figura A.1 a Anexei A, ilustrează interacțiunea dintre utilizator, interfața sistemului, componenta logică a aplicației și baza de date în procesul de creare a unui cont. Utilizatorul accesează interfața, completează datele necesare (nume, prenume, email, parolă, rol) și transmite cererea de înregistrare. Sistemul procesează informațiile primite, le validează și inițiază operația de creare a utilizatorului în baza de date. După confirmarea salvării, răspunsul este returnat către interfață și, implicit, către utilizator, finalizând astfel fluxul de autentificare inițială și integrarea acestuia în sistem.

Un alt scenariu important îl constituie cel de verificare a similitudinii documentului, scenariu reprezentat în figura A.2 din cadrul Anexei A. Această diagramă descrie succesiunea mesajelor schimbate între utilizator, interfața sistemului, componenta de procesare și baza de date în cadrul procesului de verificare a unui document. Sunt evidențiate două scenarii alternative: compararea cu un document de referință furnizat de utilizator și compararea cu baza de date internă. În primul caz, sistemul transmite datele pentru identificarea similitudinilor și returnează rezultatele obținute. În al doilea caz, documentul este salvat în baza de date, sunt identificate documentele candidate pentru comparație, se execută verificarea și se returnează rezultatele finale către utilizator. Diagrama subliniază clar fluxul de comunicare și responsabilitățile fiecărei componente implicate în procesul de analiză.

Secvențele analizate evidențiază modul în care componentele aplicației colaborează pentru a asigura atât gestionarea utilizatorilor, cât și procesul complet de verificare a similitudinii documentelor. Prin modelarea detaliată a schimbului de mesaje, se conturează o perspectivă clară asupra arhitecturii logice și a fluxului de date în cadrul sistemului „SimilarityChecker”.

2.1.5 Fluxurile de mesaje și legăturile dintre componentele sistemului

Diagramele de colaborare sunt utilizate pentru a arăta modul în care obiectele interacționează pentru a îndeplini comportamentul unui anumit caz de utilizare sau al unei părți a unui caz de utilizare. Împreună cu diagramele de secvență, colaborarea este utilizată pentru a defini și clarifica rolurile obiectelor care efectuează un anumit flux de evenimente al unui caz de utilizare. Acestea reprezintă principala sursă de informații utilizată pentru determinarea responsabilităților clasei și a interfețelor.

O colaborare este o colecție de obiecte și actori denumiți, cu legături care îi conectează. Aceștia colaborează la îndeplinirea unei sarcini. Colaborarea definește un set de participanți și relații care sunt semnificative pentru un set dat de scopuri. Fiecare obiect suportă parțial funcționalități emergente și este capabil să producă funcționalități de nivel înalt prin colaborare [7].

Diagrama de colaborare din figura 2.9 evidențiază interacțiunea structurală dintre utilizator, interfața aplicației, sistemul aplicației și baza de date în cadrul procesului de verificare a similitudinii. Mesajele numerotate indică ordinea logică a comunicării dintre componente, de la accesarea interfeței și încărcarea documentului, până la interogarea bazei de date și returnarea rezultatelor verificării. Diagrama pune accent pe relațiile dintre obiecte și pe responsabilitățile fiecărei componente în cadrul colaborării, oferind o perspectivă structurală asupra schimbului de mesaje.

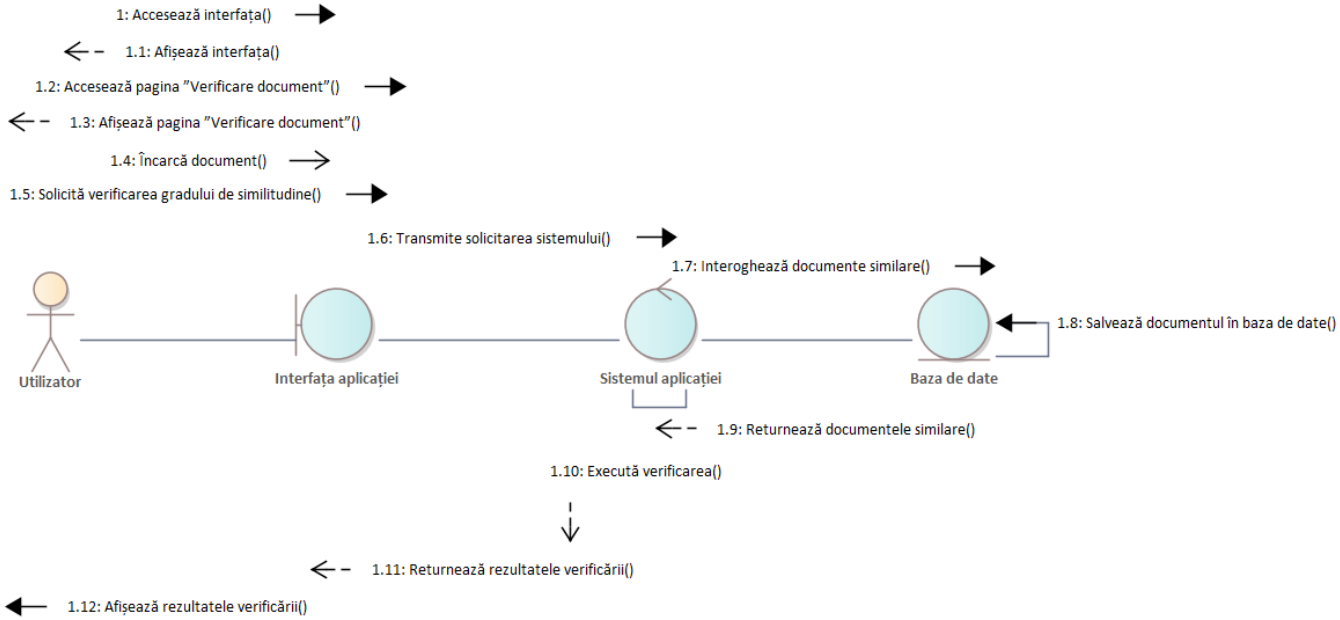

Figura 2.9 - Model de colaborare pentru procesul de verificare a similitudinii documentului

Figura 2.10 prezintă o diagramă de colaborare care scoate în evidență relația dintre aplicația anti-plagiat și funcționalitatea de verificare a plagiatului. Cazul general „Verificare plagiat" este detaliat prin colaborarea cu funcționalitatea specializată de detectare și afișare a secțiunilor plagiate. Diagrama evidențiază modul în care componentele sistemului cooperează pentru a realiza analiza documentului și pentru a furniza rezultate detaliate utilizatorului.

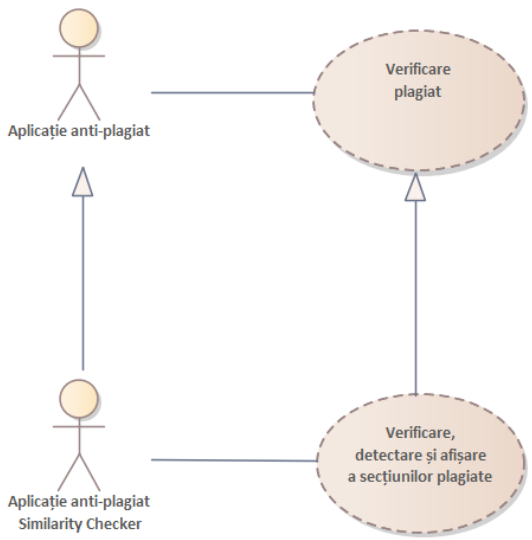

Figura 2.10 - Colaborarea funcțională în cadrul aplicației anti-plagiat

Figura 2.11 prezintă colaborarea dintre diferitele tipuri de utilizatori (Profesor, Student, Utilizator general) și funcționalitățile asociate procesului de verificare a similitudinii. Mesajele și relațiile ilustrate indică modul în care fiecare rol interacționează cu funcționalitatea generală de verificare, precum și specializările acesteia, în funcție de contextul utilizării. Diagrama subliniază distribuirea responsabilităților și adaptarea funcționalităților la cerințele fiecărei categorii de utilizatori.

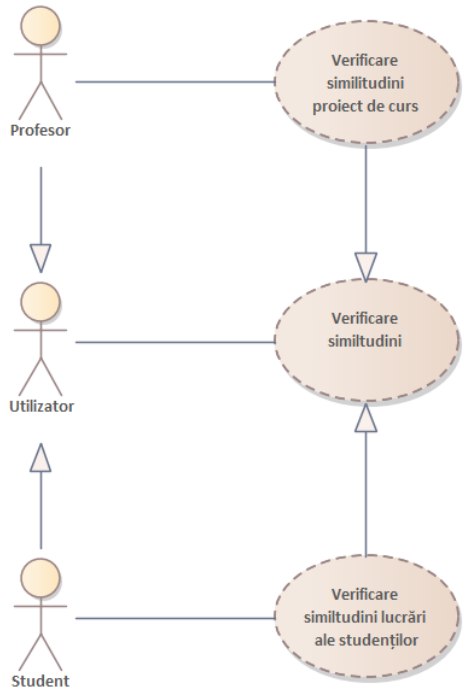

Figura 2.12 - Colaborarea între roluri și funcționalități în procesul de verificare a similitudinii

O diagramă de colaborare reprezintă un instrument important în modelarea comportamentului dinamic al sistemelor informatice, oferind o perspectivă clară asupra modului în care obiectele și componentele interacționează pentru realizarea unei funcționalități. Prin evidențierea relațiilor structurale

și a schimbului de mesaje dintre elemente, aceste diagrame facilitează înțelegerea distribuției responsabilităților și a mecanismelor de cooperare internă. Utilizarea lor contribuie la clarificarea arhitecturii logice a sistemului, la identificarea dependențelor dintre componente și la prevenirea eventualelor incoerențe în etapa de proiectare.

2.2 Descrierea structurală a sistemului

Descrierea structurală a sistemului evidențiază arhitectura statică a acestuia, prezentând elementele care îl compun și relațiile dintre ele, independent de execuția în timp. Spre deosebire de modelarea comportamentală, care se concentrează asupra fluxurilor și interacțiunilor dinamice, descrierea structurală urmărește organizarea internă a sistemului, structura logică a componentelor și modul în care acestea sunt interconectate. Această etapă stă la baza definirii arhitecturii aplicației, stabilirii responsabilităților fiecărei entități și asigurării coerenței structurale înainte de implementare.

Diagramele structurale descriu structura statică a elementelor unui sistem, adică modul în care un obiect se raportează la altul. Acestea prezintă elementele din sistem – clase, obiecte, pachete sau module, noduri fizice, componente și interfețe. Întrucât diagramele de structură reprezintă structura, acestea sunt utilizate pe scară largă în documentarea arhitecturii software a sistemelor software. De exemplu, diagrama componentelor descrie modul în care un sistem software este împărțit în componente și arată dependențele dintre aceste componente [8].

Descrierea structurii statice a sistemului se realizează prin intermediul diagramelor de clasă, care reprezintă principalul instrument de modelare structurală în cadrul metodei orientate pe obiect. Aceasta ilustrează clasele sistemului, atributele și metodele asociate, precum și relațiile dintre acestea, cum ar fi asocierea, agregarea, compoziția sau moștenirea. Diagrama claselor oferă o imagine clară asupra modelului de date și asupra organizării logice a aplicației, constituind baza pentru implementarea codului sursă și pentru definirea structurii bazei de date. Prin intermediul acestei diagrame se asigură o corelare directă între modelul conceptual și structura efectivă a sistemului informatic.

Diagrama componentelor răspunde de relațiile de dependență între componente și descrie organizarea sistemului la nivel modular, evidențiind componentele software majore și interfețele prin care acestea comunică. Această diagramă prezintă dependențele dintre module, serviciile furnizate și cele consumate, contribuind la înțelegerea arhitecturii globale a aplicației. Modelarea componentelor este importantă pentru proiectarea sistemelor complexe, deoarece permite identificarea clară a limitelor fiecărui modul și facilitarea mentenanței sau extinderii ulterioare a sistemului.

Procesul de modelare a echipamentelor mediului de implementare se reprezintă prin intermediul diagramelor de implementare, acestea ilustrează structura fizică a sistemului, prezentând distribuția componentelor software pe infrastructura hardware. Diagrama de implementare evidențiază nodurile de execuție (servere, stații de lucru, dispozitive mobile), conexiunile dintre acestea și artefactele software

instalate pe fiecare nod. Aceasta este inevitabilă pentru înțelegerea modului în care aplicația funcționează în mediul real, pentru evaluarea performanței și pentru planificarea resurselor necesare funcționării sistemului informatic.

2.2.1 Descrierea structurii statice a sistemului

Descrierea structurii statice a unui sistem poate fi efectuată prin intermediul unei diagrame de clasă. Aceasta reprezintă structura unui sistem prin afișarea claselor sale, a atributelor, metodelor și a relațiilor dintre ele. Clasele sunt reprezentate ca niște cutii, fiecare conținând câte trei compartimente pentru numele clasei, atribute și metode. Numele clasei este de obicei scris în compartimentul de sus al casetei clasei. Atributele, cunoscute și sub denumirea de proprietăți sau câmpuri, reprezintă membrii de date ai clasei. Acestea sunt listate în al doilea compartiment al casetei clasei și includ adesea vizibilitatea și tipul de date al fiecărui atribut. Metodele, cunoscute și sub denumirea de funcții sau operații, reprezintă comportamentul sau funcționalitatea clasei. Acestea sunt listate în al treilea compartiment al casetei clasei și includ vizibilitatea, tipul de returnare și parametrii fiecărei metode. Notațiile de vizibilitate indică nivelul de acces al atributelor și metodelor. Notațiile comune de vizibilitate includ:

- „+” pentru public;
- „-” pentru privat;
- „#” pentru protejat;
- „~” pentru vizibilitatea pachetului sau implicită [9].

Diagrama claselor afișată în figura 2.13 ilustrează structura statică a sistemului de verificare a similitudinii, evidențiind entitățile principale, atributele și operațiile acestora, precum și relațiile existente între ele. Clasa centrală „Sistem Similitudine” coordonează procesele de analiză și generare a rapoartelor, colaborând cu entități precum „Document”, „Analiză”, „Rezultat Similitudine” și „Raport”. Clasa „Utilizator” gestionează aspectele legate de autentificare și administrarea contului, iar interfața „ISistemSimilitudine” definește contractul funcțional pe care sistemul îl implementează, susținând principiul separării responsabilităților. Relațiile de asociere, agregare și implementare reflectă interdependențele dintre componente și delimitarea clară a rolurilor în cadrul arhitecturii aplicației.

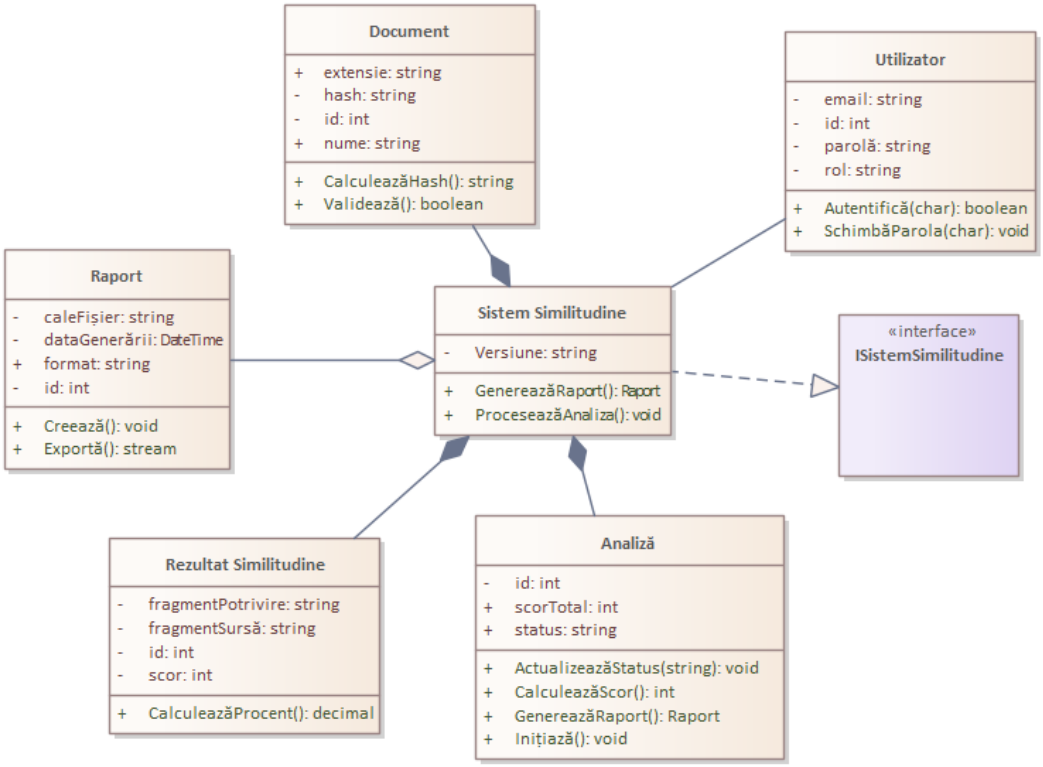

Figura 2.13 - Model structural al componentelor sistemului de verificare a similitudinii

Diagrama de clase oferă o reprezentare clară a arhitecturii structurale a sistemului, evidențiind organizarea entităților și responsabilitățile fiecăreia. Prin modelarea atributelor, metodelor și relațiilor dintre clase, se asigură o fundamentare riguroasă a implementării și o delimitare coerentă a componentelor în cadrul aplicației.

2.2.2 Relațiile de dependență între componentele sistemului

Diagramele de componente UML sunt utilizate în modelarea aspectelor fizice ale sistemelor orientate pe obiecte, care sunt folosite pentru vizualizarea, specificarea și documentarea sistemelor bazate pe componente, precum și pentru construirea sistemelor executabile prin inginerie directă și inversă. Diagramele de componente sunt, în esență, diagrame de clase care se concentrează pe componentele unui sistem și sunt adesea folosite pentru a modela vizualizarea implementării statice a unui sistem.

O diagramă a componentelor împarte sistemul propriu-zis aflat în dezvoltare în diverse niveluri înalte de funcționalitate. Fiecare componentă este responsabilă pentru un scop clar în cadrul întregului sistem și interacționează cu alte elemente esențiale doar pe baza principiului „nevoii de a cunoaște” [10].

Diagrama de componente din figura 2.14 evidențiază organizarea stratului de control din cadrul aplicației. Componenta centrală „Controllers” coordonează funcționalitățile expuse prin intermediul controlerelor specializate, precum DocumentsController, PlagiarismController, GlobalScanController, InternalScanController și PlagiarismReportController. Relațiile de dependență dintre componente reflectă

distribuirea responsabilităților și separarea clară a operațiilor specifice fiecărui tip de proces – încărcare document, analiză globală, scanare internă sau generare raport. Diagrama subliniază modularitatea stratului de prezentare și organizarea logică a punctelor de acces către funcționalitățile sistemului.

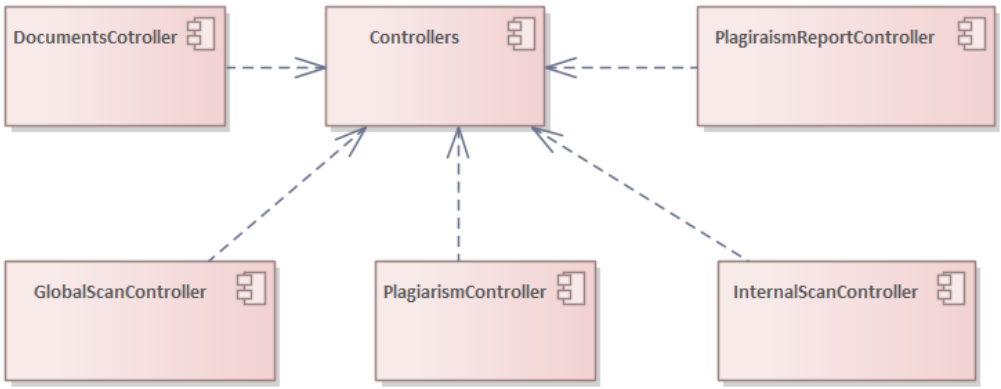

Figura 2.14 - Structura componentelor din stratul de control al aplicației

Figura 2.15 ilustrează structura arhitecturală a aplicației, evidențiind relațiile dintre componentele principale ale sistemului. Componenta centrală „SimilarityChecker" implementează interfața „ISimilarityChecker", respectând principiul programării orientate pe contract. Sistemul este organizat pe straturi distincte, incluzând componente pentru servicii (Services), acces la date (Data), controlere (Controllers), modele (Models) și interfața utilizatorului (SimilarityChecker.UI), care conține paginile aplicației. De asemenea, componenta „Bază de date" și structura tabelelor aferente reflectă nivelul de persistență al sistemului. Diagrama evidențiază clar separarea responsabilităților, dependențele dintre module și structura modulară a aplicației, susținând principiile unei arhitecturi coerente și scalabile.

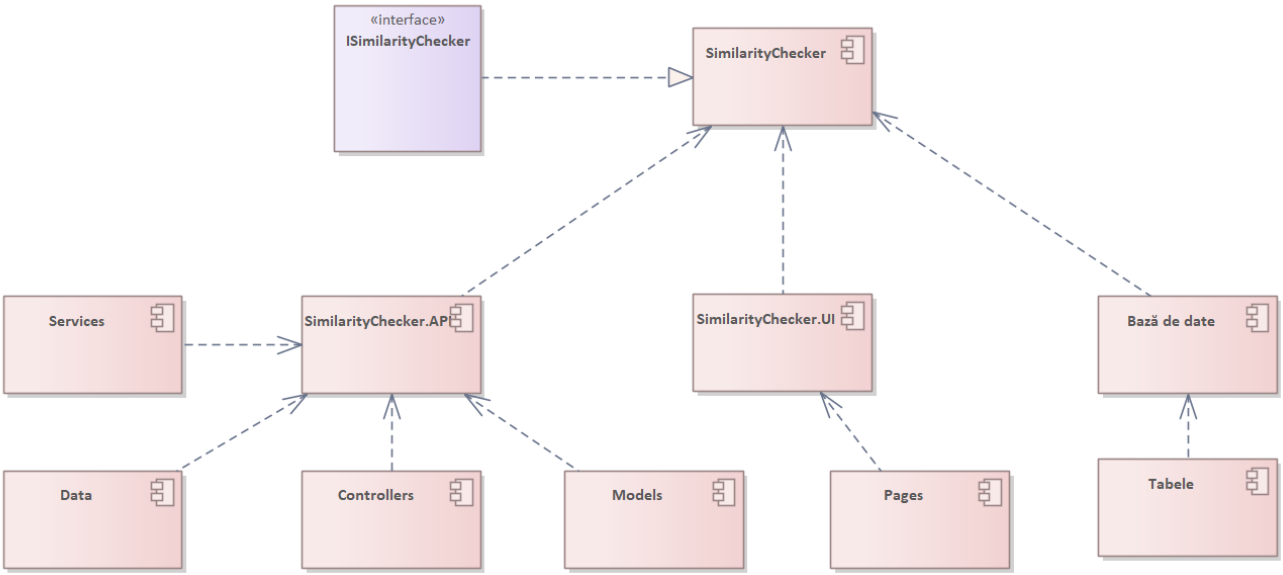

Figura 2.15 - Arhitectura componentelor aplicației „SimilarityChecker"

Diagrama componentelor oferă o reprezentare structurală a arhitecturii aplicației, evidențiind organizarea modulelor și relațiile de dependență dintre acestea. Prin modelarea clară a stratificării și a interacțiunilor dintre componente, se asigură o fundamentare solidă a implementării și o înțelegere coerentă a structurii sistemului la nivel macro-arhitectural.

2.2.3 Modelarea echipamentelor mediului de implementare

O diagramă de plasare UML este o diagramă care prezintă configurația nodurilor de procesare în timpul execuției și a componentelor care se află pe acestea. Diagramele de plasare sunt un tip de diagramă structurală utilizată în modelarea aspectelor fizice ale unui sistem orientat pe obiecte. Acestea sunt adesea folosite pentru a modela vizualizarea statică a implementării unui sistem (topologia hardware-ului).

Diagramele de plasare sunt importante pentru vizualizarea, specificarea și documentarea sistemelor integrate, client/server și distribuite, precum și pentru gestionarea sistemelor executabile prin inginerie directă și inversă. O diagramă de plasare este doar un tip special de diagramă de clase, care se concentrează pe nodurile unui sistem [11].

Diagrama de desfășurare ilustrată în figura 2.16 (de plasare) prezintă configurarea fizică a elementelor hardware și relațiile de comunicare dintre acestea în cadrul infrastructurii sistemului. Nodul central „PC” reprezintă stația de lucru care interacționează atât cu dispozitive periferice, precum tastiera, mouse-ul și imprimanta, cât și cu serverul prin intermediul unei conexiuni de rețea. Dispozitivele marcate cu stereotipul «device» indică echipamente hardware externe implicate în procesul de utilizare și operare a sistemului. Diagrama evidențiază distribuția fizică a resurselor și modul în care acestea sunt conectate pentru a asigura funcționarea aplicației în mediul de execuție.

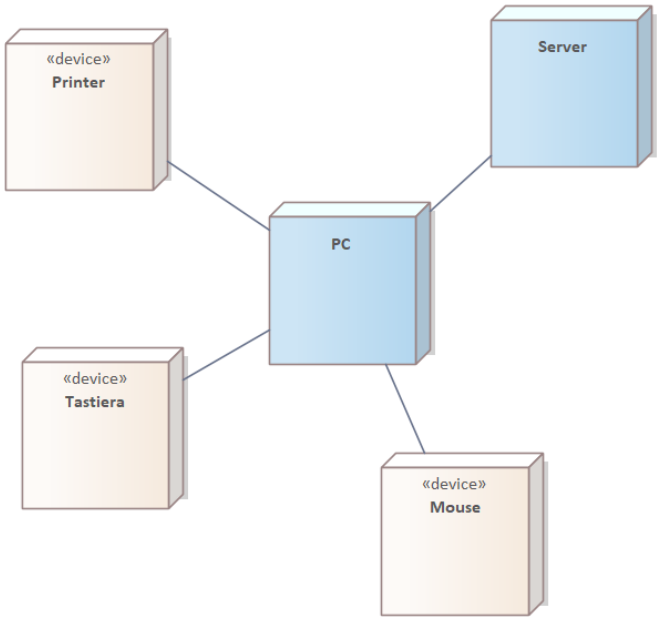

Figura 2.16 – Plasarea a componentelor sistemului în infrastructura hardware

Diagramele de desfășurare oferă o perspectivă asupra arhitecturii fizice a sistemului, evidențiind modul de distribuire a componentelor software pe infrastructura hardware. Prin reprezentarea nodurilor și a conexiunilor dintre acestea, se asigură o înțelegere clară a mediului de implementare și a relațiilor dintre resursele implicate în funcționarea aplicației.

3 REALIZAREA SISTEMULUI INFORMATIC

Dezvoltarea unui sistem informatic presupune un proces complex, care necesită o planificare detaliată și o abordare organizată pentru a asigura realizarea unei soluții care să răspundă cerințelor utilizatorilor și să îndeplinească scopurile stabilite. Este important ca întregul proces să fie bine coordonat, iar fiecare etapă să fie documentată și verificată pe parcurs. Acest proces implică nu doar proiectarea și implementarea funcționalităților fundamentale, dar și optimizarea acestora pentru a asigura performanță și ușurință în întreținere pe termen lung. Astfel, dezvoltarea trebuie să fie flexibilă, adaptabilă cerințelor viitoare, având în vedere posibilitatea extinderii aplicației pe măsură ce cerințele utilizatorilor evoluează.

În primul rând, alegerea tehnologiilor care vor sprijini dezvoltarea sistemului reprezintă un pas important. Tehnologiile selectate trebuie să fie compatibile cu cerințele proiectului, iar alegerea unui cadru de dezvoltare robust pune bazele realizării unei aplicații de încredere. Tehnologiile alese trebuie să permită o integrare rapidă a funcționalităților și să sprijine dezvoltarea rapidă a aplicației. În general, procesul de selecție a tehnologiilor ia în considerare factori precum performanța, ușurința în utilizare și capacitatea de a integra diverse componente ale sistemului. Alegerea corectă a tehnologiilor poate contribui semnificativ la succesul pe termen lung al proiectului, asigurându-se că soluțiile implementate sunt scalabile și ușor de întreținut.

După stabilirea tehnologiilor, se trece la proiectarea arhitecturii aplicației, care definește structura sistemului. Arhitectura aleasă trebuie să asigure scalabilitatea aplicației și să permită o separare clară a responsabilităților între diferitele componente, facilitând întreținerea și extinderea acesteia pe măsură ce sunt implementate noi funcționalități. Aceasta include definirea unui flux eficient al datelor între backend și frontend, astfel încât fiecare modul să își îndeplinească rolul fără a compromite performanța sistemului. În această etapă se stabilesc interacțiunile dintre modulele aplicației, precum și fluxurile de date, având în vedere necesitatea unei comunicări fluide între acestea.

Un alt aspect important este proiectarea și dezvoltarea bazei de date. Modelul de date trebuie să susțină procesarea și stocarea informațiilor într-un mod organizat, asigurând integritatea acestora. Este esențial ca baza de date să fie optimizată pentru performanță, având în vedere că aplicația va manipula un volum mare de date. Totodată, trebuie implementat un sistem eficient de interogare a datelor, având în vedere impactul pe care volumul de date îl poate avea asupra performanței aplicației. Acest proces include stabilirea relațiilor dintre tabele și crearea unor indecși care să accelereze operațiile de căutare și stocare.

După stabilirea arhitecturii și a bazei de date, se trece la dezvoltarea componentelor backend și frontend. Partea de backend se ocupă cu logica de procesare a datelor și interacțiunea cu baza de date, fiind responsabilă de funcționarea corectă a proceselor de backend, cum ar fi autentificarea utilizatorilor și manipularea documentelor. În paralel, frontend-ul oferă interfața cu utilizatorul, iar dezvoltarea acestuia trebuie să fie axată pe crearea unei experiențe intuitive și clare pentru utilizator. Dezvoltarea celor două

componente trebuie să fie coordonată pentru a asigura o integrare corectă și o experiență de utilizator coerentă, fără întreruperi în fluxul de lucru.

În procesul de realizare, testarea aplicației joacă un rol important. Testele funcționale sunt cele care vor asigura că aplicația răspunde corespunzător cerințelor utilizatorilor. Testarea funcționalităților include verificarea încărcării corecte a documentelor, procesarea acestora și generarea rapoartelor de similitudine. De asemenea, se va verifica interfața cu utilizatorul pentru a asigura o utilizare intuitivă și fără erori. Testele vor fi realizate pe scenarii variate de utilizare pentru a se asigura că aplicația funcționează corect în orice condiții și răspunde corespunzător cerințelor impuse. Aceste teste vor include și validarea performanței aplicației, pentru a se asigura că aceasta poate gestiona volume mari de date fără a compromite viteza de procesare.

3.1 Alegerea tehnologiilor și arhitecturii aplicației

Alegerea unor tehnologii adecvate este fundamentală în dezvoltarea unui sistem informatic, acestea influențând direct performanța, scalabilitatea, întreținerea și securitatea aplicației. În primul rând, tehnologiile selectate trebuie să fie compatibile cu cerințele proiectului și să sprijine procesarea rapidă a datelor și performanța optimă a aplicației. Tehnologiile bine alese permit dezvoltarea rapidă a aplicației și reduc complexitatea întreținerii pe termen lung.

Utilizarea .NET 8 pentru backend oferă performanță și stabilitate fiind un framework de programare gratuit, multiplatformă și open-source, care permite dezvoltarea diverselor aplicații web, mobile și cloud computing. .NET 8 nu este doar o versiune a framework-ului, ci mult mai mult decât atât. Redefinește modul în care aplicațiile software sunt construite și implementate, permițând dezvoltatorilor să îndeplinească cerințele în continuă evoluție ale calculului modern. Oferă performanțe îmbunătățite, diagnosticare și observabilitate îmbunătățite, suport extins între platforme, instrumente și integrare avansate, suport pe termen lung și multe altele. [12].

Blazor este un framework web open-source, susținut de Microsoft, care permite dezvoltarea interfețelor web interactive utilizând C# în loc de JavaScript. Acesta oferă posibilitatea de a construi aplicații web moderne și eficiente folosind tehnologii deja familiare dezvoltatorilor din ecosistemul .NET. Blazor Server reprezintă o implementare server-side a acestui framework, care permite scrierea de componente în C# și .NET care sunt executate pe server și interacționează cu clientul direct prin browser.

Componentele Blazor sunt gestionate pe server, iar interacțiunea cu partea clientului, cum ar fi actualizarea interfeței utilizatorului și gestionarea evenimentelor, se realizează printr-o conexiune în timp real, utilizând SignalR. SignalR este un component din cadrul ASP.NET care adaugă funcționalități de comunicare în timp real în aplicațiile web, oferind serverului posibilitatea de a trimite date către client fără a fi necesare solicitări din partea acestuia.

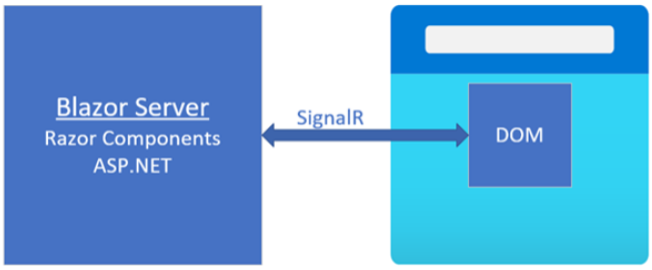

Figura 3.1 – Ilustrarea arhitecturii Blazor Server [13]

În figura 3.1, este ilustrată arhitectura Blazor Server, unde componentele Razor sunt executate pe server și interacționează cu DOM-ul browserului client prin intermediul conexiunii SignalR [13].

O altă tehnologie utilizată în realizarea sistemului este reprezentată de Entity Framework Core care reprezintă un instrument puternic de mapare obiect-relațională multiplatformă, dezvoltat de Microsoft pentru dezvoltatorii .NET. Acesta permite dezvoltatorilor să interacționeze cu bazele de date folosind obiecte .NET în loc să scrie interogări SQL, simplificând astfel accesul la date și făcându-l mai flexibil la construirea de aplicații .NET.

În figura 3.2 este ilustrat fluxul de lucru al Entity Framework Core care gestionează interacțiunea dintre aplicație și baza de date. La nivelul superior se află Layer-ul aplicației, care interacționează cu DbContext, componenta centrală ce conectează aplicația cu baza de date. DbContext utilizează Entity Classes pentru a reprezenta tabelele bazei de date, iar Modelul definește configurările și relațiile între acestea. Change Tracker urmărește modificările entităților, iar Query Pipeline se ocupă de pregătirea și trimiterea interogărilor către baza de date.

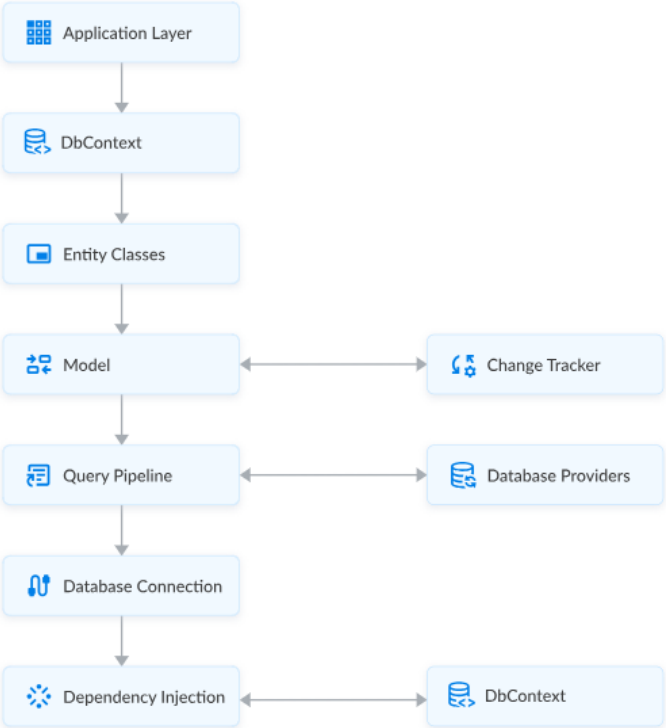

Figura 3.2 – Arhitectura Entity Framework Core [14]

Pe parcursul acestui flux, Database Providers sunt folosiți pentru a interacționa cu diverse tipuri de baze de date, iar Database Connection asigură conectivitatea efectivă. Dependency Injection (DI) permite injectarea DbContext în aplicație, facilitând gestionarea instanțelor și asigurând separarea preocupărilor. Acest flux detaliază modul în care EF Core optimizează accesul și manipularea datelor într-o aplicație, asigurând performanță și integritate [14].

SQL Server este un sistem de gestionare a bazelor de date relaționale dezvoltat de Microsoft, care este utilizat pentru stocarea, procesarea și securizarea datelor. În cadrul proiectului, s-a folosit SQL Server Management Studio, un instrument puternic care facilitează administrarea bazelor de date, permițând crearea, gestionarea și optimizarea bazelor de date într-un mod intuitiv. SSMS oferă o interfață grafică care ajută la administrarea serverelor SQL și la executarea interogărilor, gestionarea securității și asigurarea performanței aplicației. În figura 3.3 sunt ilustrate principalele componente ale SQL Server, incluzând managementul bazelor de date, monitorizarea performanței, și garantarea securității și conformității datelor, asigurându-se astfel o gestionare completă a datelor într-un mediu de afaceri dinamic.

Figura 3.3 – Principalele componente ale SQL Server [15]

Arhitectura SQL Server include un motor de baze de date care gestionează interogările și accesul la date, optimizând performanța și asigurând integritatea datelor. Acesta utilizează tehnici avansate pentru securizarea și recuperarea datelor, fiind important pentru aplicații care gestionează volume mari de informații, cum ar fi aplicațiile de linie de afaceri sau cele de tip enterprise. Desigur, SQL Server beneficiază de Database Providers pentru a asigura accesul la diverse platforme de baze de date și permite o integrare rapidă în medii de lucru moderne, de la servere locale la soluții în cloud.

Swagger este un framework open-source folosit pentru documentarea și testarea API-urilor, care facilitează interacțiunea dintre frontend și backend. În cadrul proiectului, Swagger a fost utilizat pentru a documenta API-ul aplicației și pentru a asigura o integrare calitativă între componentele frontend și

backend. Utilizând Swagger, s-a creat o interfață vizuală care permite dezvoltatorilor să testeze rapid endpoint-urile API, fără a fi necesar să scrie cod suplimentar pentru validarea acestora.

Swagger permite descrierea completă a fiecărui endpoint, inclusiv a metodelor HTTP utilizate, a parametrilor de intrare și a formatelor de răspuns, ajutând la menținerea unui control clar asupra funcționalităților expuse. Această documentație clară și interactivă a fost un instrument de bază pentru a asigura o comunicare eficientă între echipa de dezvoltare și utilizatorii API-ului, reducând timpul necesar pentru integrarea funcționalităților și facilitând depanarea eventualelor erori. În plus, utilizarea Swagger înseamnă și testarea rapidă a API-urilor într-o interfață simplă, ceea ce a permis verificarea performanței și corectitudinii fiecărei rute și asigurarea că aplicația răspunde corespunzător cerințelor utilizatorilor.

În cadrul proiectului dezvoltat, arhitectura aleasă se bazează pe principiile Clean Architecture, care pune accent pe separarea clară a responsabilităților între diferitele componente ale aplicației. Aceasta asigură o structură modulară, ușor de întreținut și extins, fiind importantă pentru dezvoltarea unui sistem fiabil și scalabil. Arhitectura Clean Architecture facilitează implementarea unor aplicații care pot evolua și se pot adapta rapid la cerințele noi, fără a compromite performanța sau integritatea sistemului existent.

Arhitectura este împărțită în mai multe straturi care sunt independente de tehnologiile utilizate. Aceste straturi sunt: stratul de prezentare (Frontend), care include interfața utilizatorului, stratul de aplicație (Backend), care gestionează logica de business și procesarea datelor, stratul de infrastructură, care se ocupă cu accesul la baze de date și servicii externe (cum ar fi SQL Server și Swagger), și stratul de domeniu, care conține entitățile și regulile de business. Fiecare strat are roluri bine definite și interacționează între ele prin metode standardizate, iar acest lucru este ilustrat și în imaginea din figura 3.4, care descrie fluxul între straturi: Prezentare, Domeniu și Date.

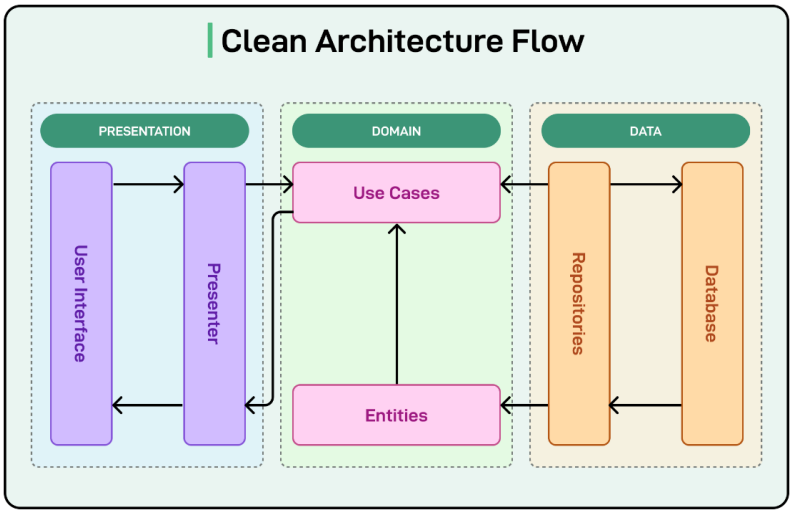

Figura 3.4 – Arhitectura Clean Architecture [16]

Această separare clară a componentelor permite o independență de tehnologie, deoarece dezvoltatorii pot schimba sau înlocui componentele fără a afecta restul aplicației, ceea ce asigură flexibilitate și adaptabilitate. În plus, arhitectura Clean Architecture susține testabilitatea, fiecare strat

putând fi testat independent, facilitând implementarea de teste unitare și de integrare. De asemenea, aceasta susține scalabilitatea și întreținerea aplicației pe termen lung, permițând adăugarea de noi funcționalități fără a afecta stabilitatea sistemului existent.

Alegerea tehnologiilor și arhitecturii în cadrul proiectului a fost fundamentată pe necesitatea de a asigura un sistem robust, scalabil și ușor de întreținut. Utilizarea ASP.NET Core pentru backend și Blazor Server pentru frontend, combinate cu arhitectura Clean Architecture, garantează separarea clară a responsabilităților, facilitând astfel dezvoltarea și testarea independentă a fiecărei componente. Aceste alegeri tehnologice și arhitecturale permit aplicabilitatea soluției într-un mediu dinamic, asigurând totodată o extindere ușoară a funcționalităților în viitor.

3.2 Realizarea sistemului informatic

Realizarea sistemului informatic SimilarityChecker a început cu o planificare detaliată a cerințelor și tehnologiilor necesare pentru construirea unei aplicații eficace și fiabile. În acest context, a fost important să se stabilească o arhitectură modulară și flexibilă, care să permită extinderea și întreținerea ușoară pe termen lung. Procesul de dezvoltare a fost ghidat de necesitatea de a crea o aplicație scalabilă, care să asigure o experiență de utilizare intuitivă și calitativă pentru verificarea similitudinii între documente. Astfel, fiecare componentă a fost concepută pentru a lucra împreună într-o arhitectură clar definită, începând cu baza de date, continuând cu backend-ul și frontend-ul, și încheind cu testarea rigurosă a funcționalităților implementate.

Primul pas important în dezvoltarea aplicației a fost crearea bazei de date, care a fost construită folosind SQL Server Management Studio. Proiectarea bazei de date a presupus definirea tabelelor necesare pentru stocarea documentelor, utilizatorilor și rezultatelor analizei de similitudine. Fiecare document încărcat de utilizatori a fost asociat cu informații relevante, precum data încărcării și scorul de similitudine calculat. EF Core a fost utilizat pentru a gestiona interacțiunea cu baza de date, permițând crearea și aplicarea migrațiilor, precum și maparea entităților la tabelele bazei de date. Această abordare a asigurat o gestionare fermă a datelor și o structură coerentă care facilitează interogările și extinderea aplicației pe măsură ce aceasta evoluează.

Partea de backend a aplicației a fost dezvoltată folosind ASP.NET Core, care a asigurat crearea unei arhitecturi robuste și scalabile. Backend-ul este responsabil pentru logica de procesare a documentelor încărcate, aplicarea algoritmilor de similitudine și interacțiunea cu baza de date. Algoritmii implementați, precum Cosine Similarity și TF-IDF, permit calcularea unui scor de similitudine între documentele încărcate de utilizatori și cele deja stocate în sistem. De asemenea, Swagger a fost folosit pentru documentarea și testarea API-ului, asigurându-se că toate rutele și funcționalitățile sunt corect implementate și pot fi verificate eficient de către dezvoltatori și utilizatori.

Pentru frontend, am utilizat Blazor Server, care permite crearea unei interfețe interactive utilizând C#, integrând perfect frontend-ul cu backend-ul. Acest framework a fost ales datorită ușurinței în integrarea logicii de backend direct în aplicația frontend, eliminând nevoia de a scrie JavaScript pentru majoritatea funcționalităților. Interfața utilizator a fost gândită pentru a fi intuitivă și simplă, permițând utilizatorilor să încarce documente rapid, să vizualizeze rezultatele analizelor și să descarce rapoartele de similitudine într-un format accesibil. În plus, Blazor Server permite actualizări în timp real ale interfeței, astfel încât utilizatorii beneficiază de o experiență fluidă și fără întreruperi în procesul de verificare a similitudinii documentelor.

3.2.1 Proiectarea și crearea bazei de date

În cadrul proiectului „SimilarityChecker”, baza de date a fost realizată utilizând SQL Server, iar pentru gestionarea acesteia s-a folosit Entity Framework Core (EF Core). Această librărie permite interacțiunea eficientă cu baza de date, definirea entităților și gestionarea migrațiilor. Baza de date include tabelele necesare pentru stocarea documentelor încărcate, rezultatele analizei de similitudine, utilizatorii și raporatele generate.

Pentru a defini aceste entități în cod, s-a folosit EF Core pentru a crea clase care corespund tabelelor din baza de date. De exemplu, pentru a crea entitatea „Document”, s-a creat clasa „DocumentEntity”. Pentru a crea coloanele tabelului, clasei „DocumentEntity” i s-au atribuit anumite proprietăți.

```
public sealed class DocumentEntity
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid UserId { get; set; }
    public AppUserEntity User { get; set; } = null!;
    [MaxLength(260)]
    public string FileName { get; set; } = string.Empty;
    [MaxLength(20)]
    public string FileType { get; set; } = string.Empty;
    [MaxLength(64)]
    public string Sha256 { get; set; } = string.Empty;
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
    public int WordCount { get; set; }
    [MaxLength(10)]
    public string? Language { get; set; }
    public string ExtractedText { get; set; } = string.Empty;
    public List<SearchQueryEntity> SearchQueries { get; set; } = new();
    public List<MatchEntity> Matches { get; set; } = new();
    public List<ReportEntity> Reports { get; set; } = new();
}
```

În cadrul aplicației, DbContext a fost utilizat pentru a defini entitățile care corespund tabelelor din baza de date și pentru a gestiona interacțiunea între aplicație și baza de date. În interiorul clasei SmilarityCheckerDbContext s-au definit relațiile dintre entități și s-a configurat modul în care sunt stocate și gestionate datele în baza de date:

```
modelBuilder.Entity<DocumentEntity>()
    .HasMany(x => x.SearchQueries)
    .WithOne(x => x.Document)
    .HasForeignKey(x => x.DocumentId)
    .OnDelete(DeleteBehavior.Cascade);

modelBuilder.Entity<DocumentEntity>()
    .HasMany(x => x.Matches)
    .WithOne(x => x.Document)
    .HasForeignKey(x => x.DocumentId)
    .OnDelete(DeleteBehavior.Cascade);
```

După ce s-au creat și definit toate entitățile necesare aplicației, s-a gestionat interacțiunea dintre aplicație și baza de date prin intermediul DbContext. Acesta a permis maparea entităților la tabelele bazei de date și configurarea relațiilor dintre ele. Prin utilizarea Entity Framework Core, au fost implementate migrațiile pentru a crea și actualiza baza de date. Astfel, s-a obținut structura bazei de date care este ilustrată în diagrama din figura B.1 din componența Anexei B, care include tabelele existente la momentul de față în baza de date. Această structură reflectă modul în care datele sunt stocate și gestionate în aplicație, asigurându-se că toate informațiile sunt accesibile rapid și corect, iar interacțiunile cu baza de date sunt eficiente.

3.2.2 Realizarea componentei backend

În cadrul dezvoltării aplicației SimilarityChecker, structura backend-ului a fost organizată pentru a permite gestionarea utilă a tuturor componentelor necesare. Imaginea din figura 3.5 ilustrează clar structura proiectului backend, care include directoare și fișiere fundamentale pentru buna funcționare a aplicației.

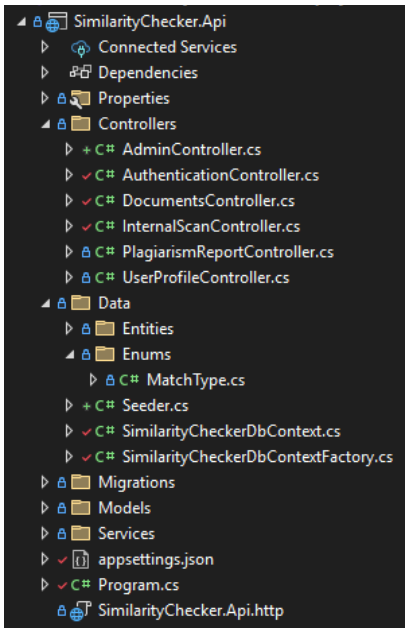

Figura 3.5 – Structura componentei backend a proiectului

În folderul „Controllers” se află clasele care gestionează rutele API și interacțiunile între frontend și backend. Controller-ele sunt responsabile pentru gestionarea cererilor HTTP, procesarea acestora și

trimiterea răspunsurilor corespunzătoare. Ele interacționează cu serviciile backend pentru a efectua operațiile de procesare a datelor. În exemplul de cod ce urmează, „DocumentsController” reprezintă un controller API pentru gestionarea documentelor încărcate în aplicația „SimilarityChecker”. Acesta oferă un endpoint pentru încărcarea documentelor, ce va fi accesat de către utilizatori printr-o cerere POST.

```
[ApiController]
[Authorize]
[Route("api/documents")]
public sealed class DocumentsController : ControllerBase
{
    private readonly SimilarityCheckerDbContext _db;
    private readonly TextExtractionService _textExtraction;
    private static readonly HashSet<string> AllowedExtensions =
new(StringComparer.OrdinalIgnoreCase)
    {".pdf", ".docx", ".txt"};
    private const long MaxFileSizeBytes = 20 * 1024 * 1024; // 20 MB
    public DocumentsController(SimilarityCheckerDbContext db, TextExtractionService
textExtraction)
    {
        _db = db;
        _textExtraction = textExtraction;
    }
    [HttpPost("upload")]
    [Consumes("multipart/form-data")]
    public async Task<ActionResult<DocumentUploadResponse>> Upload(
        [FromForm] DocumentUploadRequest request,
        CancellationToken ct)
    {
        if (request.File is null)
            return BadRequest("Fișierul este obligatoriu.");
        var currentUserId = GetCurrentUserId();
        var result = await ProcessSingleFileAsync(request.File, currentUserId, ct);
        if (!result.IsSuccess)
            return BadRequest(result.Message);
        return Ok(new DocumentUploadResponse
        {
            DocumentId = result.DocumentId!.Value,
            FileName = result.FileName,
            WordCount = result.WordCount ?? 0,
            Sha256 = result.Sha256 ?? string.Empty
        });
    }
```

În folderul „Migrations” al proiectului SimilarityChecker.Api se conțin migrațiile care sunt folosite pentru a aplica modificările structurii bazei de date, astfel încât să reflecte modificările din modelele aplicației. EF Core utilizează migrațiile pentru a crea și actualiza tabelele bazei de date fără a pierde datele existente. Una din migrații este migrarea InitialCreate.cs, care construiește schema bazei de date pentru aplicație. Această migrare include definirea tabelelor, relațiilor dintre ele și a constrângerilor asociate.

Migrarea definește tabelele ce vor fi create în baza de date, fiecare având coloane specifice pentru stocarea diferitelor tipuri de date.

```
migrationBuilder.CreateTable(
    name: "AppUsers",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
        Email = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: false),
```

```
        FirstName = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable:
false),
        LastName = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable:
false),
        PasswordHash = table.Column<string>(type: "nvarchar(max)", nullable: false),
        RolesCsv = table.Column<string>(type: "nvarchar(200)", maxLength: 200, nullable:
false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_AppUsers", x => x.Id);
    });
```

Între tabelele „Documents” și „AppUsers” există o relație de tipul „un utilizator poate avea mai multe documente”. Această relație este realizată prin utilizarea unei chei străine pe coloana „UserId” din tabelul „Documents”, care face legătura între documente și utilizatori.

În mod similar, există relații între „Matches” și „Documents”, respectiv „InternalMatches” și „Documents”, care sunt stabilite folosind chei străine. Aceste relații sunt configurate astfel încât să asigure integritatea datelor și să permită legătura între documentele comparate și rezultatele analizei de similitudine.

```
migrationBuilder.CreateTable(
    name: "Matches",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
        DocumentId = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
        OnlineSourceId = table.Column<Guid>(type: "uniqueidentifier", nullable: true),
        ComparedDocumentId = table.Column<Guid>(type: "uniqueidentifier", nullable: true),
        MatchType = table.Column<int>(type: "int", nullable: false),
        DocStart = table.Column<int>(type: "int", nullable: false),
        DocEnd = table.Column<int>(type: "int", nullable: false),
        SourceStart = table.Column<int>(type: "int", nullable: false),
        SourceEnd = table.Column<int>(type: "int", nullable: false),
        Score = table.Column<double>(type: "float", nullable: false),
        AlgorithmVersion = table.Column<string>(type: "nvarchar(max)", nullable: false),
        CreatedAtUtc = table.Column<DateTime>(type: "datetime2", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Matches", x => x.Id);
        table.ForeignKey(
            name: "FK_Matches_Documents_ComparedDocumentId",
            column: x => x.ComparedDocumentId,
            principalTable: "Documents",
            principalColumn: "Id");
        table.ForeignKey(
            name: "FK_Matches_Documents_DocumentId",
            column: x => x.DocumentId,
            principalTable: "Documents",
            principalColumn: "Id",
            onDelete: ReferentialAction.Cascade);
        table.ForeignKey(
            name: "FK_Matches_OnlineSources_OnlineSourceId",
            column: x => x.OnlineSourceId,
            principalTable: "OnlineSources",
            principalColumn: "Id",
            onDelete: ReferentialAction.Cascade);
    });
```

Pentru a optimiza performanța interogărilor asupra bazei de date, migrarea include crearea de indici pe coloanele cele mai frecvent accesate, precum „Email” din tabelul „AppUsers” și „Sha256” din tabelul „Documents”. Acești indici contribuie la îmbunătățirea vitezei de căutare și acces la date, mai ales în cazul interogărilor care filtrează sau sortează după aceste coloane. Acești indici permit aplicației să acceseze rapid informațiile și să reducă timpul de răspuns la interogările complexe.

```
migrationBuilder.CreateIndex(
    name: "IX_AppUsers_Email",
    table: "AppUsers",
    column: "Email",
    unique: true);
migrationBuilder.CreateIndex(
    name: "IX_Documents_Sha256",
    table: "Documents",
    column: "Sha256");
```

Metoda Down definită în migrarea bazei de date permite anularea modificărilor aplicate în cazul în care migrarea trebuie anulată. Astfel, această metodă șterge tabelele create și menține consistența bazei de date în cazul unei erori sau al unei modificări ulterioare. Utilizarea acestei metode asigură că baza de date poate reveni într-o stare anterioară, fără a compromite integritatea datelor.

```
migrationBuilder.DropTable(
    name: "InternalMatches");

migrationBuilder.DropTable(
    name: "Matches");
```

În cadrul aplicației „SimilarityChecker”, entitățile care corespund tabelelor bazei de date sunt definite în directorul „Models”. Aceste entități reprezintă obiectele care sunt stocate în baza de date și care conțin informațiile necesare pentru funcționarea aplicației. De exemplu, entitatea „DocumentUploadResponse”, definită mai jos, este un model care conține informațiile ce sunt returnate utilizatorilor după ce un document a fost încărcat și procesat cu succes. Acesta include ID-ul documentului, numele fișierului, numărul de cuvinte din document și hash-ul documentului, folosit pentru verificarea integrității acestuia.

```
public sealed class DocumentUploadResponse
{
    public Guid DocumentId { get; set; }
    public string FileName { get; set; } = string.Empty;
    public int WordCount { get; set; }
    public string Sha256 { get; set; } = string.Empty;
}
```

În codul din Anexa C, „InternalScanService” este un serviciu care implementează interfața IInternalScanService și este responsabil pentru procesul de scanare internă a documentelor. Acesta primește un obiect SimilarityCheckerDbContext în constructor, care este folosit pentru interacțiunea cu baza de date.

DbContext asigură accesul la entitățile din aplicație și permite efectuarea operațiunilor de citire, salvare și actualizare a datelor.

Metoda „StartAsync” este responsabilă pentru inițierea procesului de scanare a unui document, identificat prin documentId. Aceasta verifică dacă documentul există în baza de date, utilizând „DbContext” pentru a interoga tabela „Documents”. Dacă documentul nu este găsit, se aruncă o excepție, iar procesul de scanare nu va începe. Această metodă este esențială pentru validarea documentului înainte de a trece la compararea acestuia cu alte documente din sistem.

```
public async Task<InternalScanStartResponse> StartAsync(Guid documentId, double threshold,
CancellationToken ct)
{
    var doc = await _db.Documents.FirstOrDefaultAsync(d => d.Id == documentId, ct);
    if (doc is null)
        throw new InvalidOperationException("Documentul nu există în baza de date.");
}
```

După validarea documentului inițial, metoda preia toate celelalte documente din baza de date, excluzând documentul curent. Se extrag doar informațiile esențiale pentru comparație, cum ar fi „Id”, „FileName”, „ExtractedText” și „Sha256”. Aceste date vor fi utilizate pentru a calcula similitudinea între documentul curent și celelalte documente, ceea ce face posibilă identificarea documentelor care sunt similare din punct de vedere al conținutului.

```
var others = await _db.Documents
    .Where(d => d.Id != documentId)
    .Select(d => new { d.Id, d.FileName, d.ExtractedText, d.Sha256 })
    .ToListAsync(ct);
```

Înainte de a începe procesul de scanare pentru similitudini, se șterg rezultatele scanărilor anterioare pentru documentul respectiv. Acest lucru se realizează prin interogarea tabelei „InternalMatches” pentru a găsi toate înregistrările asociate documentului curent și eliminarea acestora. Această abordare asigură că datele vechi nu vor interfera cu noile rezultate ale scanării.

```
var oldInternal = _db.InternalMatches.Where(m => m.DocumentId == documentId);
_db.InternalMatches.RemoveRange(oldInternal);
```

În „List<InternalScanHiDto>()” , se calculează scorurile de similitudine între documentul curent și celelalte documente din sistem. Dacă documentele au același hash „Sha256”, scorul de similitudine este setat la 1.0, indicând un duplicat exact. În caz contrar, se compară textul extras din documente folosind „InternalSimilarityEngine.ComputeSimilarity”, un algoritm care calculează un scor bazat pe similitudinea textului. Dacă scorul obținut depășește pragul specificat (threshold), rezultatele sunt adăugate la lista hits și sunt salvate în tabela „InternalMatches” din baza de date.

```
var hits = new List<InternalScanHitDto>();
var corpusTexts = others
    .Select(x => x.ExtractedText)
    .Where(x => !string.IsNullOrWhiteSpace(x))
    .ToList();
foreach (var other in others)
{
    double score;

    if (!string.IsNullOrWhiteSpace(doc.Sha256) && doc.Sha256 == other.Sha256)
    {
        score = 1.0;
    }
    else
    {
        var source = doc.ExtractedText ?? "";
        var target = other.ExtractedText ?? "";
        if (string.IsNullOrWhiteSpace(source) || string.IsNullOrWhiteSpace(target))
            continue;
        score = InternalSimilarityEngine.ComputeSimilarity(source, target, corpusTexts);
    }
    if (score >= threshold)
    {
        hits.Add(new InternalScanHitDto
        {
            ComparedDocumentId = other.Id,
            ComparedFileName = other.FileName,
            SimilarityScore = score
        });
        _db.InternalMatches.Add(new InternalMatchEntity
        {
            DocumentId = doc.Id,
            ComparedDocumentId = other.Id,
            Score = score,
            AlgorithmVersion = "internal-v2"
        });
    }
}
```

„InternalScanService” este serviciul complex care gestionează scanarea documentelor pentru similitudini interne, atât exacte cât și parafrazate, și generează rapoarte detaliate. Acesta interacționează cu baza de date pentru a stoca rezultatele și pentru a oferi utilizatorilor feedback privind similitudinile între documente.

Implementarea componentelor backend din proiectul propus asigură o gestionare robustă a documentelor și a procesului de comparație a acestora. Utilizarea .NET 8 și a arhitecturii Clean Architecture a permis o structură clară, cu un backend modular și scalabil. Funcționalitățile de încărcare a documentelor, compararea acestora și generarea rapoartelor de similitudine sunt implementate eficient, iar interacțiunea cu baza de date se face prin Entity Framework și DbContext, garantând integritatea și performanța aplicației. Aceste caracteristici contribuie la crearea unei aplicații fiabile, capabile să răspundă cerințelor utilizatorilor într-un mod rapid și precis.

3.2.3 Realizarea componentei frontend

Partea frontend a unei aplicații reprezintă partea vizibilă și interactivă cu care utilizatorii interacționează direct. Acestea includ toate elementele vizuale, cum ar fi paginile, formularele, butoanele și alte componente de UI (interfață cu utilizatorul), care permit utilizatorilor să efectueze acțiuni și să vizualizeze informațiile procesate de backend.

În contextul aplicației „SimilarityChecker”, frontend-ul este responsabil pentru gestionarea interfeței utilizator, permițând utilizatorilor să încarce documente, să vizualizeze rapoarte și să interacționeze cu sistemul. În cazul proiectului de față, frontend-ul este realizat folosind Blazor Server, un framework care permite construirea de aplicații web interactive utilizând C#, integrându-se perfect cu backend-ul și oferind o soluție completă în cadrul ecosistemului .NET. În figura 3.6 este ilustrată structura aplicației SimilarityChecker.UI, care conține directoare și fișiere esențiale pentru realizarea componentelor frontend, organizate astfel încât să permită o dezvoltare modulară și ușor de întreținut a aplicației.

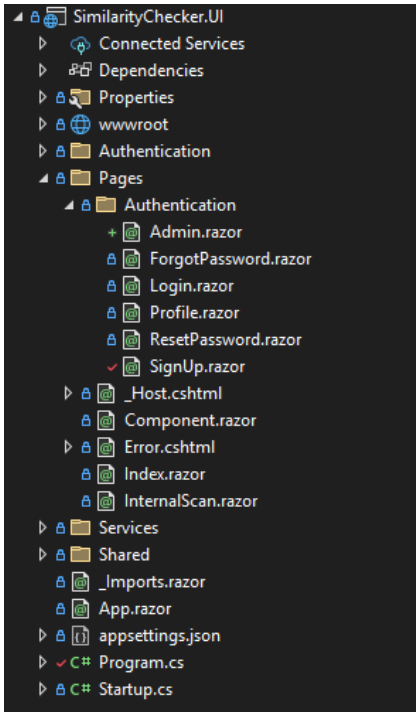

Figura 3.6 – Structura componentei frontend

În imaginea afișată în figura 3.7 este prezentată interfața principală a aplicației SimilarityChecker, care permite utilizatorilor să interacționeze cu sistemul pentru a analiza documentele și a obține rapoarte de similitudine. În partea stângă a interfeței se află un meniu de navigare, care include opțiuni pentru profilul utilizatorului, scanarea internă și verificarea documentelor. În centrul paginii, utilizatorii pot începe procesul de scanare, pot încărca documentele și pot accesa rapoartele detaliate de similitudine. Această interfață este concepută pentru a oferi o experiență de utilizare simplă și intuitivă, facilitând fluxul de lucru al utilizatorilor și optimizând accesul la funcționalitățile aplicației.

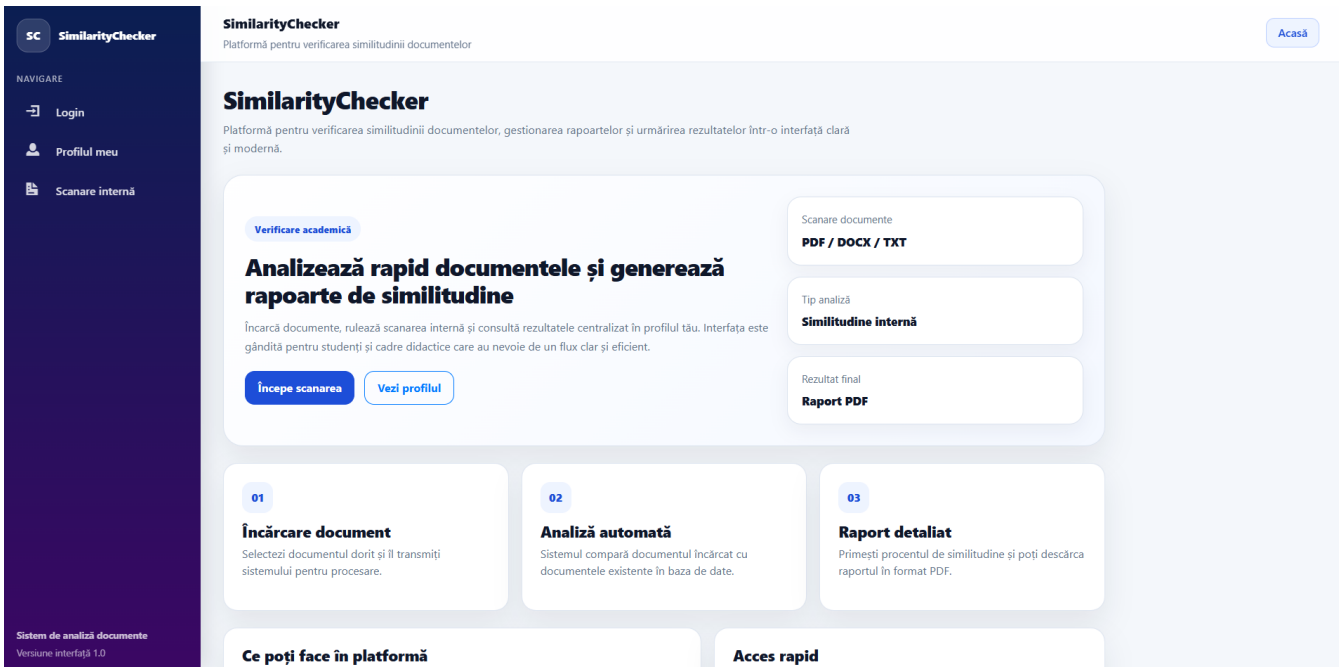

Figura 3.7 – Interfața principală a aplicației SimilarityChecker

Funcțiile implementate în componenta Blazor asigură că utilizatorii pot încărca fișiere, porni procesul de scanare, compara documente și vizualiza rapoartele de similitudine. Aceste funcționalități sunt de bază pentru o experiență de utilizator completă și eficientă. Funcția „HandlePrimaryFileChange” se ocupă de gestionarea fișierului selectat de utilizator. Când utilizatorul alege un fișier, funcția salvează fișierul selectat în variabila „_selectedFile” și resetează orice rezultat anterior. Acesta este un pas important în procesul de încărcare al documentului, pregătind fișierul pentru procesare. Funcția „ToggleCompareMode” permite utilizatorului să comute între două moduri de scanare: un mod simplu, în care se compară un document cu cele din baza de date, și un mod de comparație între două documente specifice. Această funcție modifică valoarea variabilei „_compareMode” și resetează starea aplicației, pregătind interfața pentru noul mod de comparație.

```
private Task HandlePrimaryFileChange(InputFileChangeEventArgs e)
{
    _error = null;
    _selectedFile = e.File;
    ClearResultsOnly();
    return Task.CompletedTask;
}

private void ToggleCompareMode()
{
    _compareMode = !_compareMode;
    ResetState();
}
```

„StartScan” este funcția care inițiază procesul de scanare, începând cu verificarea autentificării utilizatorului. Dacă utilizatorul nu este autentificat, se va deschide un modal de login. Dacă utilizatorul este

autentificat, funcția va determina ce tip de scanare trebuie să se execute (scanare simplă sau comparare între două documente), apelând fie „RunCompareAsync”, fie „RunScanAsync”.

```
private async Task StartScan()
{
    _error = null;
    var authState = await AuthStateProvider.GetAuthenticationStateAsync();
    var isAuthenticated = authState.User.Identity?.IsAuthenticated ?? false;
    if (!isAuthenticated)
    {
        _showLoginModal = true;
        return;
    }

    if (_compareMode)
        await RunCompareAsync();
    else
        await RunScanAsync();
}
```

„RunScanAsync” este funcția responsabilă pentru procesul complet de încărcare și scanare a unui document. După ce fișierul este selectat, fișierul este încărcat și trimis către backend pentru procesare. În această funcție se setează și fazele de procesare ale aplicației. Dacă apare vreo eroare în proces, aceasta este gestionată corespunzător, iar utilizatorul primește un mesaj de eroare.

```
private async Task RunScanAsync()
{
    if (_selectedFile is null)
        return;

    try
    {
        _busy = true;
        _error = null;
        _phase = UiPhase.Upload;
        await using var stream = _selectedFile.OpenReadStream(maxAllowedSize: 20 * 1024 *
1024);
        _uploadResult = await Api.UploadDocumentAsync(stream, _selectedFile.Name);

        _phase = UiPhase.Scan;
        _scanStart = await Api.StartInternalScanAsync(_uploadResult.DocumentId);

        _phase = UiPhase.Report;
        _report = await Api.GetInternalReportAsync(_uploadResult.DocumentId);
    }
    catch (Exception ex)
    {
        _error = ex.Message;
    }
    finally
    {
        _busy = false;
        _phase = UiPhase.None;
    }
}
```

Funcția „RunCompareAsync” se ocupă de procesul de comparare între două documente. Aceasta presupune încărcarea a două fișiere (documentul principal și documentul de referință) și trimiterea acestora

către backend pentru a calcula similitudinile între ele. Rezultatele comparării sunt salvate într-un raport care va fi disponibil utilizatorului. Dacă există erori în proces, acestea sunt gestionate corespunzător.

```
private async Task RunCompareAsync()
{
    if (_selectedFile is null || _referenceFile is null)
        return;

    try
    {
        _busy = true;
        _error = null;
        _phase = UiPhase.Upload;
        await using var primaryStream = _selectedFile.OpenReadStream(maxAllowedSize: 20 * 1024
* 1024);
        _primaryUploadResult = await Api.UploadDocumentAsync(primaryStream,
_selectedFile.Name);
        _phase = UiPhase.UploadReference;
        await using var referenceStream = _referenceFile.OpenReadStream(maxAllowedSize: 20 *
1024 * 1024);
        _referenceUploadResult = await Api.UploadDocumentAsync(referenceStream,
_referenceFile.Name);
        _phase = UiPhase.Scan;
        _compareReport = await Api.CompareTwoDocumentsAsync(
            _primaryUploadResult.DocumentId,
            _referenceUploadResult.DocumentId);
        _report = null;
        _scanStart = null;
        _uploadResult = null;
    }
    catch (Exception ex)
    {
        _error = ex.Message;
    }
    finally
    {
        _busy = false;
        _phase = UiPhase.None;
    }
}
```

Funcția „LoadReport” încarcă și vizualizează rapoartele generate după scanarea documentelor. În funcție de modul activ (comparare sau scanare simplă), funcția solicită raportul corespunzător și îl afișează utilizatorului. Această funcționalitate asigură că utilizatorii pot vizualiza rezultatele scanării într-un mod clar și organizat.

```
private async Task LoadReport()
{
    try
    {
        _busy = true;
        _phase = UiPhase.Report;
        _error = null;
        if (_compareMode)
        {
            if (_primaryUploadResult is null || _referenceUploadResult is null)
                return;
            _compareReport = await Api.CompareTwoDocumentsAsync(
                _primaryUploadResult.DocumentId,
                _referenceUploadResult.DocumentId);
        }
        else
```

```
        {
            if (_uploadResult is null)
                return;
            _report = await Api.GetInternalReportAsync(_uploadResult.DocumentId);
        }
    }
    catch (Exception ex)
    {_error = ex.Message;}
    finally
    {_busy = false;_phase = UiPhase.None;}
}
```

Funcția „DownloadPdf” permite utilizatorilor să descarce rapoartele generate sub formă de fișiere PDF. În funcție de modul activ (comparare între două documente sau scanare simplă), funcția generează linkul pentru descărcarea raportului corespunzător și redirecționează utilizatorul către acel link. Aceasta facilitează accesul rapid la raportul complet.

```
private void DownloadPdf()
{
    if (_compareMode)
    {
        if (_primaryUploadResult is null || _referenceUploadResult is null)
            return;

        var url = $"https://localhost:7260/api/internal-
scan/report/compare/{_primaryUploadResult.DocumentId}/{_referenceUploadResult.DocumentId}/pdf"
;
        Nav.NavigateTo(url, forceLoad: true);
        return;
    }

    if (_uploadResult is null)
        return;

    var singleUrl = $"https://localhost:7260/api/internal-
scan/report/{_uploadResult.DocumentId}/pdf";
    Nav.NavigateTo(singleUrl, forceLoad: true);
}
```

În cadrul aplicației realizate, componentele din directorul „Services” au rorlul de gestionarea logicii de business și facilitarea comunicării între frontend și backend. Aceste servicii sunt implementate pentru a trimite cereri API către server, a procesa datele și a returna răspunsurile corespunzătoare către utilizatori. Fiecare serviciu îndeplinește o funcționalitate specifică, de la gestionarea documentelor și detectarea plagiatului până la administrarea profilurilor utilizatorilor. În figura 3.8 este afișată structura acestiu director pentru aplicația realizată.

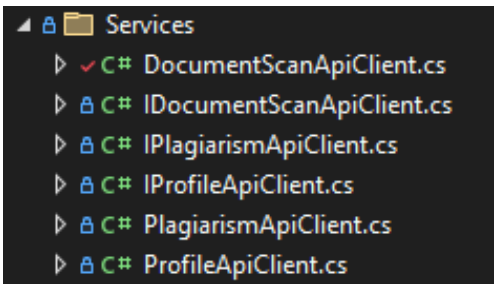

Figura 3.8 – Structura folderului Services

IDocumentScanApiClient este interfața care definește metodele utilizate de frontend pentru a interacționa cu backend-ul în scopul procesării documentelor. Funcțiile incluse în această interfață sunt:

- „UploadDocumentAsync” - permite încărcarea unui document (de tip fișier) către backend pentru procesare;
- „StartInternalScanAsync” - inițiază procesul de scanare internă a documentului încărcat pentru a verifica similitudinea cu alte documente;
- „GetInternalReportAsync” - permite obținerea unui raport detaliat despre procesul de scanare și rezultatele similitudinii documentului;
- „CompareTwoDocumentsAsync” - permite compararea a două documente între ele pentru a determina nivelul de similitudine dintre acestea.

Aceste metode permit încărcarea documentelor, inițierea scanării interne a documentelor pentru similitudini și obținerea rapoartelor generate.

```
namespace SimilarityChecker.UI.Services;

public interface IDocumentScanApiClient
{
    Task<DocumentUploadResponseDto> UploadDocumentAsync(Stream fileStream, string fileName,
CancellationToken ct = default);
    Task<InternalScanStartResponseDto> StartInternalScanAsync(Guid documentId,
CancellationToken ct = default);
    Task<InternalScanReportDto> GetInternalReportAsync(Guid documentId, CancellationToken ct =
default);
    Task<InternalScanReportDto> CompareTwoDocumentsAsync(Guid primaryDocumentId, Guid
referenceDocumentId, CancellationToken ct = default);
}
```

„IPlagiarismApiClient” este o interfață care permite frontend-ului să comunice cu backend-ul pentru verificarea plagiatului într-un document sau între două documente. Această interfață include metoda „CheckAsync” care efectuează verificarea plagiatului pentru un fișier principal și, opțional, un fișier de referință. Utilizând această metodă, aplicația poate verifica similitudinea documentelor în raport cu alte surse sau documente. Se acceptă fișiere de tip „IBrowserFile”, care sunt încărcate din frontend. Funcția returnează un obiect care conține răspunsul verificării plagiatului.

```
namespace SimilarityChecker.UI.Services;

public interface IPlagiarismApiClient
{
    Task<PlagiarismCheckResponse> CheckAsync(
        IBrowserFile mainFile,
        IBrowserFile? referenceFile,
        CancellationToken ct = default);
}
```

„IProfileApiClient” este o interfață care oferă funcționalități legate de gestionarea profilurilor utilizatorilor în aplicație. Aceasta conține metoda „GetMyProfileAsync” care permite obținerea informațiilor profilului utilizatorului curent. Această metodă returnează un obiect „UserProfileDto”, care conține date relevante despre utilizator, cum ar fi numele, email-ul și alte informații asociate contului său.

```
namespace SimilarityChecker.UI.Services
{
    public interface IProfileApiClient
    {
        Task<UserProfileDto> GetMyProfileAsync(CancellationToken ct = default);
    }
}
```

Componenta frontend a proiectului realizat asigură o interfață interactivă și eficace pentru utilizatori. Utilizând Blazor Server, aplicația permite o integrare completă între logica de backend și interfața utilizatorului, facilitând încărcarea documentelor, procesarea acestora și vizualizarea rapoartelor de similitudine. Funcționalitățile implementate, precum autentificarea utilizatorilor, compararea documentelor și gestionarea rapoartelor, sunt fundamentale pentru o experiență coerentă și fluidă. Designul responsiv și gestionarea adecvată a erorilor contribuie la îmbunătățirea accesibilității și utilizabilității aplicației pe diverse dispozitive, asigurând o interacțiune optimă în orice context.

1.3 Testarea și verificarea funcționalităților

În dezvoltarea soluției propuse, procesul de testare a avut rolul de asigurare a calității și fiabilității acesteia. Deși nu au fost realizate teste unitare automatizate, s-a acordat o atenție deosebită testării funcționale, care a fost realizată manual. Aceasta a implicat verificarea detaliată a fiecărei funcționalități, astfel încât aplicația să răspundă corect în scenarii de utilizare reale. Testele funcționale au fost realizate pentru a evalua comportamentul întregii aplicații, inclusiv interacțiunea cu utilizatorii, procesarea documentelor și generarea rapoartelor de similitudine.

În cadrul aplicației, procesul de încărcare a documentelor și scanare internă este implementat într-un mod interactiv. În imaginea din figura 3.9, sunt prezentate etapele interfeței utilizatorului în timpul încărcării și procesării unui document pentru detectarea similitudinii.

În primul pas, utilizatorul este invitat să încarce un fișier din sistemul său pentru a fi procesat. Odată încărcat, fișierul este prezentat în interfață cu detalii relevante, iar utilizatorul poate vizualiza starea încărcării fișierului. După ce fișierul a fost încărcat cu succes, aplicația trece la pasul următor, unde se lansează procesul de scanare. În acest pas, se începe analiza documentului, comparându-l cu cele existente în baza de date, iar utilizatorul poate vedea un indicator vizual al procesului, care arată că scanarea este în curs de desfășurare. De asemenea, utilizatorul poate opta pentru reîncărcarea raportului sau descărcarea unui raport PDF, dacă procesul a fost finalizat.

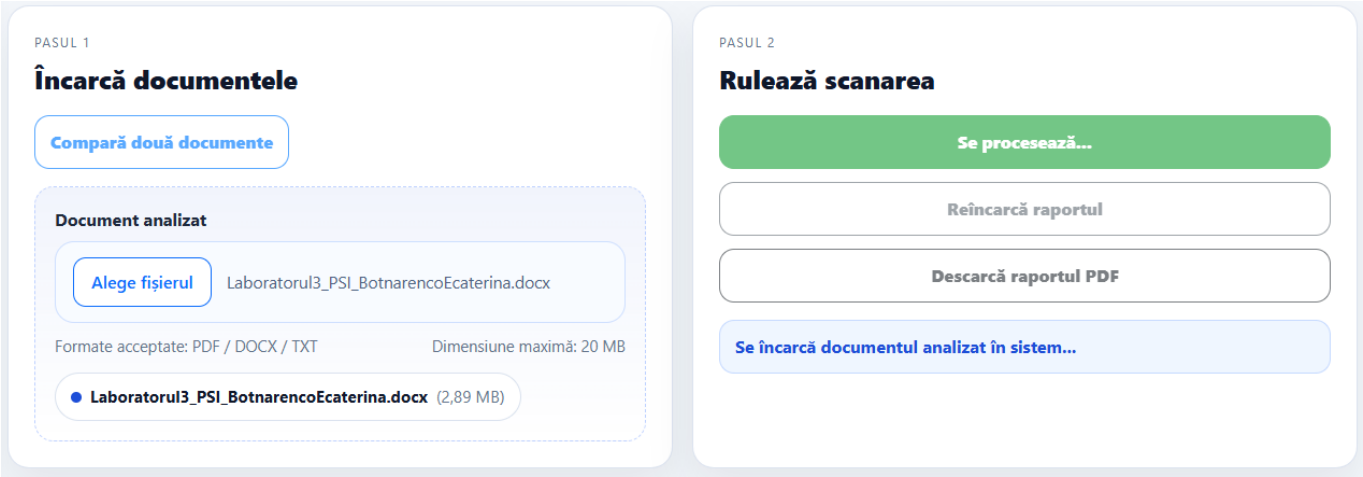

Figura 3.9 – Testarea pentru încărcarea unui document

După încheierea procesului de scanare, utilizatorul primește un mesaj de confirmare a succesului încărcării documentului, care afișează fișierul încărcat și numărul de cuvinte procesate. Acest feedback este important pentru utilizator, oferindu-i informații clare despre documentul său și statusul procesului de scanare. Mesajul de succes este afișat în imaginea din figura 3.10.

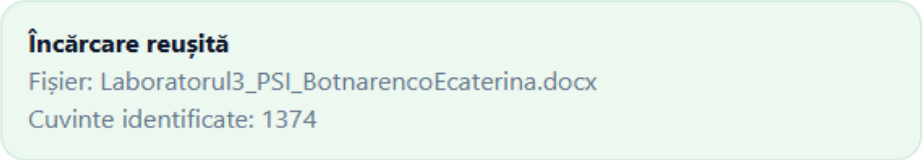

Figura 3.10 – Mesajul de confirmare a încărcării documentului

În imaginea prezentată în figura 3.11, aplicația ilustrează un scenariu în care un utilizator încearcă să încarce un fișier, dar fișierul selectat este gol. În acest caz, aplicația afișează un mesaj de eroare în partea de sus a paginii, semnalizând utilizatorului că fișierul nu poate fi procesat din cauza lipsei de conținut. Acest mecanism de validare este necesar pentru a preveni erorile ulterioare în timpul procesării documentelor. După ce fișierul gol este detectat, utilizatorul are posibilitatea de a selecta un fișier valid, care ulterior va fi încărcat în aplicație pentru a începe procesul de scanare.

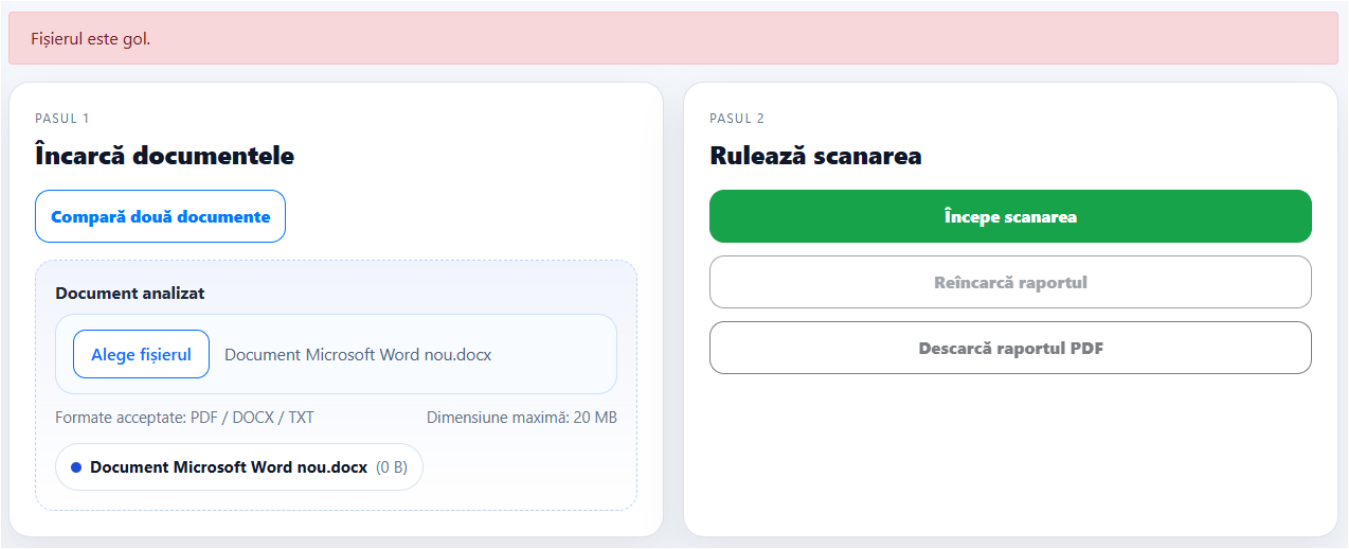

Figura 3.11 – Testarea pentru încărcarea unui fișier gol

În scenariul afișat în figura 3.12, utilizatorul a realizat o comparare între două documente. După ce ambele fișiere au fost încărcate cu succes în aplicație, procesul de scanare a început, iar utilizatorul a fost informat că documentul analizat și documentul de referință sunt în curs de procesare. Interfața afișează starea acestui proces și indică faptul că documentele sunt încărcate corect și pregătite pentru a fi comparate. După finalizarea procesului de comparare, aplicația confirmă succesul încărcării celor două documente, iar utilizatorul poate vizualiza rezultatele scanării, mesajul de succes este afișat în figura 3.13.

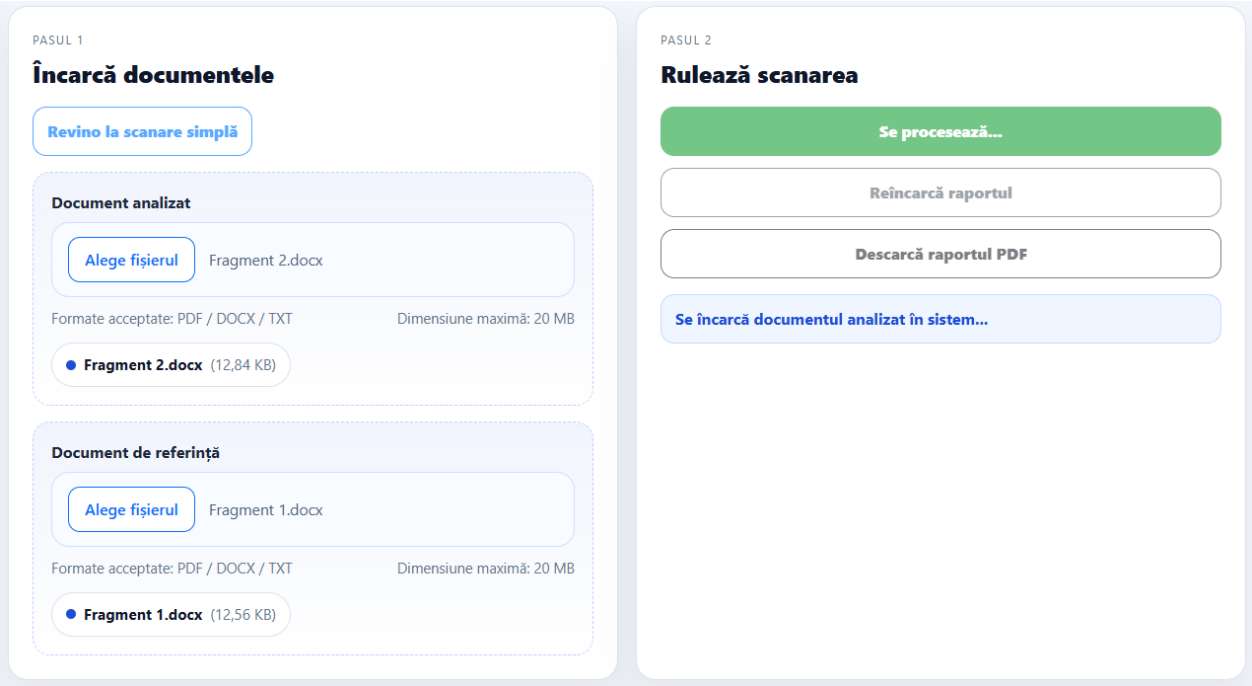

Figura 3.12 – Testarea pentru compararea a două documente

Mesajul de succes „Ambele documente au fost încărcate cu succes” asigură utilizatorul că operațiunea de încărcare a fișierelor a avut loc fără erori, iar documentele sunt acum disponibile în sistem pentru comparare. Acest proces permite utilizatorilor să compare două documente, să detecteze similitudini

și să obțină rapoarte detaliate despre conținutul acestora. Aceste funcționalități sunt esențiale în aplicațiile de verificare a plagiatului, asigurându-se că documentele sunt procesate corect și că utilizatorii primesc informațiile dorite.

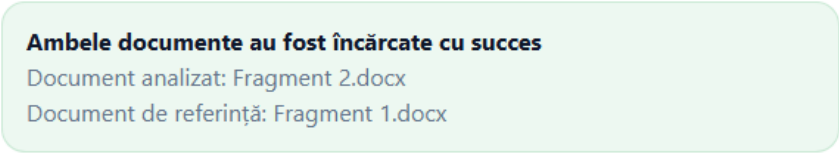

Figura 1.13 -Mesajul de confirmare a încărcării documentelor

Testele efectuate au confirmat că încărcarea documentelor și procesarea acestora pentru detectarea similitudinii se desfășoară fără erori, iar feedback-ul oferit utilizatorului, precum mesajele de succes și starea procesării, asigură o experiență de utilizare clară și transparentă. Aceste funcționalități importante garantează un flux de lucru eficient, în care utilizatorii pot accesa, analiza și compara documentele într-un mod simplu, dar robust, completând astfel procesul de verificare a similitudinii într-un cadru sigur și util.

4 DOCUMENTAREA PRODUSULUI REALIZAT

Documentarea produsului realizat este un pas important în realizarea unui produs software. Unul din documentele componente este ar fi ghidul de utilizare, acesta reprezentând un document care furnizează utilizatorilor toate informațiile necesare pentru a interacționa corect cu aplicația. Acesta oferă un set de instrucțiuni clare și pas cu pas, care ajută utilizatorul să înțeleagă cum să utilizeze funcționalitățile aplicației, să îndeplinească diverse sarcini și să rezolve posibile dificultăți pe parcursul utilizării. Ghidul de utilizare include explicații pentru fiecare secțiune a aplicației, descrierea proceselor de lucru și recomandări pentru utilizatorii mai puțin experimentați.

Un ghid de utilizare bine elaborat joacă un rol foarte important în procesul de integrare a unui sistem sau al unei aplicații într-un mediu de lucru. Acesta este crucial mai ales atunci când aplicația are multiple funcționalități sau atunci când utilizatorii nu sunt familiarizați cu instrumentele tehnologice implicate. În cazul aplicației „SimilarityChecker”, ghidul de utilizare devine un instrument valoros care ajută utilizatorul să navigheze printre opțiunile aplicației, să înțeleagă cum să încarce documentele, să ruleze procesele de scanare a similitudinii și să interpreteze rapoartele generate. Astfel, ghidul nu doar că oferă informații despre cum se folosește aplicația, ci și clarifică pașii necesari pentru a obține cele mai bune rezultate.

Un ghid de utilizare complet și bine structurat trebuie să fie ușor de urmărit și să includă secțiuni specifice care răspund întrebărilor frecvente ale utilizatorilor. De exemplu, ghidul ar trebui să detalieze procesul de înregistrare și autentificare, să explice cum să încarce fișiere, cum să înțeleagă rapoartele de similitudine și ce trebuie să facă în cazul în care întâmpină dificultăți. Este recomandat să se adauge capturi de ecran care să ilustreze fiecare pas al procesului, pentru a ajuta utilizatorii să înțeleagă mai ușor ce trebuie să facă. În plus, ghidul ar trebui să includă o secțiune de întrebări frecvente (FAQ), care să ajute utilizatorii să rezolve rapid orice problemă ar putea apărea.

Prin includerea unor exemple concrete și scenarii practice, ghidul permite utilizatorilor să aplice rapid cunoștințele dobândite, fără a fi necesar un suport suplimentar. În plus, un ghid bine structurat reduce semnificativ timpul necesar pentru formarea utilizatorilor, oferindu-le un punct de referință accesibil pentru a rezolva orice problemă sau întrebare pe care o pot avea în legătură cu aplicația. Astfel, utilizatorii pot deveni mai independenți în utilizarea aplicației, contribuind la o experiență mai fluentă și mai satisfăcătoare.

Pe lângă ajutorul direct oferit utilizatorilor, ghidul de utilizare reprezintă și un instrument important pentru dezvoltatori, deoarece poate clarifica modurile în care aplicația trebuie să funcționeze într-un context real de utilizare. În plus, documentarea completă a funcționalităților și a proceselor poate ajuta echipele de suport să rezolve rapid problemele utilizatorilor, întrucât sunt deja descrise clar pașii și erorile posibile. Un ghid bine realizat nu doar că îmbunătățește eficiența utilizatorilor, dar poate contribui la îmbunătățirea continuă a aplicației, încurajând feedback-ul și oferind o bază solidă pentru modificări viitoare ale aplicației.

4.1 Ghidul de utilizare a aplicației și fluxurile operaționale

Ghidul de utilizare a aplicației de față oferă o descriere detaliată a pașilor necesari pentru utilizarea aplicației, asigurându-se că utilizatorii înțeleg pe deplin cum să utilizeze fiecare funcționalitate disponibilă. De asemenea, sunt prezentate fluxurile operaționale care descriu procesele implicate în fiecare etapă a aplicației, de la autentificare și încărcare documente, până la analiza și compararea documentelor, astfel încât utilizatorii să poată naviga rapid și eficient prin aplicație.

Imaginea afișată în figura 4.1 prezintă interfața de autentificare a aplicației SimilarityChecker. Utilizatorul este invitat să introducă email-ul și parola pentru a accesa aplicația. Este oferită și opțiunea de a recupera parola în cazul în care aceasta a fost uitată, precum și posibilitatea de a crea un cont nou, în cazul în care utilizatorul nu are încă un cont. Acesta este primul pas în interacțiunea cu aplicația, asigurându-se că doar utilizatorii autorizați pot accesa documentele și rapoartele generate în cadrul aplicației. Interfața este simplă, oferind o experiență ușor de utilizat și directă.

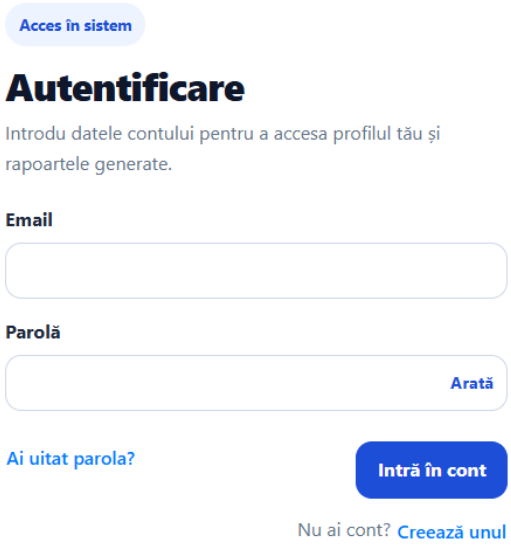

Figura 4.1 – Ecranul de autentificare

Figura 4.2 prezintă formularul de creare cont din cadrul aplicației. Utilizatorii sunt ghidați să completeze informațiile personale de bază, precum nume, prenume, email, parolă și rolul lor (de exemplu, student). După completarea formularului, utilizatorul poate crea un cont, iar aplicația va permite accesul la funcționalitățile de scanare și generare de rapoarte de similitudine. Acest pas este esențial pentru utilizatorii noi, care doresc să folosească aplicația și să își gestioneze documentele într-un mediu securizat.

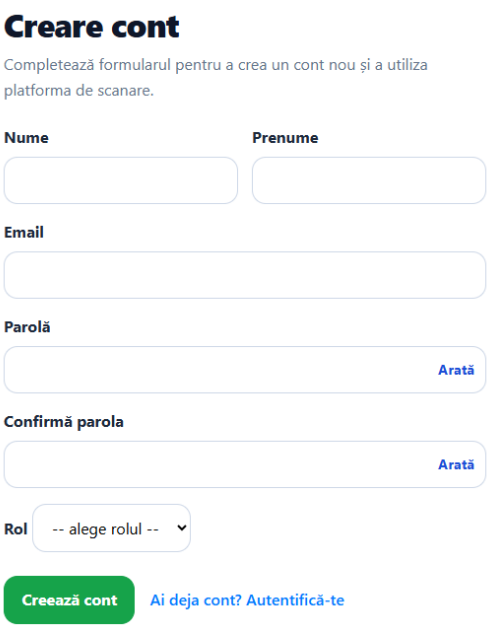

Figura 4.2 – Ecranul de creare cont

În imaginea din figura 4.3 este afișat profilul utilizatorului, care include informații precum numele complet, email-ul, rolul (în acest caz, Student) și numărul de rapoarte generate. Secțiunea de activitate recentă prezintă detalii despre ultimul document analizat, inclusiv numele fișierului și data procesării, iar utilizatorii pot vizualiza sau descărca rapoartele anterioare. De asemenea, există opțiunea de a încărca un document nou, facilitând accesul rapid la funcționalitățile aplicației. Acesta este un mod eficient de a urmări istoricul activității și de a avea un acces rapid la documentele și rapoartele generate.

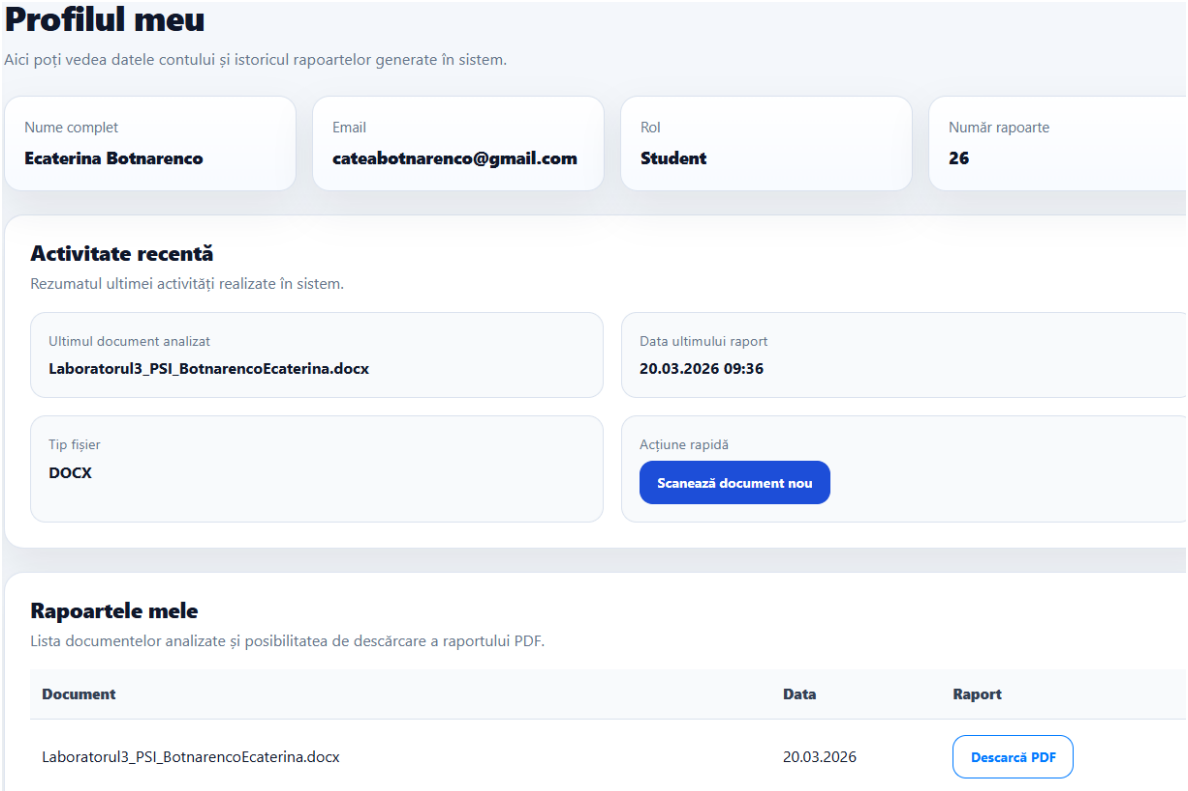

Figura 4.3 – Profilul utilizatorului

Figura 4.4 - ilustrează procesul de încărcare și scanare a documentelor. Utilizatorul a selectat un document pentru analiza similitudinii, iar aplicația indică starea procesului, cu mesajul „Se procesează...”. În această etapă, documentul este încărcat în sistem și procesat pentru a compara similitudinea cu alte documente din baza de date. Utilizatorul poate alege să reîncarce raportul sau să descarce raportul PDF odată ce scanarea este completă. Această interfață clară ajută utilizatorul să urmeze pașii necesari fără a fi necesară o intervenție suplimentară.

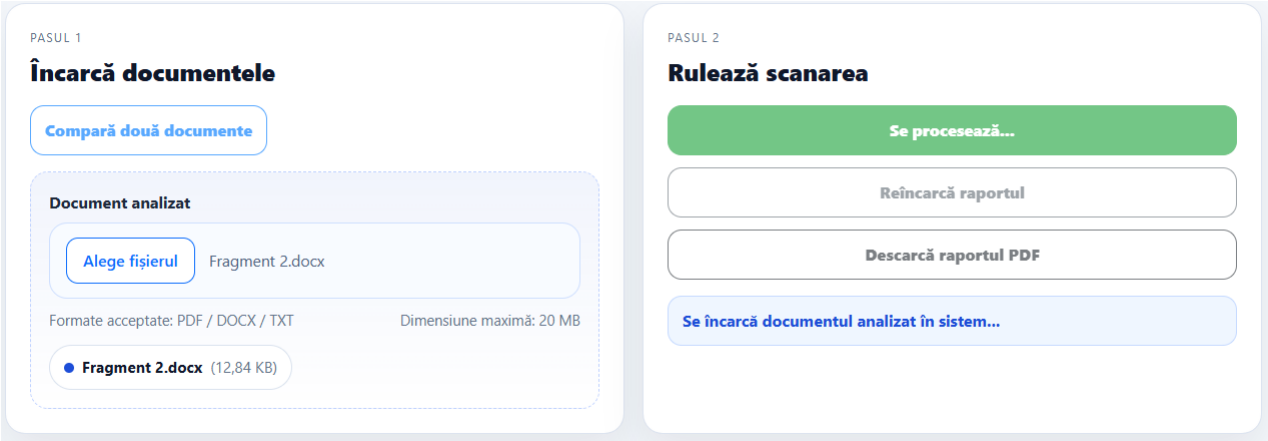

Figura 4.4 – Încărcarea și procesarea documentelor

În imaginea din figura 4.5 este prezentat rezultatul final al procesului de scanare, indicând procentajul de conținut plagiat detectat în documentul analizat, precum și scorurile de similitudine pentru documentele comparate. Aplicația afișează un rezumat vizual, evidențiind scorul de similitudine în procente (de exemplu, 47% de conținut plagiat) și afișează documentele cele mai similare. De asemenea, utilizatorii pot analiza detaliile comparării, precum fragmentele de text exacte sau parafrazate între documente, având acces rapid la comparațiile efectuate.

Figura 4.5 – Rezumatul gradului de similitudini

În figura D.1 din componența Anexei D este atașată imaginea unui raport detaliat generat de aplicație, care oferă o descriere completă a procesului de comparare a documentelor. Raportul include un rezumat al nivelului de similitudine, o listă cu documentele comparate și scorurile de similitudine pentru fiecare document, și fragmentele exacte sau parafrazate între documente. În plus, raportul este prezentat într-un format PDF, care poate fi descărcat și salvat de către utilizator pentru referințe ulterioare. Aceasta este o metodă convenabilă de a obține un raport formal al procesului de analiză a similitudinii, pe care utilizatorii îl pot utiliza pentru a-și documenta rezultatele și pentru a le împărtăși cu alții.

Ghidul de utilizare al aplicației și fluxurile operaționale din cadrul aplicației SimilarityChecker sunt fundamentale pentru o experiență de utilizare coerentă și eficientă. Procesul de autentificare, crearea unui cont și încărcarea documentelor pentru comparare sunt ușor de înțeles și de urmat, iar interfața intuitivă asigură un flux de lucru clar, fără complicații. De la încărcarea documentelor până la analiza similitudinii și generarea rapoartelor, fiecare etapă a fost gândită pentru a răspunde nevoilor utilizatorilor. În plus, funcționalitățile de feedback și opțiunile de descărcare a rapoartelor contribuie la o utilizare optimă a aplicației, îmbunătățind accesibilitatea și ușurința în utilizare. Astfel, fluxurile operaționale și ghidul de utilizare ajută la maximizarea eficienței aplicației și facilitează integrarea acesteia în activitățile utilizatorilor.

CONCLUZII

Lucrarea de față a avut ca scop dezvoltarea unei aplicații software pentru verificarea gradului de similitudine între documente, în vederea combaterea plagiatului academic. Începând cu analiza domeniului de studiu, s-au identificat provocările și soluțiile existente în domeniul educațional, subliniind importanța combaterii plagiatului prin utilizarea tehnologiilor moderne. De asemenea, au fost analizate platformele similare evidențiind importanța abordării bazate pe analiza semantică pentru identificarea plagiatului în formele sale cele mai subtile, cum ar fi parafrazările sau preluarea ideilor fără citare corespunzătoare.

În etapa de modelare și proiectare a sistemului, a fost descrisă arhitectura propusă, utilizând metoda OOD și tehnologiile UML pentru a structura clar și coerent componentele aplicației. Diagramele de cazuri de utilizare, activitate și stare au oferit o imagine detaliată a fluxurilor operaționale și a comportamentului aplicației, iar alegerea arhitecturii client-server a asigurat o separare eficientă a responsabilităților între frontend și backend. Astfel, designul a fost realizat pentru a facilita o extensibilitate și o întreținere ușoară a sistemului.

În implementarea efectivă a sistemului, a avut loc dezvoltarea funcționalităților principale, precum încărcarea documentelor, prelucrarea și compararea textului, utilizând algoritmi avansați precum TF-IDF și Cosine Similarity, dar și SimHash pentru optimizarea procesării volumelor mari de date. Testarea sistemului a asigurat validarea corectă a fiecărei funcționalități, iar procesul de generare a rapoartelor de similitudine a demonstrat eficiența și acuratețea soluției propuse.

În ceea ce privește documentarea produsului realizat, s-a detaliat procesul de utilizare a aplicației și fluxurile operaționale care facilitează utilizatorilor – profesori și studenți – o verificare rapidă și obiectivă a lucrărilor academice.

În concluzie, proiectul propus contribuie semnificativ la îmbunătățirea integrității academice, oferind o soluție performantă și ușor de utilizat pentru detectarea plagiatului. Tehnologiile implementate, precum analiza semantică, permit identificarea unor forme complexe de plagiat și aduc un aport valoros în mediul educațional, promovând transparența și corectitudinea în procesul de evaluare academică.

BIBLIOGRAFIE

[1] "Plagiarism | University of Oxford." Accessed: Feb. 19, 2026. [Online]. Available: https://www.ox.ac.uk/students/academic/guidance/skills/plagiarism

[2] "Ce este plagierea și cum se pedepsește. Tot ce trebuie să știi despre unul dintre cele mai răspândite fenomene din școli și universități - #diez." Accessed: Feb. 19, 2026. [Online]. Available: https://diez.md/2024/03/14/ce-este-plagierea-si-cum-se-pedepseste-tot-ce-trebuie-sa-stii-despre-unul-dintre-cele-mai-raspandite-fenomene-din-scoli-si-universitati/

[3] "Enterprise Architect | Brands of the World™ | Download vector logos and logotypes." Accessed: Mar. 24, 2026. [Online]. Available: https://www.brandsoftheworld.com/logo/enterprise-architect

[4] "Crearea unei diagrame de cazuri de utilizare UML - Asistență Microsoft." Accessed: Feb. 19, 2026. [Online]. Available: https://support.microsoft.com/ro-ro/topic/crearea-unei-diagrame-de-cazuri-de-utilizare-uml-92cc948d-fc74-466c-9457-e82d62ee1298

[5] "Diagrama activității UML: definiție și tutorial de creare." Accessed: Feb. 19, 2026. [Online]. Available: https://www.mindonmap.com/ro/blog/uml-activity-diagram/

[6] "Sequence Diagrams - Unified Modeling Language (UML)," GeeksforGeeks. Accessed: Feb. 19, 2026. [Online]. Available: https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams/

[7] "What is UML Collaboration Diagram?" Accessed: Feb. 19, 2026. [Online]. Available: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml-collaboration-diagram/

[8] "UML - Behavioral Diagram vs Structural Diagram." Accessed: Feb. 19, 2026. [Online]. Available: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/behavior-vs-structural-diagram/

[9] "UML Class Diagram," GeeksforGeeks. Accessed: Feb. 19, 2026. [Online]. Available: https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/

[10] "What is Component Diagram?" Accessed: Feb. 19, 2026. [Online]. Available: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/

[11] "What is Deployment Diagram?" Accessed: Feb. 19, 2026. [Online]. Available: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-deployment-diagram/

[12] S. Chauhan, ."Net 8 Features with Examples - What's new in .NET 8?" Accessed: Mar. 20, 2026. [Online]. Available: https://www.scholarhat.com/tutorial/net/new-in-net-8

[13] "Introduction to ASP.NET Core Blazor Server | Marathon Consulting." Accessed: Mar. 20, 2026. [Online]. Available: https://marathonus.com/about/blog/introduction-to-aspnet-core-blazor-server/

[14] Devart, “What is Entity Framework Core? Architecture, Features, and Workflow Examples,” Devart Software. Accessed: Mar. 20, 2026. [Online]. Available: https://www.devart.com/dotconnect/what-is-entity-framework-core.html

[15] “What is SQL Server and how does it work? | Quest.” Accessed: Mar. 20, 2026. [Online]. Available: https://www.quest.com/learn/what-is-sql-server.aspx

[16] ByteByteGo, “Clean Architecture 101: Building Software That Lasts.” Accessed: Mar. 20, 2026. [Online]. Available: https://blog.bytebytego.com/p/clean-architecture-101-building-software

ANEXA A

Diagrame de secvență

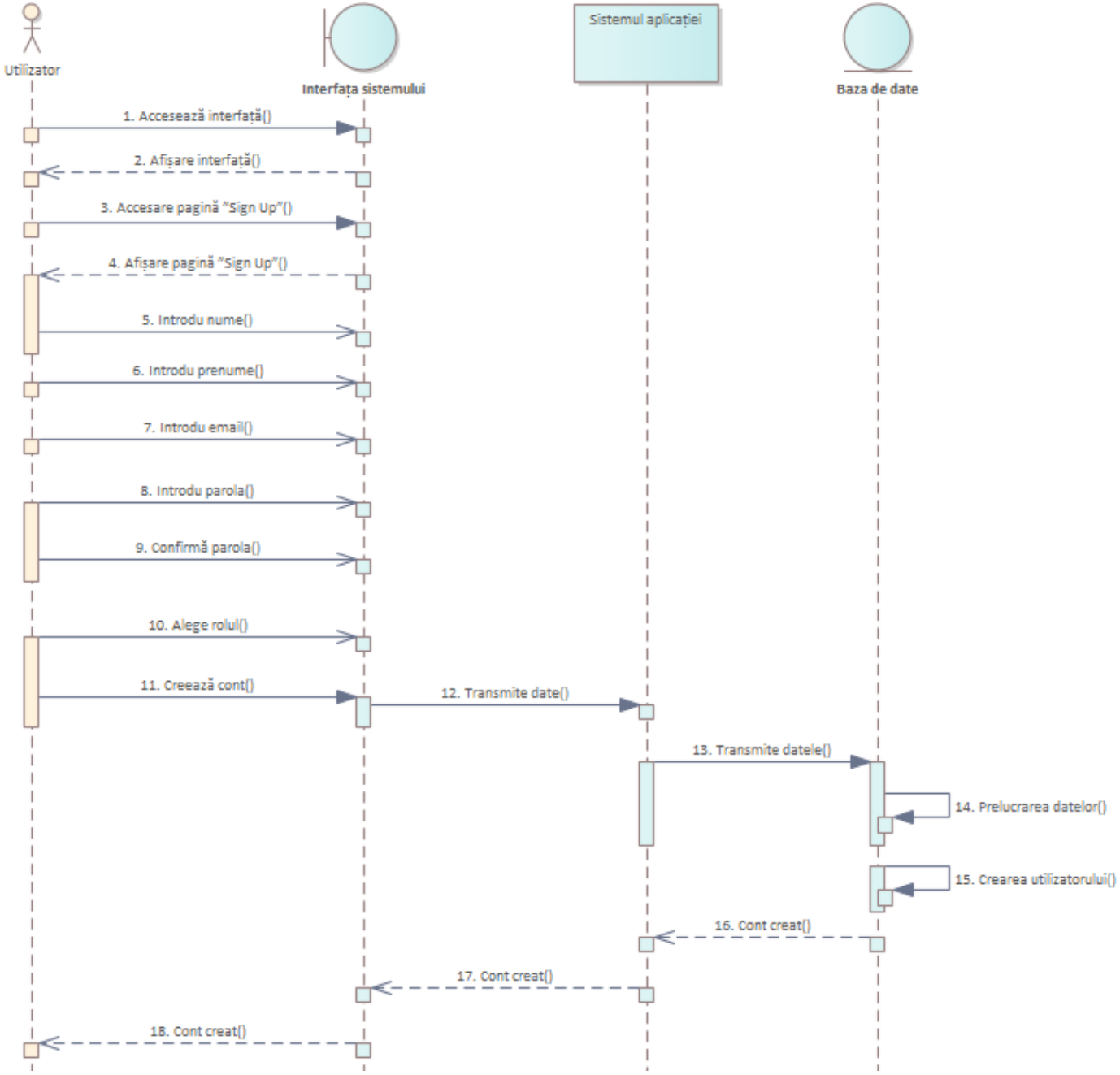

Figura A.1 – Scenariul de înregistrare a utilizatorului în aplicație

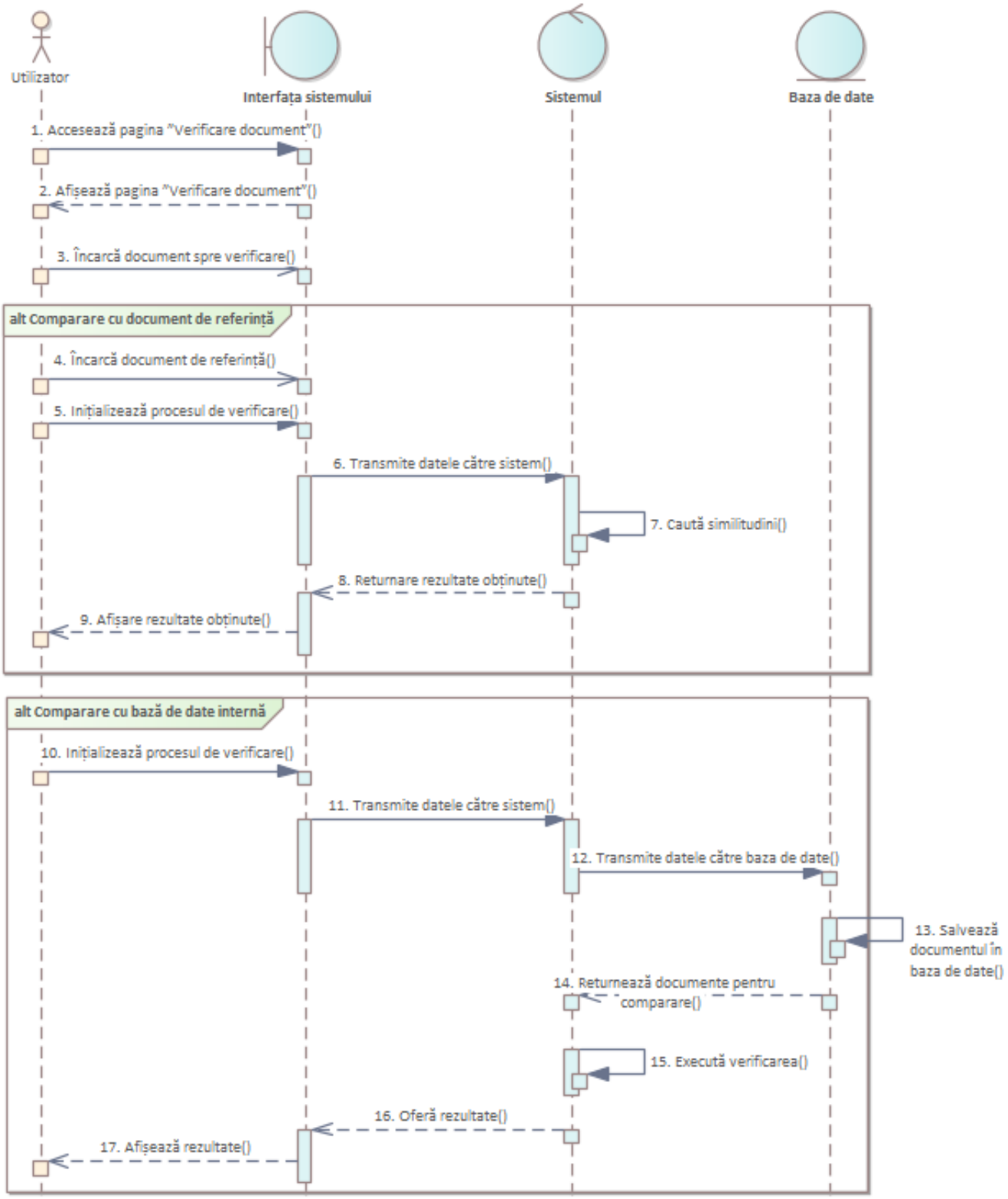

Figura A.2 – Scenariul de verificare a similitudinii documentului

ANEXA B

Proiectarea și crearea bazei de date

Figura B.1 – Structura bazei de date

ANEXA C

Serviciul pentru procesarea documentelor

```
using System.Linq;
using Microsoft.EntityFrameworkCore;
using SimilarityChecker.Api.Data;
using SimilarityChecker.Api.Data.Entities;
using SimilarityChecker.Api.Models;
using SimilarityChecker.Shared.Dto;
using System.Text.Json;
using System.Text.RegularExpressions;

namespace SimilarityChecker.Api.Services.InternalScan;

public sealed class InternalScanService : IInternalScanService
{
    private readonly SimilarityCheckerDbContext _db;

    public InternalScanService(SimilarityCheckerDbContext db) => _db = db;

    public async Task<InternalScanStartResponse> StartAsync(Guid documentId, double threshold,
CancellationToken ct)
    {
        var doc = await _db.Documents.FirstOrDefaultAsync(d => d.Id == documentId, ct);
        if (doc is null)
            throw new InvalidOperationException("Documentul nu există în baza de date.");

        var others = await _db.Documents
            .Where(d => d.Id != documentId)
            .Select(d => new { d.Id, d.FileName, d.ExtractedText, d.Sha256 })
            .ToListAsync(ct);

        // recomandat: curăță match-urile vechi pentru acest document
        var oldInternal = _db.InternalMatches.Where(m => m.DocumentId == documentId);
        _db.InternalMatches.RemoveRange(oldInternal);

        var hits = new List<InternalScanHitDto>();
        var corpusTexts = others
            .Select(x => x.ExtractedText)
            .Where(x => !string.IsNullOrWhiteSpace(x))
            .ToList();

        foreach (var other in others)
        {
            double score;

            // 1) duplicat exact (hash)
            if (!string.IsNullOrWhiteSpace(doc.Sha256) && doc.Sha256 == other.Sha256)
            {
                score = 1.0;
            }
            else
            {
                var source = doc.ExtractedText ?? "";
                var target = other.ExtractedText ?? "";

                if (string.IsNullOrWhiteSpace(source) || string.IsNullOrWhiteSpace(target))
                    continue;

                score = InternalSimilarityEngine.ComputeSimilarity(source, target,
corpusTexts);
            }

            if (score >= threshold)
            {
                hits.Add(new InternalScanHitDto
                {
                    ComparedDocumentId = other.Id,
                    ComparedFileName = other.FileName,
                    SimilarityScore = score
```

```
                });

                _db.InternalMatches.Add(new InternalMatchEntity
                {
                    DocumentId = doc.Id,
                    ComparedDocumentId = other.Id,
                    Score = score,
                    AlgorithmVersion = "internal-v2"
                });
            }
        }

        hits = hits.OrderByDescending(x => x.SimilarityScore).ToList();

        // ===== NOU: Fragmente "Exact" pentru top K rezultate =====
        const int topK = 5;
        var topRefs = hits.Take(topK).Select(h => h.ComparedDocumentId).ToHashSet();

        var refDocs = await _db.Documents
            .Where(d => topRefs.Contains(d.Id))
            .Select(d => new { d.Id, d.FileName, d.ExtractedText })
            .ToListAsync(ct);

        var fragments = new List<InternalScanFragmentDto>();

        foreach (var refDoc in refDocs)
        {
            var exacts = InternalSimilarityEngine.FindExactFragments(
                doc.ExtractedText,
                refDoc.ExtractedText,
                minConsecutiveShingles: 3);

            foreach (var ex in exacts)
            {
                fragments.Add(new InternalScanFragmentDto
                {
                    Type = FragmentTypeDto.Exact,
                    Score = ex.Score,
                    SourceTokenStart = ex.SourceTokenStart,
                    SourceTokenEnd = ex.SourceTokenEnd,
                    RefTokenStart = ex.RefTokenStart,
                    RefTokenEnd = ex.RefTokenEnd,
                    ReferenceDocumentId = refDoc.Id,
                    ReferenceFileName = refDoc.FileName,
                    SourceSnippet = ex.SourceSnippet,
                    ReferenceSnippet = ex.ReferenceSnippet
                });
            }

            var paraphrases = InternalSimilarityEngine.FindParaphraseFragments(
            doc.ExtractedText,
            refDoc.ExtractedText,
            minCombinedScore: 0.38,
            maxExactSimilarity: 0.90,
            minSharedContentWords: 2);

            foreach (var para in paraphrases)
            {
                bool overlapsExact = fragments.Any(f =>
                    f.Type == FragmentTypeDto.Exact &&
                    f.ReferenceDocumentId == refDoc.Id &&
                    RangesOverlap(f.SourceTokenStart, f.SourceTokenEnd, para.SourceTokenStart,
para.SourceTokenEnd));

                if (overlapsExact)
                    continue;

                fragments.Add(new InternalScanFragmentDto
                {
                    Type = FragmentTypeDto.Paraphrase,
                    Score = para.Score,
```

```
            SourceTokenStart = para.SourceTokenStart,
            SourceTokenEnd = para.SourceTokenEnd,
            RefTokenStart = para.RefTokenStart,
            RefTokenEnd = para.RefTokenEnd,
            ReferenceDocumentId = refDoc.Id,
            ReferenceFileName = refDoc.FileName,
            SourceSnippet = para.SourceSnippet,
            ReferenceSnippet = para.ReferenceSnippet
        });
    }
}

// ===== NOU: Procente tri-color =====
var sourceTokenCount = CountTokens(doc.ExtractedText);

var exactCovered = EstimateCoveredTokens(
    fragments.Where(f => f.Type == FragmentTypeDto.Exact));

var paraphraseCovered = EstimateCoveredTokens(
    fragments.Where(f => f.Type == FragmentTypeDto.Paraphrase));

int exactPercent = 0;
if (sourceTokenCount > 0 && exactCovered > 0)
{
    exactPercent = (int)Math.Round(100.0 * exactCovered / sourceTokenCount);
    if (exactPercent == 0)
        exactPercent = 1;
}

int paraphrasePercent = 0;
if (sourceTokenCount > 0 && paraphraseCovered > 0)
{
    paraphrasePercent = (int)Math.Round(100.0 * paraphraseCovered / sourceTokenCount);
    if (paraphrasePercent == 0)
        paraphrasePercent = 1;
}

if (exactPercent + paraphrasePercent > 100)
{
    paraphrasePercent = Math.Max(0, 100 - exactPercent);
}

int cleanPercent = Math.Max(0, 100 - exactPercent - paraphrasePercent);

var reportDto = new InternalScanReportDto
{
    ReportId = Guid.Empty, // completăm după SaveChanges
    DocumentId = doc.Id,
    GeneratedAtUtc = DateTime.UtcNow,
    Hits = hits,
    Fragments = fragments,
    ExactPercent = exactPercent,
    ParaphrasePercent = paraphrasePercent,
    CleanPercent = cleanPercent
};

var report = new ReportEntity
{
    DocumentId = doc.Id,
    UserId = doc.UserId,
    GeneratedAtUtc = DateTime.UtcNow,
    Status = "Completed",
    ReportJson = JsonSerializer.Serialize(reportDto)
};

_db.Reports.Add(report);
await _db.SaveChangesAsync(ct);

// update ReportId în JSON
reportDto.ReportId = report.Id;
report.ReportJson = JsonSerializer.Serialize(reportDto);
```

```
        await _db.SaveChangesAsync(ct);

        return new InternalScanStartResponse
        {
            ReportId = report.Id,
            DocumentId = doc.Id,
            ComparedDocuments = others.Count
        };
    }

    public async Task<InternalScanReportDto?> GetReportAsync(Guid documentId,
CancellationToken ct)
    {
        var report = await _db.Reports
            .Where(r => r.DocumentId == documentId)
            .OrderByDescending(r => r.GeneratedAtUtc)
            .FirstOrDefaultAsync(ct);

        if (report is null) return null;

        var dto = JsonSerializer.Deserialize<InternalScanReportDto>(report.ReportJson);
        if (dto is null) return null;

        dto.ReportId = report.Id;
        return dto;
    }

    private static int CountTokens(string text)
    {
        if (string.IsNullOrWhiteSpace(text)) return 0;
        var cleaned = Regex.Replace(text.ToLowerInvariant(), @"[^\p{L}\p{N}\s]+", " ");
        return Regex.Matches(cleaned, @"\S+").Count;
    }

    private static int EstimateCoveredTokens(IEnumerable<InternalScanFragmentDto> frags)
    {
        var intervals = frags
            .Select(f => (s: f.SourceTokenStart, e: f.SourceTokenEnd))
            .Where(x => x.e > x.s)
            .OrderBy(x => x.s)
            .ToList();

        if (intervals.Count == 0) return 0;

        int covered = 0;
        int curS = intervals[0].s, curE = intervals[0].e;

        for (int i = 1; i < intervals.Count; i++)
        {
            var (s, e) = intervals[i];
            if (s <= curE) curE = Math.Max(curE, e);
            else
            {
                covered += (curE - curS);
                curS = s;
                curE = e;
            }
        }

        covered += (curE - curS);
        return covered;
    }
    private static bool RangesOverlap(int aStart, int aEnd, int bStart, int bEnd)
    {
        return aStart < bEnd && bStart < aEnd;
    }

    public async Task<InternalScanReportDto> CompareTwoDocumentsAsync(
    Guid primaryDocumentId,
    Guid referenceDocumentId,
    double threshold,
```

```
    CancellationToken ct = default)
    {
        if (primaryDocumentId == referenceDocumentId)
            throw new InvalidOperationException("Documentul analizat și documentul de
referință nu pot fi identice.");

        var primaryDocument = await _db.Documents
            .FirstOrDefaultAsync(x => x.Id == primaryDocumentId, ct);

        if (primaryDocument is null)
            throw new InvalidOperationException("Documentul analizat nu a fost găsit.");

        var referenceDocument = await _db.Documents
            .FirstOrDefaultAsync(x => x.Id == referenceDocumentId, ct);

        if (referenceDocument is null)
            throw new InvalidOperationException("Documentul de referință nu a fost găsit.");

        var primaryText = primaryDocument.ExtractedText ?? "";
        var referenceText = referenceDocument.ExtractedText ?? "";
        var corpusTexts = await _db.Documents
            .Where(d => !string.IsNullOrWhiteSpace(d.ExtractedText))
            .Select(d => d.ExtractedText)
            .ToListAsync(ct);

        if (string.IsNullOrWhiteSpace(primaryText))
            throw new InvalidOperationException("Documentul analizat nu conține text
procesabil.");

        if (string.IsNullOrWhiteSpace(referenceText))
            throw new InvalidOperationException("Documentul de referință nu conține text
procesabil.");

        double similarityScore;

        if (!string.IsNullOrWhiteSpace(primaryDocument.Sha256) &&
            primaryDocument.Sha256 == referenceDocument.Sha256)
        {
            similarityScore = 1.0;
        }
        else
        {
            similarityScore = InternalSimilarityEngine.ComputeSimilarity(primaryText,
referenceText, corpusTexts);
        }

        var fragments = new List<InternalScanFragmentDto>();

        var exacts = InternalSimilarityEngine.FindExactFragments(
            primaryText,
            referenceText,
            minConsecutiveShingles: 3);


        foreach (var ex in exacts)
        {
            fragments.Add(new InternalScanFragmentDto
            {
                Type = FragmentTypeDto.Exact,
                Score = ex.Score,
                SourceTokenStart = ex.SourceTokenStart,
                SourceTokenEnd = ex.SourceTokenEnd,
                RefTokenStart = ex.RefTokenStart,
                RefTokenEnd = ex.RefTokenEnd,
                ReferenceDocumentId = referenceDocument.Id,
                ReferenceFileName = referenceDocument.FileName,
                SourceSnippet = ex.SourceSnippet,
                ReferenceSnippet = ex.ReferenceSnippet
            });
        }
```

```
        var paraphrases = InternalSimilarityEngine.FindParaphraseFragments(
        primaryText,
        referenceText,
        minCombinedScore: 0.38,
        maxExactSimilarity: 0.90,
        minSharedContentWords: 2);

        foreach (var para in paraphrases)
        {
            bool overlapsExact = fragments.Any(f =>
                f.Type == FragmentTypeDto.Exact &&
                RangesOverlap(f.SourceTokenStart, f.SourceTokenEnd, para.SourceTokenStart,
para.SourceTokenEnd));

            if (overlapsExact)
                continue;

            fragments.Add(new InternalScanFragmentDto
            {
                Type = FragmentTypeDto.Paraphrase,
                Score = para.Score,
                SourceTokenStart = para.SourceTokenStart,
                SourceTokenEnd = para.SourceTokenEnd,
                RefTokenStart = para.RefTokenStart,
                RefTokenEnd = para.RefTokenEnd,
                ReferenceDocumentId = referenceDocument.Id,
                ReferenceFileName = referenceDocument.FileName,
                SourceSnippet = para.SourceSnippet,
                ReferenceSnippet = para.ReferenceSnippet
            });
        }

        var sourceTokenCount = CountTokens(primaryText);

        var exactCovered = EstimateCoveredTokens(
            fragments.Where(f => f.Type == FragmentTypeDto.Exact));

        var paraphraseCovered = EstimateCoveredTokens(
            fragments.Where(f => f.Type == FragmentTypeDto.Paraphrase));

        int exactPercent = 0;
        if (sourceTokenCount > 0 && exactCovered > 0)
        {
            exactPercent = (int)Math.Round(100.0 * exactCovered / sourceTokenCount);
            if (exactPercent == 0)
                exactPercent = 1;
        }

        int paraphrasePercent = 0;
        if (sourceTokenCount > 0 && paraphraseCovered > 0)
        {
            paraphrasePercent = (int)Math.Round(100.0 * paraphraseCovered / sourceTokenCount);
            if (paraphrasePercent == 0)
                paraphrasePercent = 1;
        }

        if (exactPercent + paraphrasePercent > 100)
        {
            paraphrasePercent = Math.Max(0, 100 - exactPercent);
        }

        int cleanPercent = Math.Max(0, 100 - exactPercent - paraphrasePercent);

        var report = new InternalScanReportDto
        {
            ReportId = Guid.Empty,
            DocumentId = primaryDocumentId,
            GeneratedAtUtc = DateTime.UtcNow,
            ExactPercent = exactPercent,
            ParaphrasePercent = paraphrasePercent,
            CleanPercent = cleanPercent,
```

```
            Hits = similarityScore >= threshold
                ? new List<InternalScanHitDto>
                {
                new InternalScanHitDto
                {
                    ComparedDocumentId = referenceDocument.Id,
                    ComparedFileName = referenceDocument.FileName,
                    SimilarityScore = similarityScore
                }
                }
                : new List<InternalScanHitDto>(),
            Fragments = fragments
        };

        return report;
    }
}
```

ANEXA D

Raportul generat în urma scanării documentului

Raport de analiză a similitudinii

Scanare internă

Document analizat: 75d5ee22-917e-4e36-a9be-71b0fa0bb7f6

Generat: 2026-03-20 08:20:34 UTC

Raport ID: 41943195-e3f7-4a79-a34b-256610e471da

1. Rezumat

Rezultatul final

Conținut suspect total: 47%

Nivel estimat: Nivel mediu

Documente comparate: 6

Distribuția conținutului

Copiat exact: 47%

Parafrazat: 0%

Conținut original estimat: 53%

Vizual

Legendă

- Copiat exact
- Parafrazat
- Original estimat

2. Documente comparate

Document de referință	Scor comparativ
Fragment 1.docx	66%
Lucrul_Individual_PSI.docx	29%
Lucrul_Individual_PSI.pdf	29%
Raport_Practică_Botnarenco_Ecaterina_TI-223.docx	28%
Raport_Practică_Botnarenco_Ecaterina_TI-223.pdf	28%
Laboratorul2_PSI_Botnarenco Ecaterina.docx	28%

3. Fragmente identificate

Sursă: Fragment 1.docx

Copiat exact • 100%

Fragment din documentul analizat:

sistemul informatic propus are rolul de a analiza documentele încărcate de utilizator și de a identifica automat gradul de similitudine dintre acestea și documentele existente în baza de date procesul include prelucrarea textului eliminarea elementelor nerelevante și calcularea scorurilor de asemănare pentru generarea raportului final

Fragment din documentul de referință:

sistemul informatic propus are rolul de a analiza documentele încărcate de utilizator și de a identifica automat gradul de similitudine dintre acestea și documentele existente în baza de date procesul include prelucrarea textului eliminarea elementelor nerelevante și calcularea scorurilor de asemănare pentru generarea raportului final

*Se afișează maxim 30 fragmente.

SimilarityChecker • Raport generat automat

Pagina 1 / 1

Figura D.1 – Pagina raportului generat